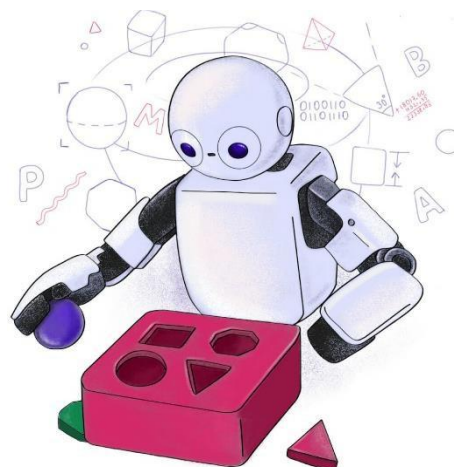


TP555 - Inteligência Artificial e Machine Learning: *Regressão Linear*



Esse material foi desenvolvido e gentilmente cedido pelo Prof. Dr. Felipe Augusto Pereira de Figueiredo, do Inatel. (felipe.figueiredo@inatel.br)

Inatel

Prof. Dr. Luiz Augusto Melo Pereira
luiz.melo@inatel.br

Motivação

- **Exemplo 1:** Estimar o preço de casas.

5 quartos, 3 vagas na garagem, 1 piscina, cond. de luxo, 500 m^2



R\$ 1.000.000,00

1 quarto, sem vaga na garagem, bairro afastado, 70 m^2



R\$ 200.000,00

2 quartos, 1 vaga na garagem, centro, 200 m^2

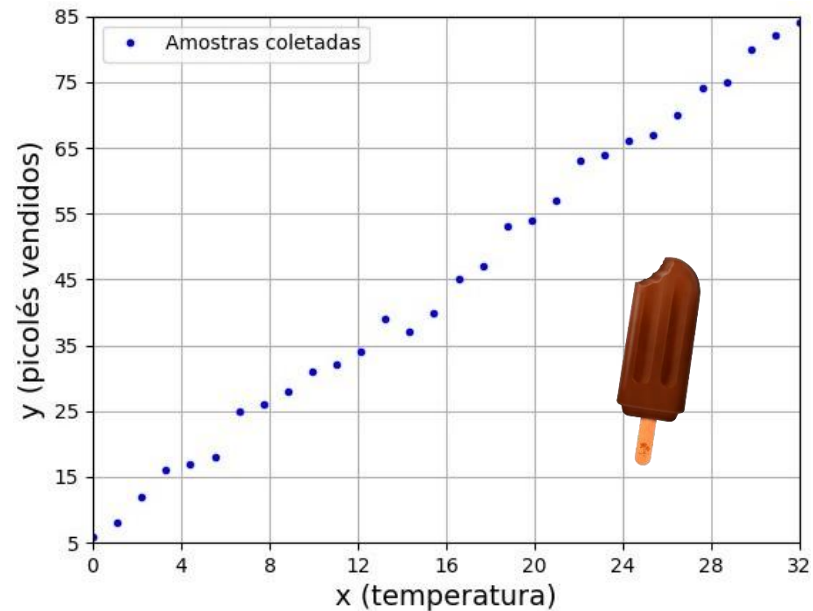


???

- Podemos usar **regressão** para encontrar uma **relação matemática**, $h(x)$, entre o n° de quartos, n° de vagas na garagem, piscina, localização, área, etc. de uma casa e seu valor.
- **Objetivo:** agilizar o processo de avaliação de propriedades.

Motivação

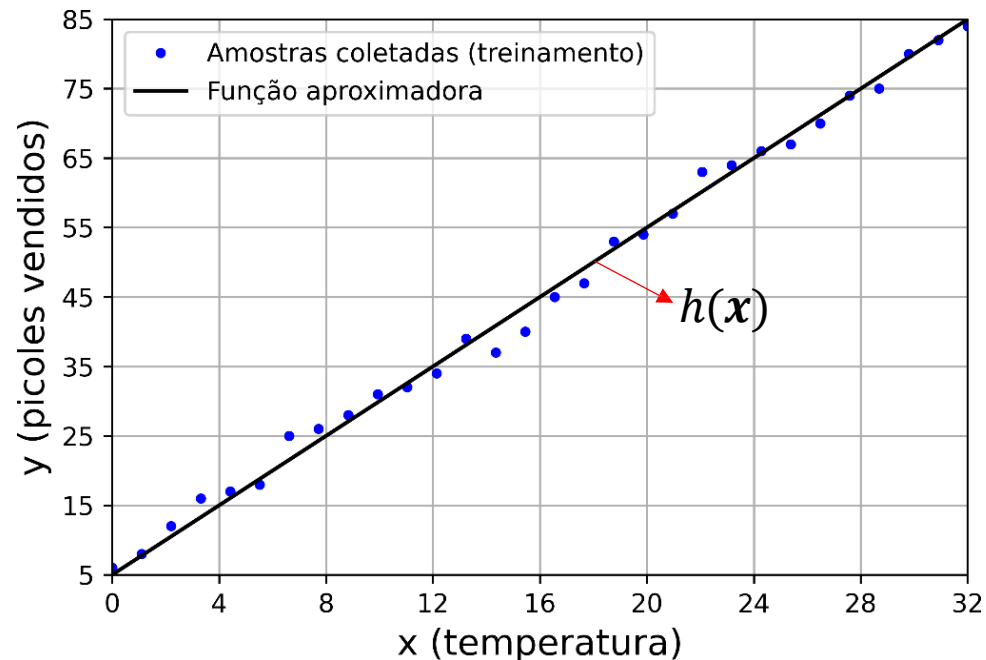
- **Exemplo 2:** Estimar as vendas de picolés.



- Podemos usar regressão para encontrar um **mapeamento**, $h(x)$, entre a temperatura média de um dia e a quantidade de picolés vendidos.
- **Objetivo:** reduzir o desperdício e aumentar os lucros.

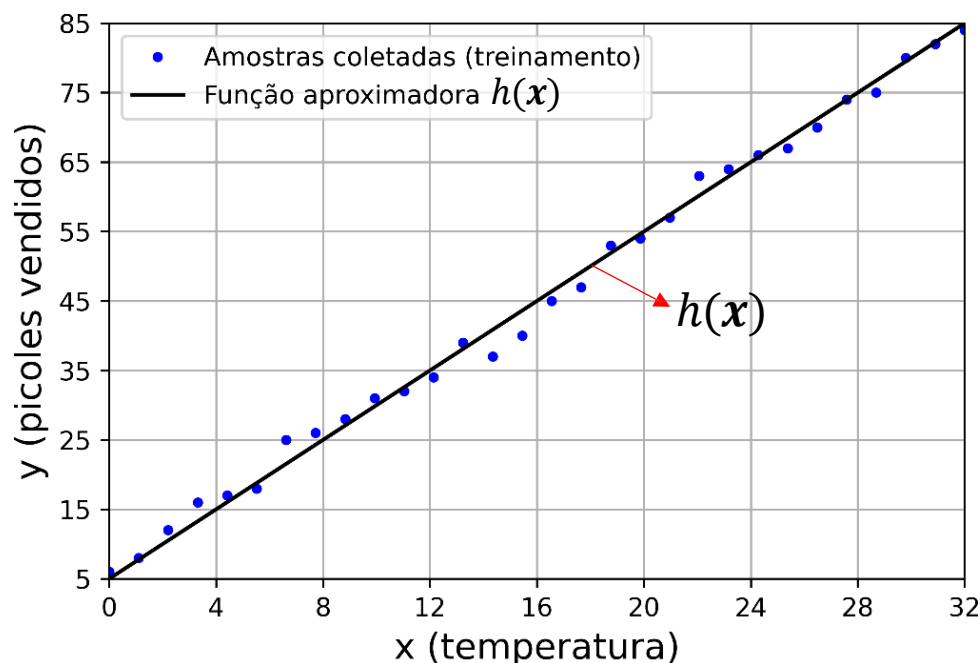
*Como podemos resolver esses problemas
de estimação sem conhecer as regras de
mapeamento?*

Regressão linear



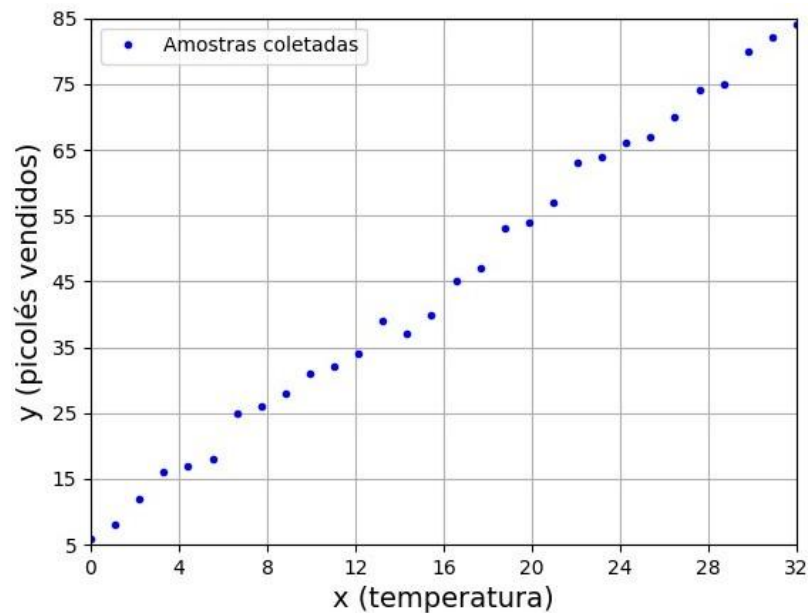
- É um dos mais simples, antigos e usados algoritmos de aprendizado de máquina.
- Regressão linear também é conhecida como ***aproximação de funções*** ou ***ajuste de curvas***.
 - Por exemplo, ajustar uma curva (i.e., uma função) aos dados de vendas de picolés.
- Vai nos dar ***intuições*** importantes para o ***entendimento*** de ***algoritmos mais complexos***, como ***classificadores*** e ***redes neurais***.

Qual o objetivo da regressão linear?



- **Objetivo:** encontrar uma função, $h(x)$, que **mapeie** as entradas, x , em uma variável de saída $\hat{y} = h(x)$, de tal forma que $h(x)$ se ajuste aos dados coletados de **forma ótima**.
 - Ótimo no sentido da **minimização da diferença** entre as previsões feitas por $h(x)$, (i.e., os valores de $\hat{y}$), e os valores esperados de saída, y .
- No contexto da regressão linear, nos chamamos as entradas, x , de **atributos** e os valores esperados de saída, y , de **rótulos**.
 - Vários pares de x e y formam o **conjunto de treinamento**.

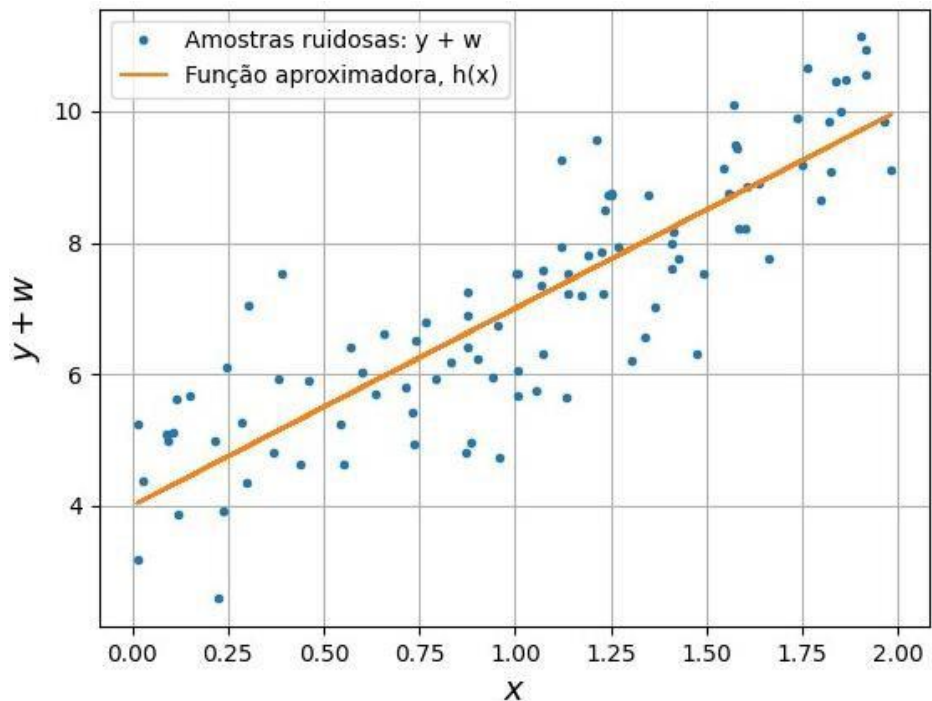
Relação matemática real entre x e y



Não existe realmente uma função por trás desse comportamento, mas podemos encontrar uma função que o aproxime usando regressão.

- Na grande maioria dos casos, **não existe uma relação matemática**, i.e., uma função, que realmente **descreve a relação entre as entradas, x , e as saídas esperadas, y** , dos dados em um conjunto.
- Porém, mesmo nestes casos, ainda podemos modelar a relação entre x e y da melhor forma possível usando regressão.
- Nos casos onde essa função existe, a chamamos de **função verdadeira** ou **objetivo**, a qual é, normalmente, denotada por $f(x)$.

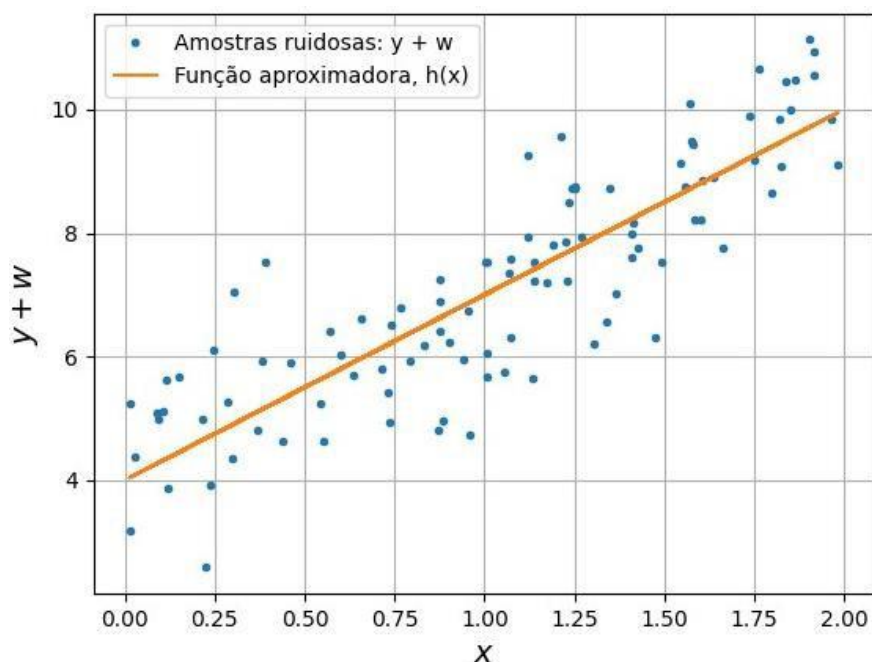
Dados ruidosos



OBS.: w representa o ruído contaminando os dados.

- Em geral, em problemas de regressão, lidamos com ***dados que estão sujeitos a ruído***.
- Exemplos de introdução de ruído aos dados:
 - **Previsão de preços de imóveis:** os preços de imóveis podem ser influenciados por mudanças na taxa de juros, inflação, preferências do comprador, valor sentimental, etc.
 - **Previsão de preços de ações:** os preços de ações podem ser influenciados por notícias, mudanças econômicas, eventos globais, etc.
 - **Deteção de símbolos:** os símbolos transmitidos passam através de um canal ruidoso.
- Nos nossos exemplos, modelaremos o ruído como uma variável aleatória Gaussiana.

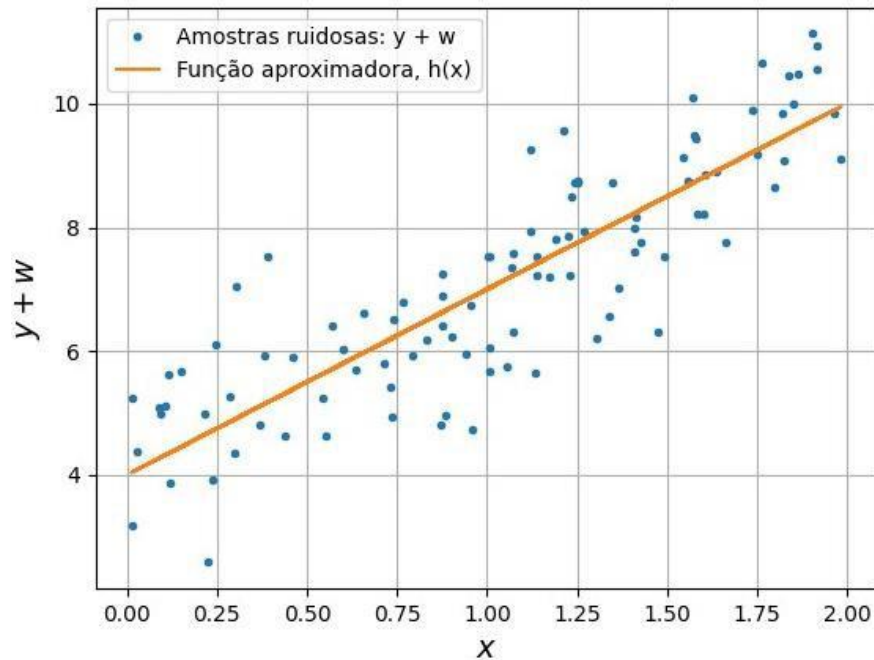
O objetivo geral da regressão linear



OBS.: w representa o ruído contaminando os dados.

- Assim, o **objetivo geral** da regressão linear é encontrar uma função $h(x)$ que descreva a relação entre x e y , **mesmo que os dados coletados estejam corrompidos por ruído**, i.e., $y + w$.
- Em outras palavras, a regressão busca **capturar** um **padrão geral por trás dos dados, mesmo que ruidosos, de forma a se obter uma boa capacidade de generalização**.
- Esse conjunto de **valores coletados**, ou **experiências prévias**, será chamado de **conjunto de treinamento**.

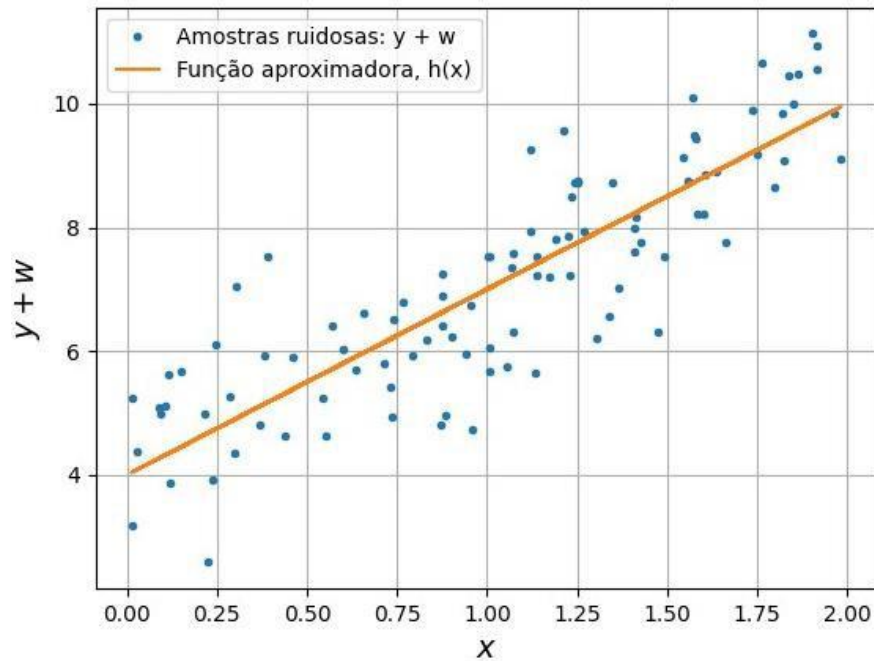
Qual é o paradigma de aprendizado?



OBS.: w representa o ruído contaminando os dados.

- Dado que possuímos um **conjunto** de valores de entrada, x (os **atributos**), e os seus respectivos valores esperados de saída, y (os **rótulos**), **que tipo de aprendizado é realizado pela regressão linear?**

Qual é o paradigma de aprendizado?



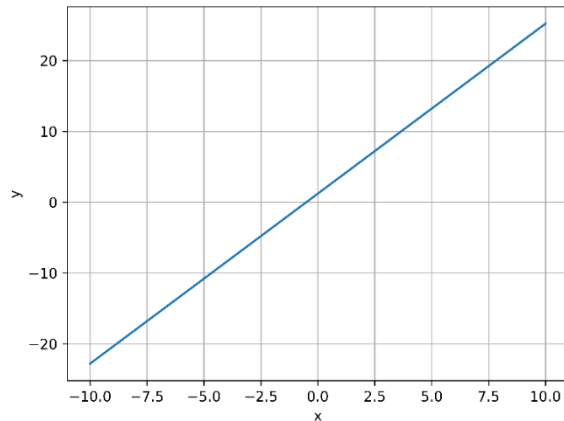
OBS.: w representa o ruído contaminando os dados.

- Dado que possuímos um **conjunto** de valores de entrada, x (os **atributos**), e os seus respectivos valores esperados de saída, y (os **rótulos**), **que tipo de aprendizado é realizado pela regressão linear?**

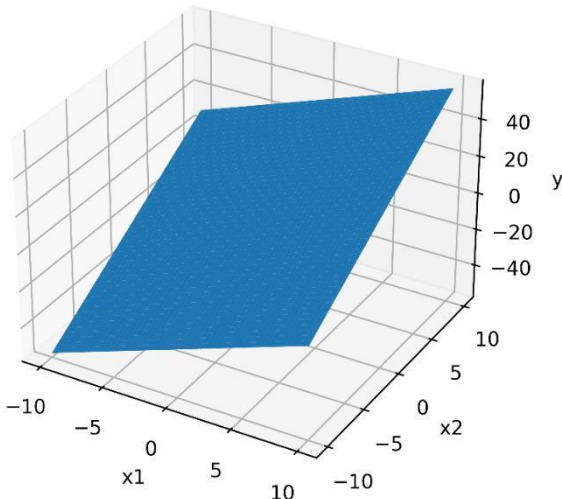
Aprendizado supervisionado

Por que linear?

$$h(x_1) = a_0 + a_1x_1 \text{ (reta)}$$



$$h(\mathbf{x}) = a_0 + a_1x_1 + a_2x_2 \text{ (plano)}$$



- A **regressão** é chamada de **linear** porque a saída da **função aproximadora**, $\hat{y} = h(\mathbf{x})$, é modelada como sendo uma **combinação linear dos atributos**, $\mathbf{x}$.
- Ou seja, usamos uma **função linear** para modelar o relacionamento entre $\mathbf{x}$ e y .
- Por exemplo, com **um atributo**, x_1 , temos a equação de uma **reta**:

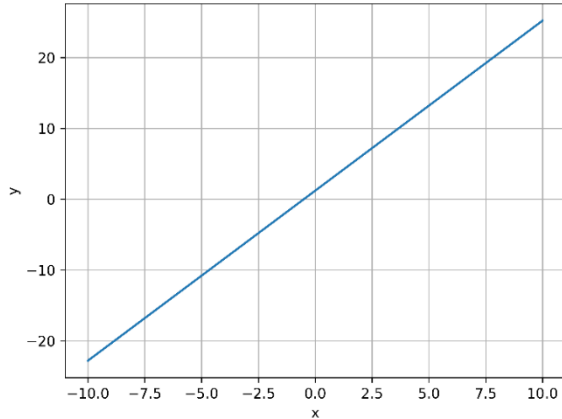
$$\hat{y} = h(\mathbf{x}) = a_0 + a_1x_1$$

- Já com **dois atributos**, x_1 e x_2 , temos a equação de um **plano**:

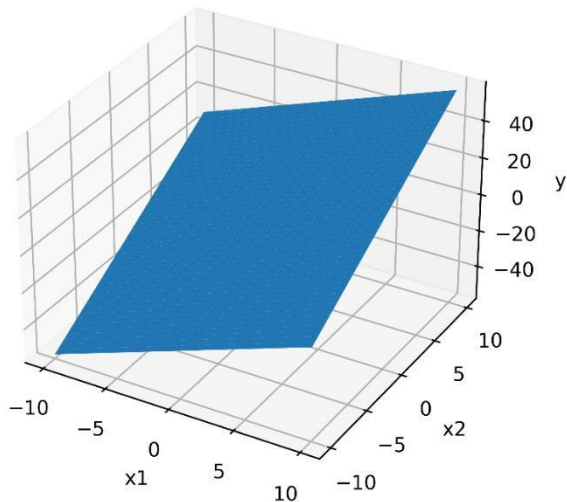
$$\hat{y} = h(\mathbf{x}) = a_0 + a_1x_1 + a_2x_2$$

Por que linear?

$$h(x_1) = a_0 + a_1x_1 \text{ (reta)}$$



$$h(\mathbf{x}) = a_0 + a_1x_1 + a_2x_2 \text{ (plano)}$$



- Uma **generalização** para qualquer **número de atributos**, K , é dada pela equação do **hiperplano**:

$$\hat{y} = h(\mathbf{x}) = a_0 + a_1x_1 + \cdots + a_Kx_K = a_0 + \sum_{i=1}^{\overset{\text{Número de atributos}}{K}} a_ix_i$$

- O **hiperplano** será o **primeiro formato** de **função aproximadora** (também, chamado de **modelo**) que iremos utilizar.
- O **modelo** é definido pela **equação** e pelos **pesos**, $a_i, \forall i$.
- Existem outros formatos de funções (i.e., modelos), como os **polinômios**, os quais veremos mais adiante e que modelarão relações não lineares entre $x_k, \forall k$ e $\hat{y}$

Por que linear?

Observação

- A palavra **linear** no contexto da regressão significa “**linear com relação aos pesos**” e não com relação aos **atributos**, x .
- Desta forma, os seguintes **modelos** também são lineares com relação aos pesos:

$$\left. \begin{array}{l} \checkmark \hat{y} = h(\mathbf{x}) = a_0 + a_1 \log x_1 + a_2 \cos x_2 \\ \checkmark \hat{y} = h(\mathbf{x}) = a_0 + a_1 e^{x_1} \\ \checkmark \hat{y} = h(\mathbf{x}) = a_0 + a_1 x_1 + a_2 x_1^2 + a_3 x_1^3 \end{array} \right\}$$

Mesmo não tendo um mapeamento linear das entradas na saída, continuamos tendo uma combinação linear dos atributos em relação aos pesos.

- Exemplo de um modelo não-linear: $\hat{y} = h(\mathbf{x}) = \frac{a_0 x_1}{a_1 + x_1}$.

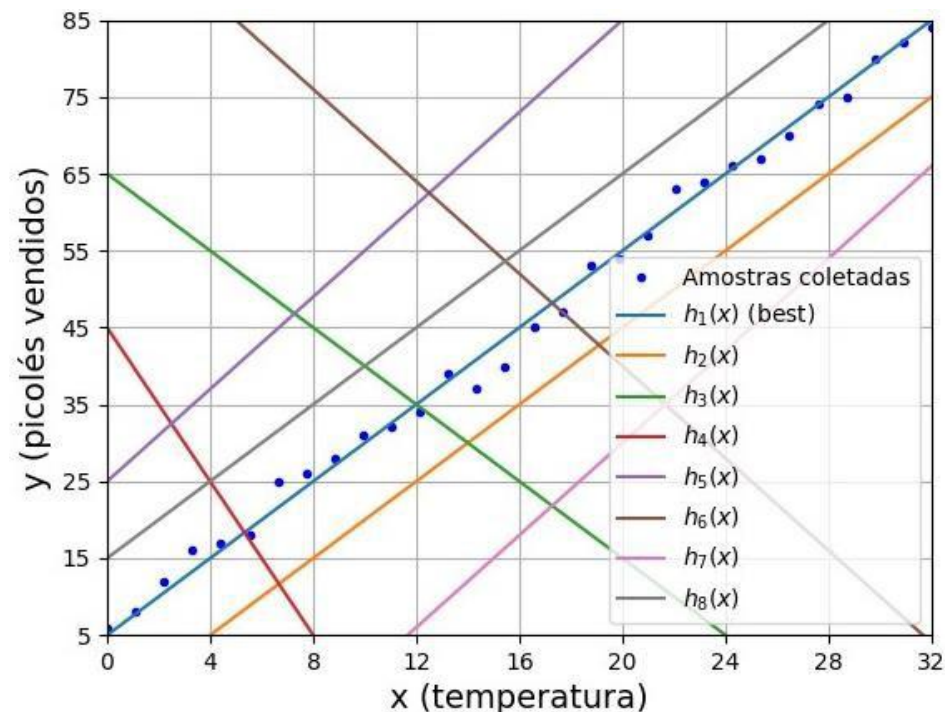
Não é possível expressar a equação como uma combinação linear dos atributos.

- Em geral, não é possível aplicar algumas das técnicas de regressão que veremos em breve a modelos não-lineares.

Função hipótese

- No contexto da regressão, a **função aproximadora**, $h(x)$, é chamada de **função hipótese**, pois refere-se à **explicação proposta para a relação entre as entradas, x e a saída esperada, y** .
 - Por exemplo, anteriormente, **hipotetizamos** que uma **reta** explicaria bem a relação entre a temperatura média de um dia e o número de picolés vendidos.
- O conjunto de todas as possíveis **funções hipótese** é chamado de **espaço de hipóteses, H** .
- Em outras palavras, é o **espaço** criado por **todas as combinações possíveis de atributos e valores dos pesos**.
- A solução que **melhor mapeia** x em y encontra-se dentro do **espaço de hipóteses** e nosso objetivo é encontra-la.

Espaço de hipóteses



A figura mostra algumas possíveis funções hipótese que modelam o relacionamento entre x e y .

- No exemplo da previsão da quantidade de picolés vendidos, onde usamos a **equação de uma reta** como **função hipótese**,

$$\hat{y} = h(\mathbf{x}) = a_0 + a_1 x_1,$$

o **espaço de hipóteses contém todas as possíveis retas** (restringimos o espaço) que podem ser usadas para mapear os dados.

- Porém, claro, **apenas uma é a melhor segundo o critério de minimização da diferença entre a saída do modelo e o valor esperado**.

Refinando o objetivo da regressão linear

- Agora que temos um formato definido para $h(x)$, ou seja, o **modelo**, podemos refinar o objetivo da regressão um pouco mais.
- **Objetivo:** **Encontrar** os **pesos**, $a_0, a_1, \dots, a_K$, que façam com que $h(x)$ se **ajuste de forma ótima** ao conjunto de dados coletado.
 - **Ótima** no sentido em que $h(x)$ minimiza o erro (i.e., diferença) entre os valores de saída do modelo, $\hat{y}$ e os esperados, y .
- Na sequência, definiremos uma **função de erro** que nos ajudará a encontrar os **pesos ótimos** do modelo.
 - **Pesos ótimos:** conjunto de pesos da função hipótese, $h(x)$, que fazem com que o erro seja minimizado.

Definição formal do problema

Informação disponível

- Conjunto de N observações ou ***pares de treinamento***: $\{x(i), y(i)\}$, $i = 0, \dots, N - 1$, onde
 - $x(i) = [1, x_1(i), \dots, x_K(i)]^T \in \mathbb{R}^{K+1 \times 1}$ é o ***vetor de atributos*** (vetor coluna) com dimensão $(K + 1 \times 1)$ contendo os i -ésimos valores dos ***atributos***.
 - $y(i) \in \mathbb{R}$ é o i -ésimo ***rótulo*** (valor esperado de saída) referente ao vetor de atributos, $x(i)$.
- As N observações formam o ***conjunto de treinamento***, ou seja, o conjunto de amostras coletadas que utilizaremos para ***treinar o modelo***.
- Notem que o ***vetor de atributos*** tem sempre um elemento a mais do que o número real de atributos, K .
- Como veremos, esse elemento, o primeiro do vetor, tem valor constante igual a 1 para que possamos representar o ***modelo*** em ***formato vetorial***.

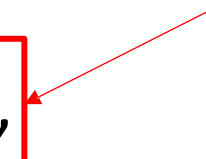
Definição formal do problema

Modelo

- Função do hiperplano

$$\hat{y}(i) = h(\mathbf{x}(i)) = a_0 + \sum_{i=1}^K a_i x_i = \mathbf{a}^T \mathbf{x}(i),$$

Representação vetorial



onde $\mathbf{a} = [a_0, \dots, a_K]^T$ é um vetor **coluna** com dimensão $(K + 1 \times 1)$ contendo os **pesos** da **função hipótese**.

Observações

- a_0 é o **coeficiente linear**, ou seja, é o valor de $h(\mathbf{x})$ que intercepta o eixo das ordenadas, y , a_0 é conhecido também como **intercept** ou **bias**.
- Como a_0 não tem um **atributo** relacionado a ele, para **facilitar a representação vetorial**, criamos um **atributo falso**, x_0 , chamado de **atributos de bias**, com valor constante igual a $x_0 = 1$.

Definição formal do problema

Objetivo

- Encontrar o **vetor de pesos**, $\mathbf{a}$, que **minimize o erro**, dado por uma **função de erro**, $J_e(\mathbf{a})$, entre o valor esperado, $y(i)$, e a aproximação, $\hat{y}(i)$, para **todos os exemplos do conjunto de treinamento**.
- Ou seja, o **treinamento** do modelo **envolve** a **minimização de uma função de erro**.
- Isso pode ser escrito como um problema de otimização

$$\min_{\mathbf{a} \in \mathbb{R}^{K+1 \times 1}} J_e(\mathbf{a})$$

- Portanto, para que consigamos treinar o modelo, precisamos **definir** uma **função de erro**.

Função de erro

- Existem várias possibilidades para se definir a **função de erro**.
- Porém, em problemas de regressão, é comum utilizar-se o **erro quadrático médio (EQM)**

$$J_e(\mathbf{a}) = \frac{1}{N} \sum_{i=0}^{N-1} (y(i) - \hat{y}(i))^2 = \frac{1}{N} \sum_{i=0}^{N-1} (y(i) - h(\mathbf{x}(i), \mathbf{a}))^2,$$

Erro entre a saída esperada e a saída da função hipótese.

onde N é o número de exemplos (ou observações).

- O **EQM** nada mais é do que a **média aritmética do quadrado dos erros**.
- Veremos mais adiante a razão pela qual o **EQM** é utilizado.
- O vetor de pesos foi evidenciado na função hipótese para mostrar que o **erro depende dos valores dos pesos**.
 - Variando os pesos, ficamos **mais próximos ou mais distantes** da função hipótese que minimiza o erro, i.e., da **função hipótese ótima**.
 - Assim, ao variar os pesos, estamos caminhando no espaço de hipóteses.

Função de erro

- A **função de erro** pode ser reescrita em **formato matricial** como

$$J_e(\mathbf{a}) = \frac{1}{N} \|\mathbf{y} - \mathbf{X}\mathbf{a}\|^2,$$

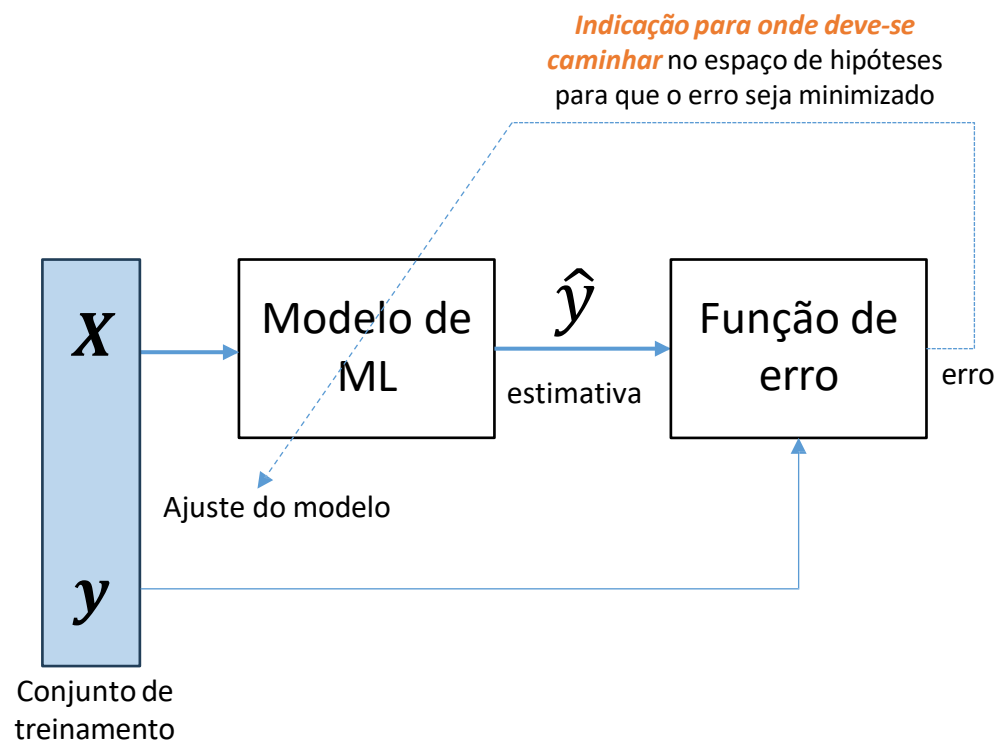
onde $\mathbf{y} = [y(0), \dots, y(N-1)]^T$ é um **vetor coluna** com dimensão $(N \times 1)$ contendo todos os N valores esperados e $\mathbf{X} = [\mathbf{x}(0), \dots, \mathbf{x}(N-1)]^T$ é uma **matriz** com dimensão $(N \times K + 1)$ contendo **todos os vetores de atributo**.

- Então, para encontrarmos o **vetor de pesos**, $\mathbf{a}$, devemos **minimizar a função de erro**

$$\min_{\mathbf{a} \in \mathbb{R}^{K+1 \times 1}} \|\mathbf{y} - \mathbf{X}\mathbf{a}\|^2.$$

OBS.: Por ser constante, o termo $1/N$ não influencia na minimização e, portanto, pode ser omitido na equação acima.

Resumo do problema da regressão

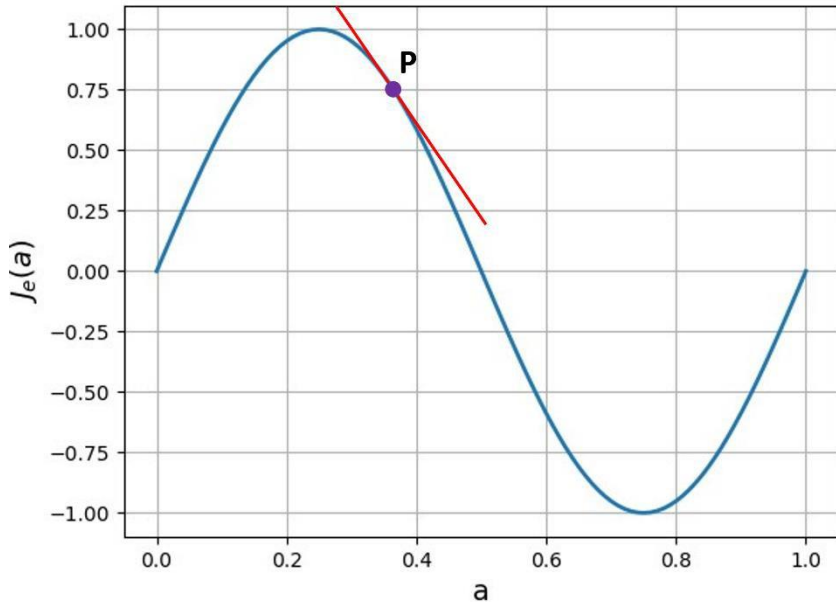


- Resumindo, nosso objetivo é **treinar** um **modelo**, a partir de um **conjunto de treinamento**, que **minimize o erro de aproximação**.
- Assim, para **treinar** o modelo, precisamos encontrar **maneiras** de **ajustar seus pesos** de forma que o erro seja minimizado.
- Na sequência veremos duas formas de se encontrar o **conjunto de pesos** através da **minimização da função de erro**.

***Como encontramos os pesos que
minimizam a função de erro?***

Como minimizar a função de erro?

Exemplo da variação da função de erro com apenas um peso.



A reta vermelha é **tangente** à curva no ponto P (roxo). Sua **inclinação** é dada pela **derivada parcial** da função naquele ponto.

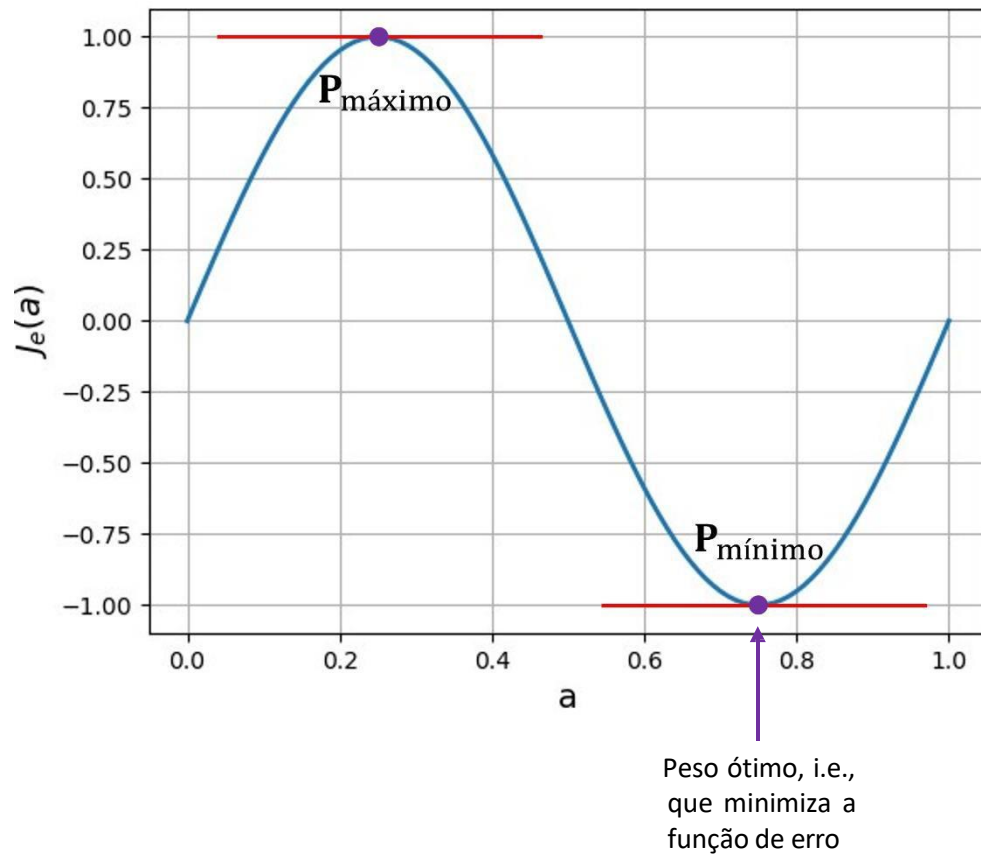
- Da disciplina de cálculo, sabemos que derivando a **função de erro** em relação ao vetor $\mathbf{a}$ e igualando a derivada à 0, nós podemos encontrar o **ponto** (i.e., os valores dos pesos) onde a **taxa de variação da função é nula** (i.e., não cresce ou decresce mais)

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \frac{\partial \|\mathbf{y} - \mathbf{X}\mathbf{a}\|^2}{\partial \mathbf{a}} = 0$$

- A resolução da derivada também é interpretada como sendo o **ponto** onde a **inclinação de um hiperplano (e.g., reta) tangente à função de erro é nula**.
- Após aplicarmos a derivada parcial, chegamos à seguinte equação

$$2\mathbf{a}^T \mathbf{X}^T \mathbf{X} - 2\mathbf{y}^T \mathbf{X} = 0.$$

Como minimizar a função de erro?



- Porém, o **ponto** onde a **taxa de variação da função de erro é nula** pode ocorrer tanto em um **mínimo** quanto em um **máximo** da função, pois a inclinação da reta tangente é zero em ambos.
- Porém, no nosso caso, estamos interessados no **ponto** onde a **função de erro tem o seu menor valor**.
- É nesse ponto que encontramos os pesos que nos dão a **função hipótese ótima**.

Então, como sabemos se o ponto encontrado é um mínimo ou um máximo da função?

Como minimizar a função de erro?

- Se a *inclinação da reta tangente* é **nula** e a *derivada parcial de segunda ordem* da função de erro for **positiva**, então o **ponto** nos dá o **mínimo da função**,

$$\frac{\partial^2 \|y - Xa\|^2}{\partial^2 a} = 2X^T X.$$

- Se a **matriz de atributos**, X , tiver **posto** igual a $K + 1$, então a matriz $X^T X$ é **positiva semi-definida** e, portanto, o **ponto** encontrado acima é **realmente o ponto de mínimo** da **função de erro**.
 - **Posto de uma matriz**: é o número de linhas ou colunas linearmente independentes da matriz.
 - Uma matriz quadrada $X^T X$ é **positiva semi-definida** se $z^T X^T X z = \|Xz\|_2^2 \geq 0, \forall z \neq 0$

Equação normal

- Agora que sabemos que se a matriz de atributos, X , tiver $K + 1$ **colunas linearmente independentes** encontramos o **ponto de mínimo da função** através da sua **derivada parcial**, nós podemos resolver a equação da derivada parcial e encontrar o **vetor de pesos ótimo**.
- Assim, resolvendo a equação em função de $\mathbf{a}$, temos

$$2\mathbf{a}^T \mathbf{X}^T \mathbf{X} - 2\mathbf{y}^T \mathbf{X} = 0,$$

Conjunto de pesos
correspondente ao ponto
mais baixo da função de erro



$$\mathbf{a}_{\text{opt}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}.$$

- Essa equação é conhecida como **equação normal** e nos dá a **solução ótima** em relação a minimização do **erro quadrático médio** para esse **sistema de equações lineares**.
- Notem que só precisamos do conjunto de amostras coletadas (i.e., de treinamento), X e y , para encontrar o **vetor de pesos ótimo**, $\mathbf{a}_{\text{opt}}$.

Observações quanto a equação normal

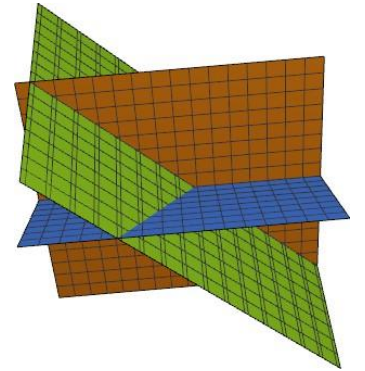
1. A **equação normal** encontra uma **solução única** se e somente se a matriz, $X^T X$, for **invertível** (i.e., **não-singular**).
 - Para que a matriz seja invertível, ela deve ter **posto** igual a $K + 1$.
2. A **equação normal** só funciona para sistemas **determinados ou sobredeterminados**, ou seja, quando o número de equações (i.e., pares x e y) é **igual ou maior** do que o número de incógnitas (i.e., pesos), ou seja, $N \geq K + 1$.
3. Para sistemas **subdeterminados**, ou seja, que têm menos equações do que incógnitas, a matriz $X^T X$ tem **posto** menor do que $K + 1$ e, portanto, é **singular** (i.e., a matriz $X^T X$ não é invertível).
 - Neste caso, não existe solução ou ela não é única.

Causas para singularidade de $\Phi^T \Phi$

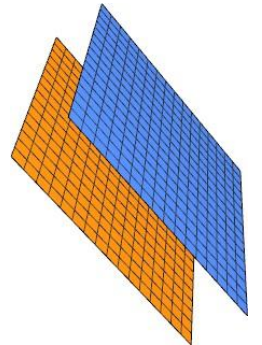
Algumas causas para a **singularidade** de $\Phi^T \Phi$ são:

- Atributos linearmente dependentes.
 - **Ex.:** $x_1 = \text{área em } m^2$ e $x_2 = \text{área em } pés^2 \Rightarrow 1 m^2 = 10.7639 pés^2$.
 - **Solução:** remover um dos atributos linearmente dependentes.
 - **Técnicas:** seleção de atributos (LASSO) e redução de dimensionalidade (PCA).
- Número de pesos $\{a_0, a_1, \dots, a_K\}$ maior do que o de observações $\{\mathbf{x}(i), y(i)\}, i = 0, \dots, N - 1$, i.e., $K + 1 > N$:
 - Este é um **sistema subdeterminado**, ou seja, não existe solução ou existe um número infinito delas.
 - **Solução:** coletar mais exemplos (se não for possível, replicar exemplos adicionando ruído a eles) e remover atributos (seleção de atributos).

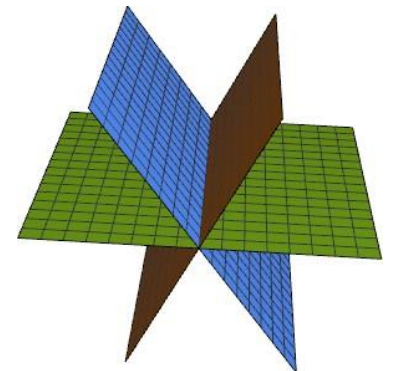
$$a_0x_0 + a_1x_1 + a_2x_2 = c$$



Sistema com uma única solução



Sistema sem solução



Sistema com infinitas soluções

Superfície de erro

$$J_e(\mathbf{a}) = \frac{1}{N} \|\mathbf{y} - \mathbf{X}\mathbf{a}\|^2$$

- Notem que o EQM é função do vetor de pesos, $\mathbf{a}$.
- Ou seja, o erro varia conforme os valores dos pesos mudam.
- $J_e(\mathbf{a})$ faz o **mapeamento** entre o **vetor de pesos** e o **erro correspondente**:
 - $J_e(\mathbf{a}): \mathbb{R}^{K+1 \times 1} \rightarrow \mathbb{R}$
- Se nós calcularmos o EQM para diversos valores de $\mathbf{a}$ e plotarmos esses erros em função dos pesos, nos obtemos uma superfície chamada de **superfície de erro**.

***Que formato vocês acham que essa
superfície tem?***

Superfície de erro

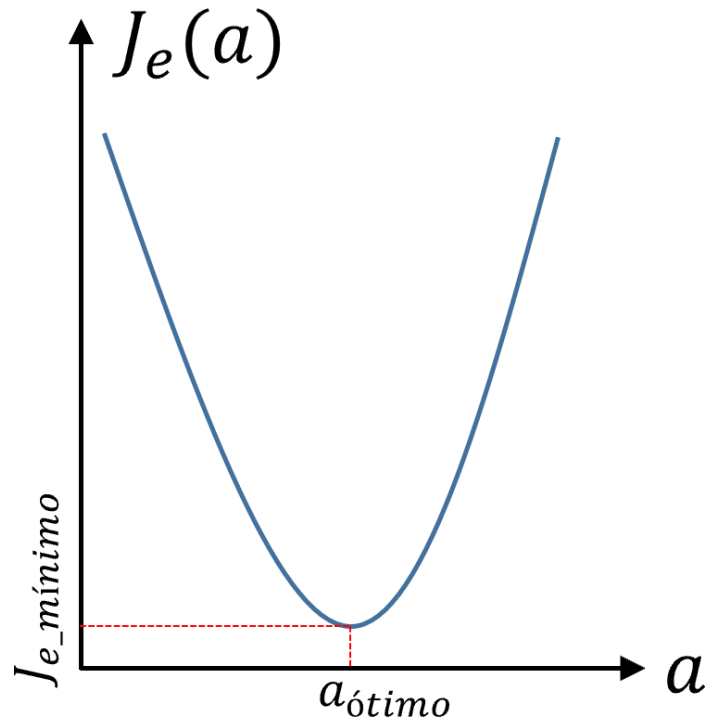
- É fácil descobrir se analisarmos a equação do EQM.
- Se expandirmos $J_e(\mathbf{a})$ notamos que ela possui forma **quadrática** com relação ao **vetor de pesos, $\mathbf{a}$** .

$$J_e(\mathbf{a}) = \frac{1}{N} \|\mathbf{y} - \mathbf{X}\mathbf{a}\|^2 = \frac{1}{N} (\mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X}\mathbf{a} - \mathbf{a}^T \mathbf{X}^T \mathbf{y} + \boxed{\mathbf{a}^T \mathbf{X}^T \mathbf{X} \mathbf{a}}) .$$

Termo
quadrático,
 $\|\mathbf{X}\mathbf{a}\|^2$.

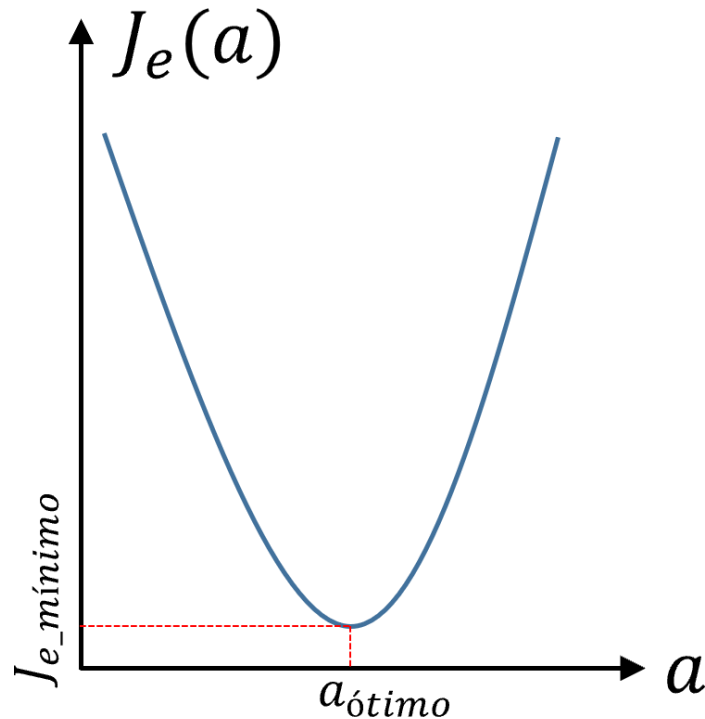
- O gráfico de uma função quadrática é uma **parábola**.
- Essa parábola tem a **concavidade voltada para cima**, pois $\|\mathbf{X}\mathbf{a}\|^2 \geq 0$
- O que isso significa?

Superfície de erro



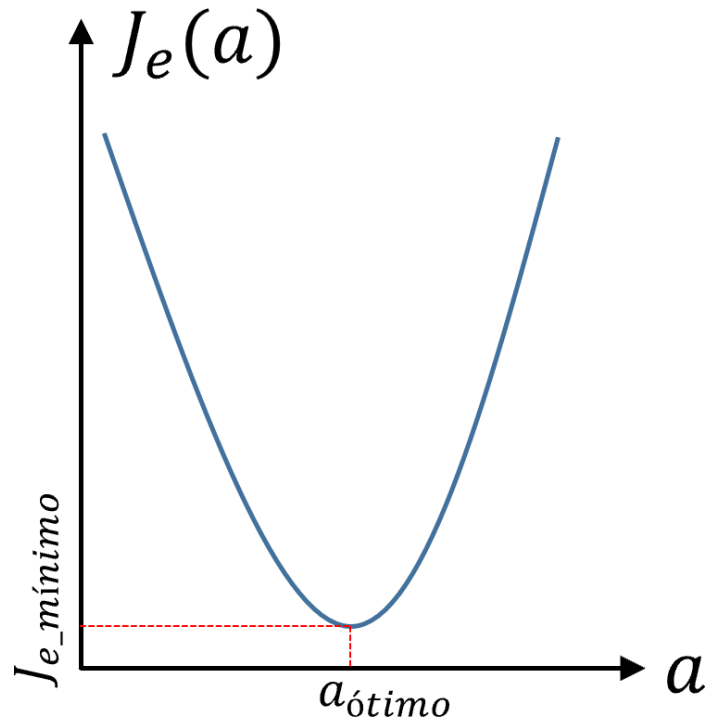
- Isso significa que a **superfície de erro** tem o formato de uma **parábola convexa** (i.e., tem formato de **tigela** ou **vale**).
- A figura ao lado mostra o formato da superfície de erro para uma **função hipótese com apenas um peso**.
- Para funções hipótese com mais pesos, teremos superfícies em 3 ou mais dimensões.

Superfície de erro



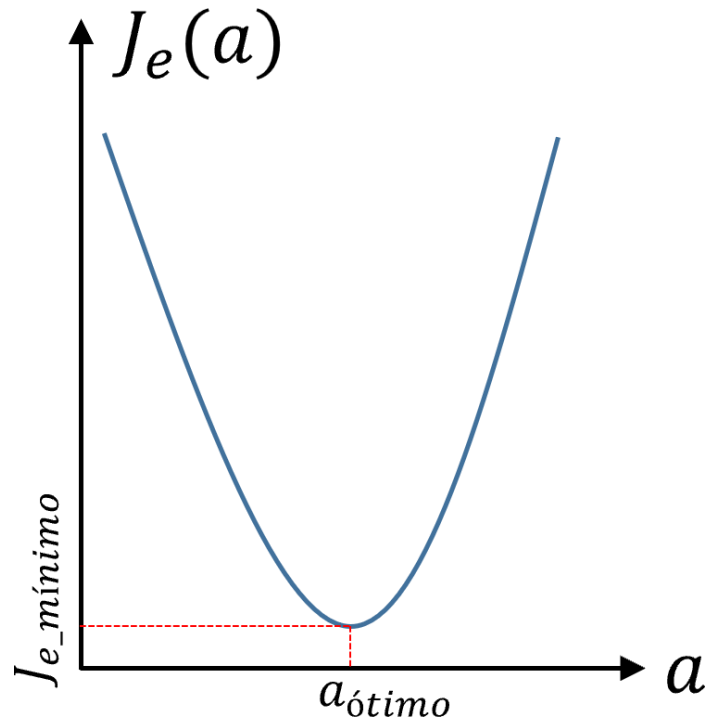
- Vejam que a superfície possui um **único valor mínimo** (i.e., o **vértice** da parábola).
- Esse ponto onde a superfície (i.e., a função de erro) tem o seu menor valor é chamado de **ponto de mínimo**.
- Ele também é chamado de **mínimo global**, pois não há outro valor do peso a que resulte em um erro menor.
- Todos os outros valores têm erro maior.

Superfície de erro



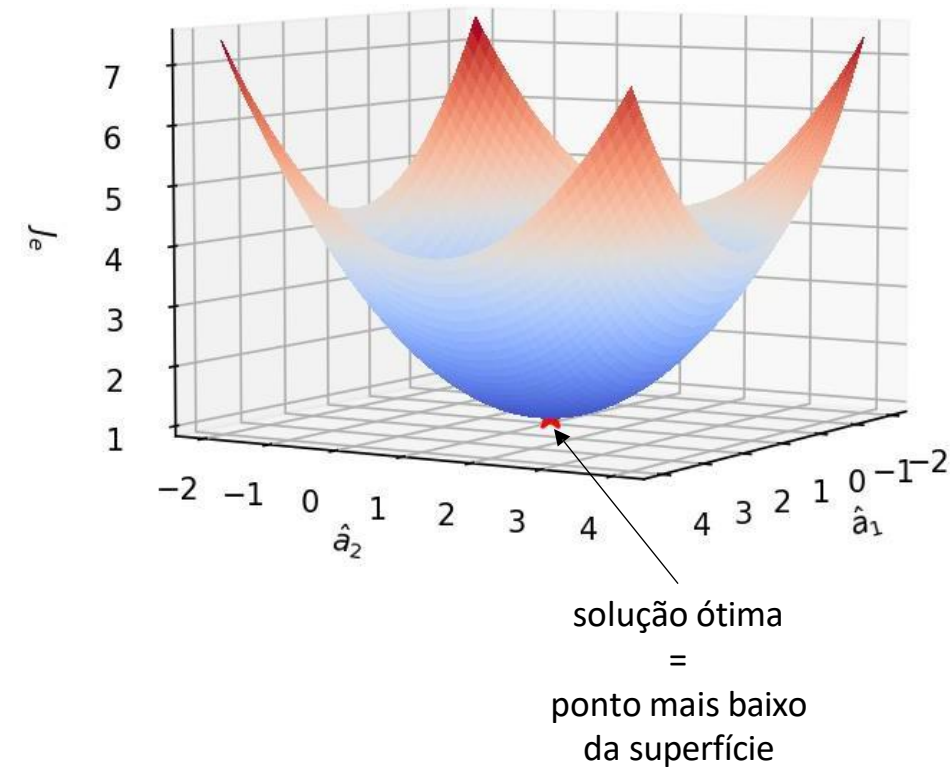
- Este é o motivo de usarmos o **erro quadrático médio** como **função de erro**, pois **só existe um único vetor de pesos, a , que minimiza a função de erro.**
- O ponto de mínimo nos dá o **valor ótimo** dos pesos da função hipótese.
- Ele é encontrado através da **equação normal**.
- Ou **visualmente**, em alguns casos, se plotarmos a superfície, pois basta **localizar seu ponto mais baixo**.

Superfície de erro



- O entendimento do **formato** e da questão da superfície ter **apenas um ponto de mínimo** nos ajudará a entender como o **algoritmo de otimização iterativa** que vamos aprender em breve funciona.
- Na sequência, plotamos a superfície de erro para uma função hipótese.

Plotando a superfície de erro

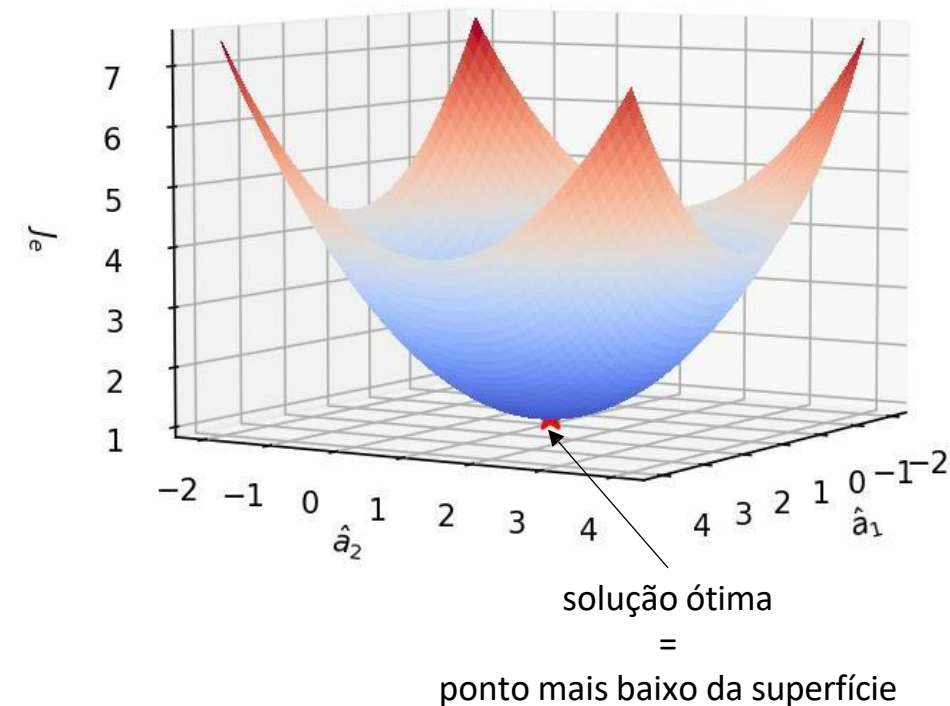


- Até o momento, conseguiríamos encontrar o **ponto de mínimo** com a **equação normal** ou **visualmente**, plotando a **superfície de erro** a partir da função do EQM

$$J_e(\hat{a}_1, \hat{a}_2) = \frac{1}{N} \sum_{n=0}^{N-1} \left(y_{noisy}(n) - \hat{y}(n) \right)^2$$
$$= \frac{1}{N} \sum_{n=0}^{N-1} \left(y_{noisy}(n) - (\hat{a}_1 x_1(n) + \hat{a}_2 x_2(n)) \right)^2$$

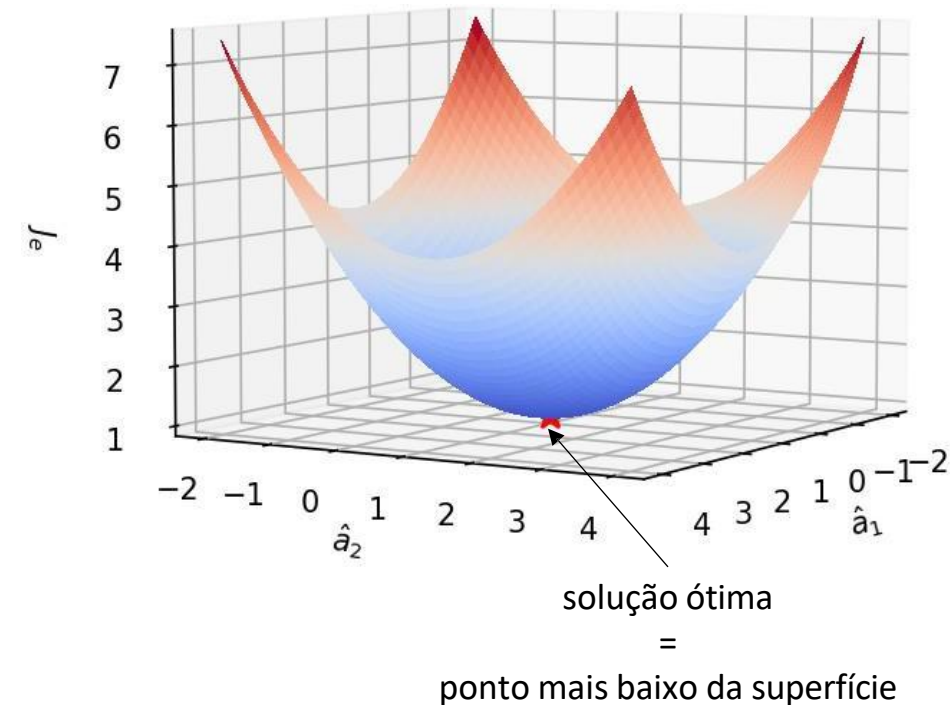
e **procurando por sua parte mais baixa**.

Plotando a superfície de erro



- Os valores de erro, $J_e(\hat{a}_1, \hat{a}_2)$, para plotarmos a **superfície de erro** são obtidos variando-se $\hat{a}_1$ e $\hat{a}_2$ na equação do EQM.
- Para este exemplo, onde temos dois atributos, a **superfície de erro** é **representada por uma figura em 3 dimensões**, onde cada par de valores $\hat{a}_1$ e $\hat{a}_2$ corresponde a um erro, $J_e(\hat{a}_1, \hat{a}_2)$.

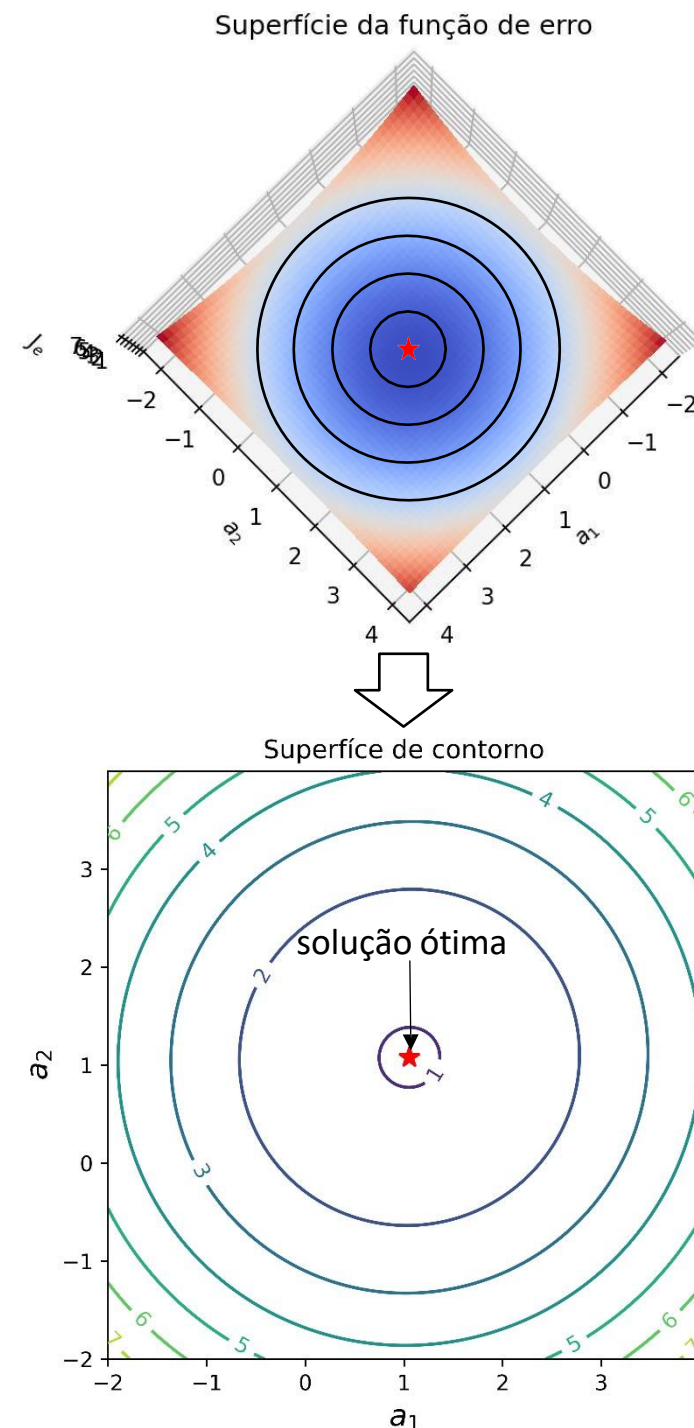
Plotando a superfície de erro



- Devido a superfície ser **convexa**, temos apenas **um ponto de mínimo**, o **mínimo global**.
- O **mínimo global** é ponto mais baixo da superfície e nos dá a solução ótima.
- **Observações**
 - Só conseguimos encontrar o ponto de mínimo de **forma visual** se tivermos funções hipóteses com até 2 atributos.
 - Além disso, é uma **solução com altíssima complexidade computacional**, pois usa a força bruta para encontrar o ponto ótimo.
 - Assim, em geral, usamos a forma fechada.

Plotando a superfície de contorno

- Outra figura importante que plotamos a partir dos valores de erro para diferentes conjuntos de pesos é chamada de **superfície de contorno**.
- Uma superfície de contorno contém várias **linhas de contorno**.
 - Uma **linha de contorno** é uma **curva ao longo da qual uma função tem um valor constante**.
- Na superfície de erro, cada linha indica uma curva ao longo da qual o **erro é constante**.
- Ou seja, **qualquer par de valores $\hat{a}_1$ e $\hat{a}_2$ ao longo de uma mesma curva terá o mesmo valor de erro**.



Para casa

- Estudar o material sobre variações do formato da superfície de erro.
- [https://github.com/luiz10ml/tp555-ml/blob/main/slides/TP555_Regress%C3%A3o_Linear%20\(Varia%C3%A7%C3%B5es%20do%20formato%20da%20superf%C3%ADcie%20de%20erro\).pdf](https://github.com/luiz10ml/tp555-ml/blob/main/slides/TP555_Regress%C3%A3o_Linear%20(Varia%C3%A7%C3%B5es%20do%20formato%20da%20superf%C3%ADcie%20de%20erro).pdf)

Porém, nem tudo são flores com a equação normal...

Desvantagens da equação normal

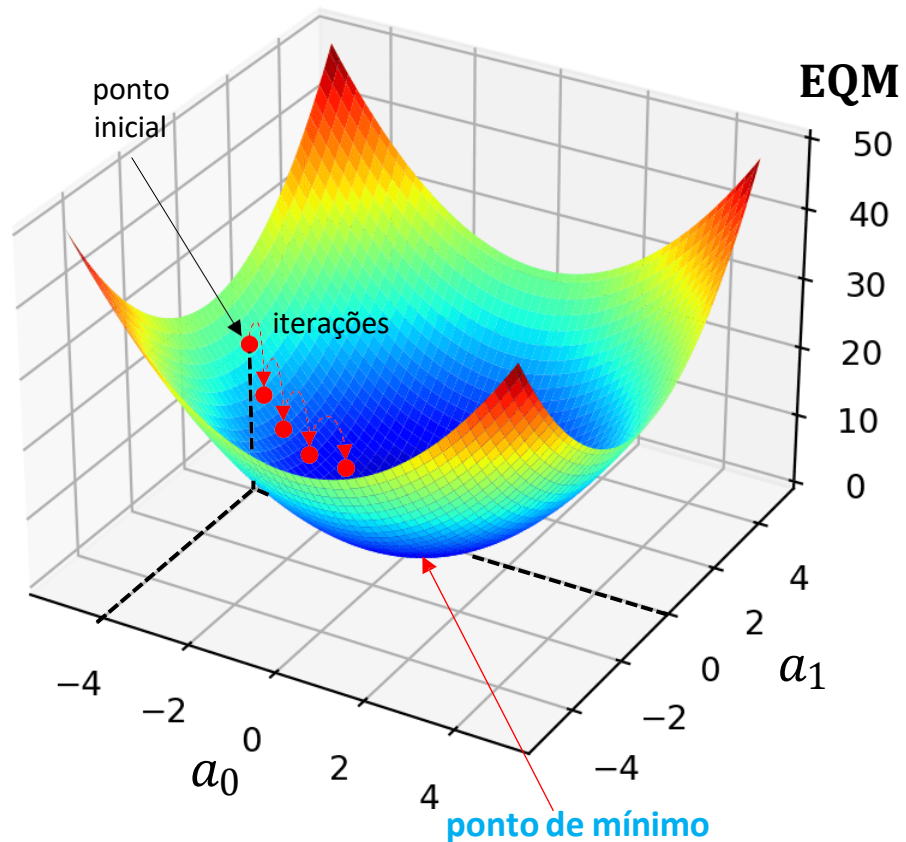
- **Alta complexidade computacional:** a solução da *equação normal* envolve o *cálculo da inversa* de $X^T X$.
- Esse cálculo tem complexidade computacional que varia de $O(K^{2.4})$ a $O(K^3)$, onde K é o número de atributos.
 - “Big O” é uma notação usada para analisar como o *tempo de execução de um algoritmo cresce à medida que o número de entradas aumenta*.
 - **Exemplo:** Se o número de *atributos*, K , passar de 1 para 2, o tempo para cálculo aumenta de $2^{2.4} = 5.3$ a $2^3 = 8$ vezes.
- Alguns conjuntos de dados podem ter centenas ou até milhares de atributos!
 - **House Prices** é um dataset com informações de casas que possui 79 atributos.
 - **ImageNet** é um dataset com imagens, onde cada imagem tem em média $256 \times 256 = 65536$ *pixels* e cada *pixel* é um atributo.

Desvantagens da forma fechada (eq. normal)

- Além disso, dependendo do número de ***exemplos do conjunto de treinamento***, N , e de ***atributos***, x , a matriz de atributos, X , pode se tornar ***muito grande***.
- Consequentemente, a inversão de matrizes e os cálculos da equação normal podem ***consumir muita CPU e memória***.
 - O que pode ser um limitante para dispositivos com recursos restritos (e.g., IoT).
- Adicionalmente, para irmos ***além dos modelos lineares*** (i.e., modelos não-lineares como classificadores e redes neurais) precisamos lidar com o fato de que ***nem sempre existem formas fechadas*** como a ***equação normal***.
- **Portanto, essa abordagem não é escalável e muito menos flexível!**

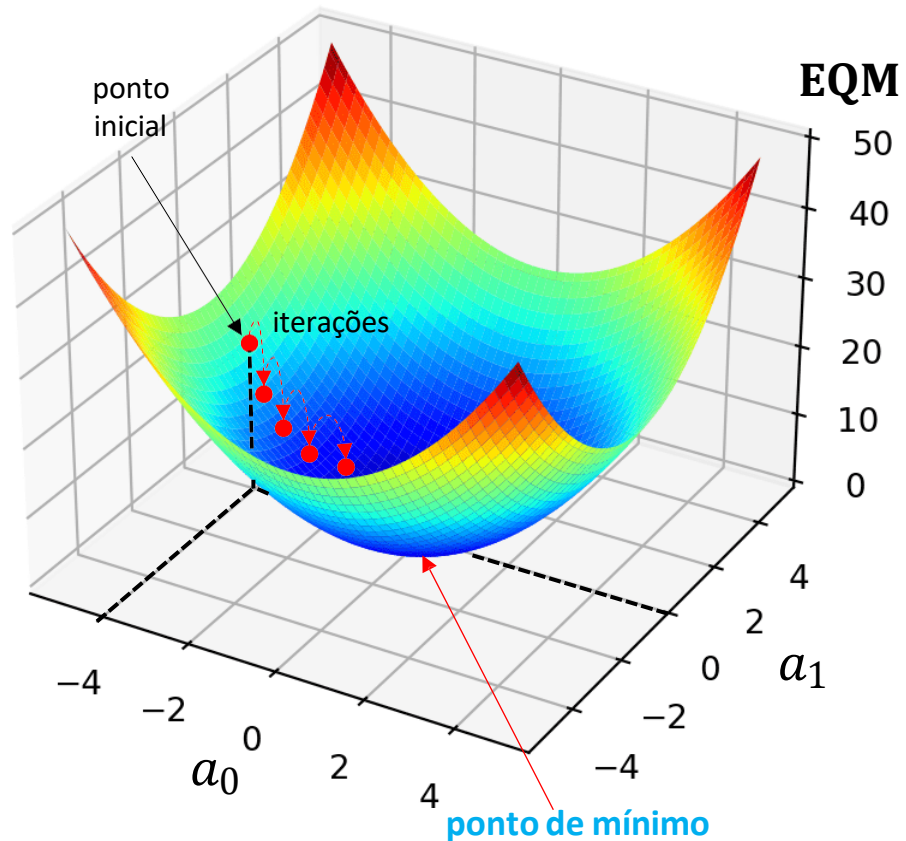
Então o que pode ser feito?

Abordagens iterativas de otimização



- Podemos usar *algoritmos de otimização iterativos*.
- Eles têm *complexidade menor e ajustável* e são *flexíveis* o suficiente para trabalhar com modelos lineares e não lineares.
- Esses algoritmos “*procuram*” de forma *iterativa* os pesos ótimos no *espaço de soluções* (i.e., *superfície de erro*).

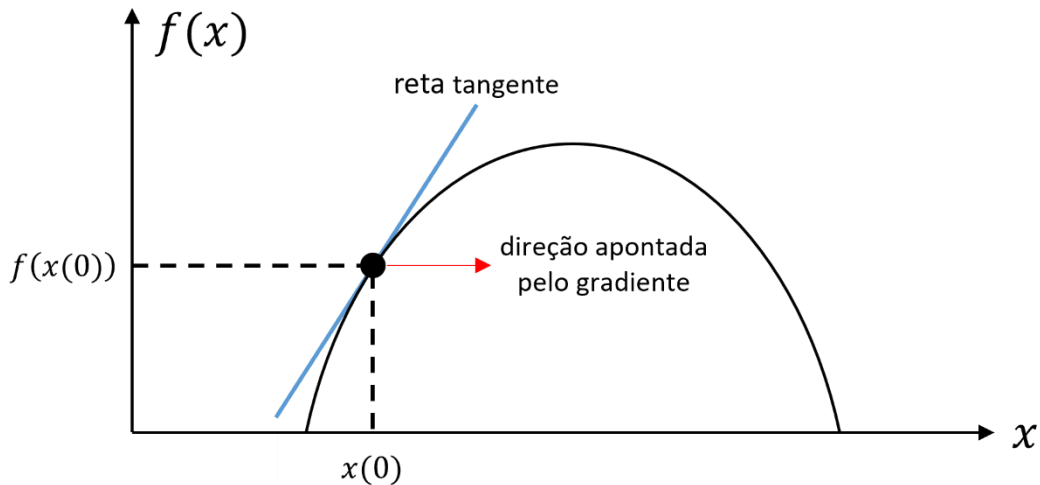
Abordagens iterativas de otimização



- A **cada iteração**, o algoritmo refina os valores dos pesos, ficando mais e mais próximo do **ponto de mínimo**.
- Um exemplo de abordagem iterativa é o algoritmo do **gradiente descendente (GD)**.
- O GD busca **iterativamente**, a partir de um **ponto inicial**, o **ponto mais baixo da superfície de erro**.

***Como o algoritmo caminha
iterativamente até o ponto de mínimo?***

Vetor gradiente

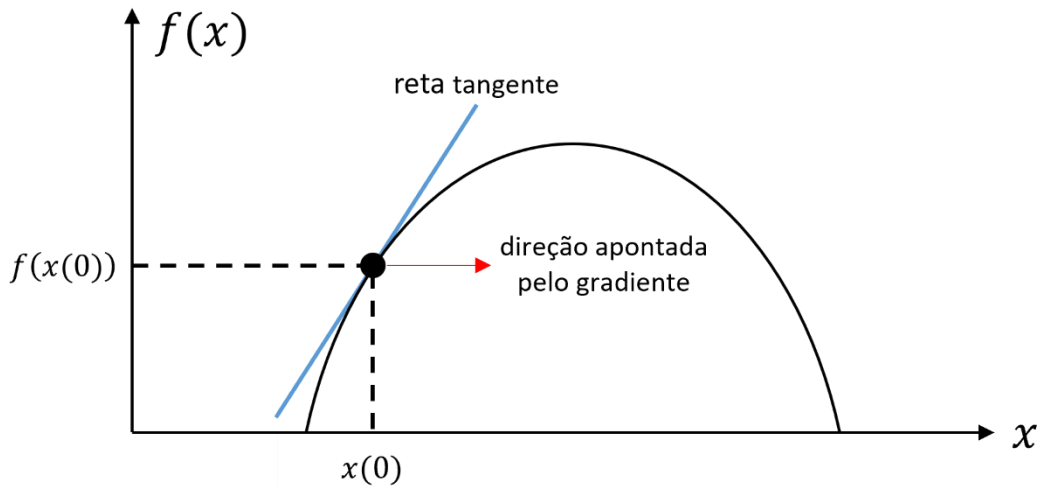


- Vocês se lembram das aulas de cálculo vetorial, onde vocês aprenderam sobre o **vetor gradiente**?

$$\nabla f(\mathbf{x}) = \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}$$

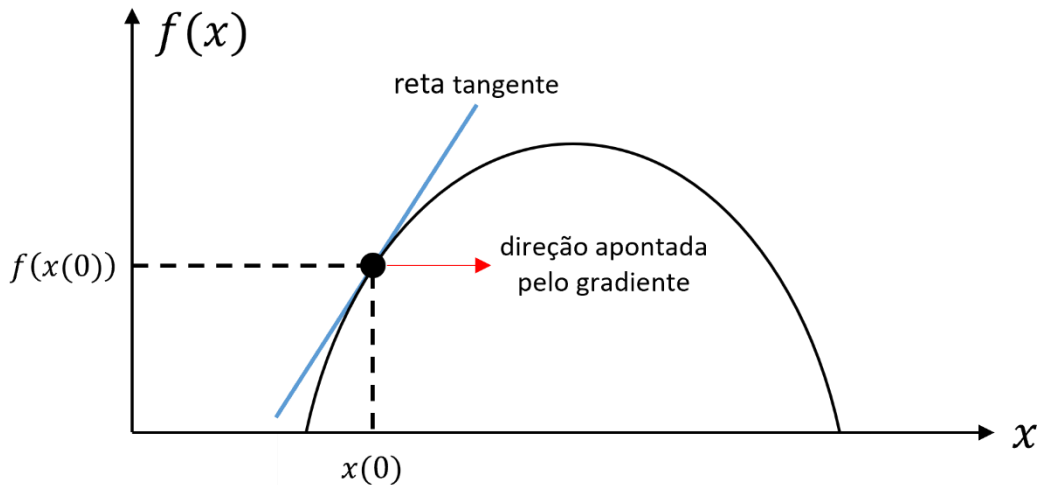
- Qual **informação** ele nos dá sobre a função, $f(\mathbf{x})$?

Vetor gradiente



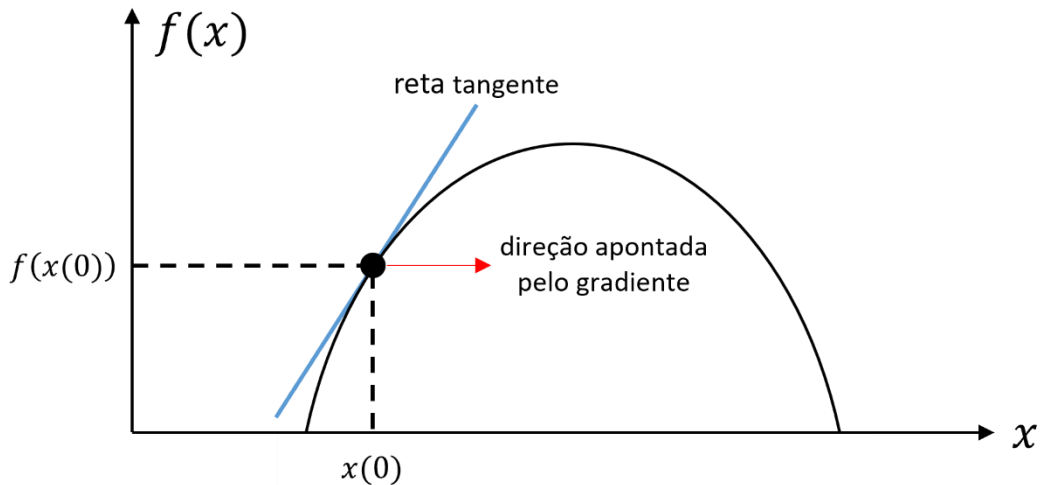
O vetor gradiente *aponta* na *direção* em que a função $f(x)$ *cresce mais rapidamente* a partir do ponto em que é avaliado

Vetor gradiente



- As **magnitudes** dos elementos do vetor gradiente indicam a **taxa de crescimento da função** na direção apontada por ele.
 - Quanto maior a magnitude, maior a taxa de crescimento naquela direção.
- Os **sinais** dos elementos do vetor dizem para que **“lado”** (**aumentar ou diminuir**) os valores dos argumentos, x , devem ir para que o **valor de $f(x)$ seja maior do que o atual**.

Vetor gradiente



Obs.: No caso da função, f , ter apenas um argumento, x , o vetor gradiente dá a inclinação de uma **reta** tangente ao ponto onde o vetor é calculado.

- O vetor gradiente pode ser também interpretado como a **inclinação de um hiperplano tangente à função no ponto onde ele é calculado**.
 - Quanto **maior o valor absoluto** do gradiente, **mais inclinado é o plano tangente** naquele ponto.
 - Portanto, **um vetor gradiente igual a 0 indica inclinação nula**.
 - Ou seja, **a função não varia mais em nenhuma direção**.
 - Onde isso ocorre? Nos **extremos da função**, ou seja, em seus **pontos de máximo e de mínimo**.

Vetor gradiente

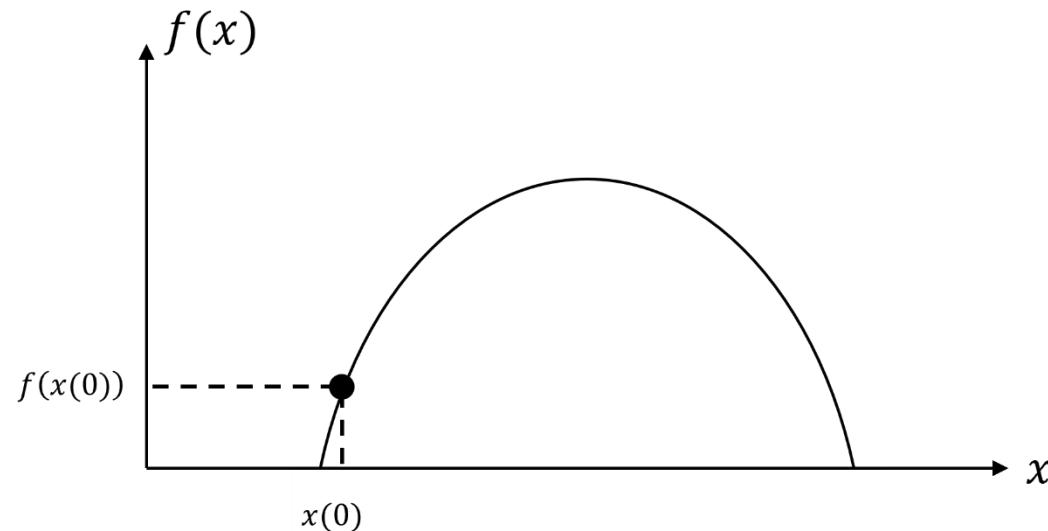
- O **vetor gradiente** de uma função com K argumentos, $f(x_1, x_2, \dots, x_K)$, é definido pela sua **derivada parcial em relação a cada um de seus argumentos** x_k , $k = 1, \dots, K$:

$$\begin{aligned} \nabla f(x_1, x_2, \dots, x_K) \\ = \left[\frac{\partial f(x_1, x_2, \dots, x_K)}{\partial x_1} \quad \frac{\partial f(x_1, x_2, \dots, x_K)}{\partial x_2} \quad \dots \quad \frac{\partial f(x_1, x_2, \dots, x_K)}{\partial x_K} \right]^T \end{aligned}$$

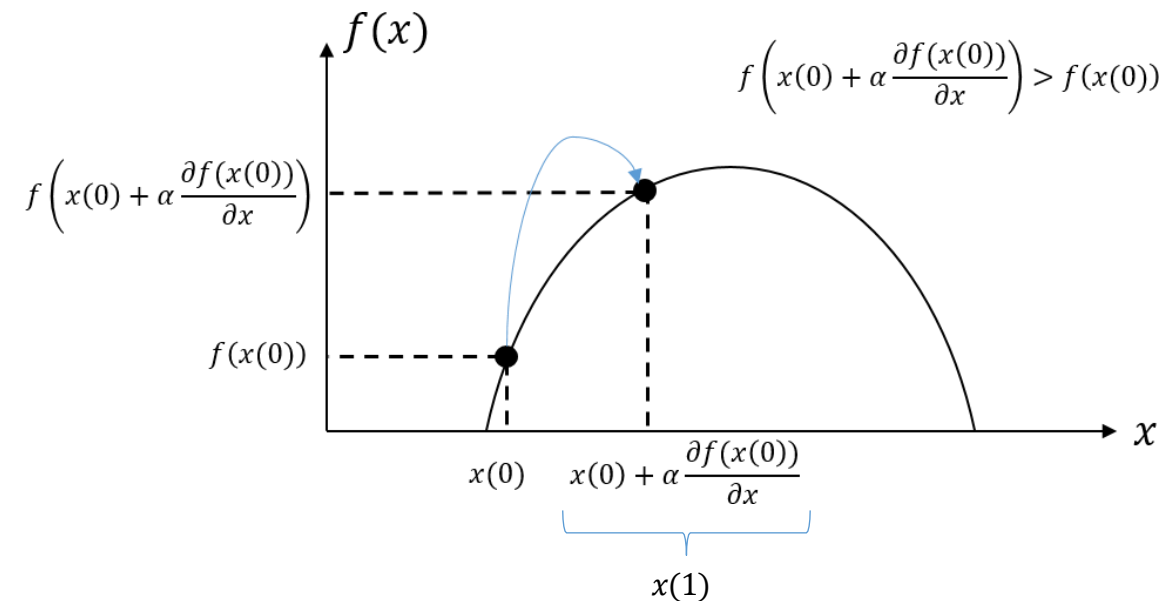
- Notem que o vetor gradiente é representado pelo símbolo *Nabla*, ∇ , e é definido como um **vetor coluna**, com número de elementos igual ao número de argumentos da função.
- **OBS.:** Na sequência, por questões didáticas, mas sem perda de generalidade, nós vamos assumir uma função com apenas um argumento, $f(x)$.

O vetor gradiente indica o caminho para o ponto de máximo da função

- Imaginem o ponto inicial, $x(0)$, com valor $f(x(0))$ na figura abaixo.
- Se quisermos que o valor de $f(x)$ aumente, devemos aumentar ou diminuir o valor de $x(0)$?
- Ou seja, qual **direção** devemos seguir para **maximizar** o valor de $f(x)$?



O vetor gradiente indica o caminho para o ponto de máximo da função

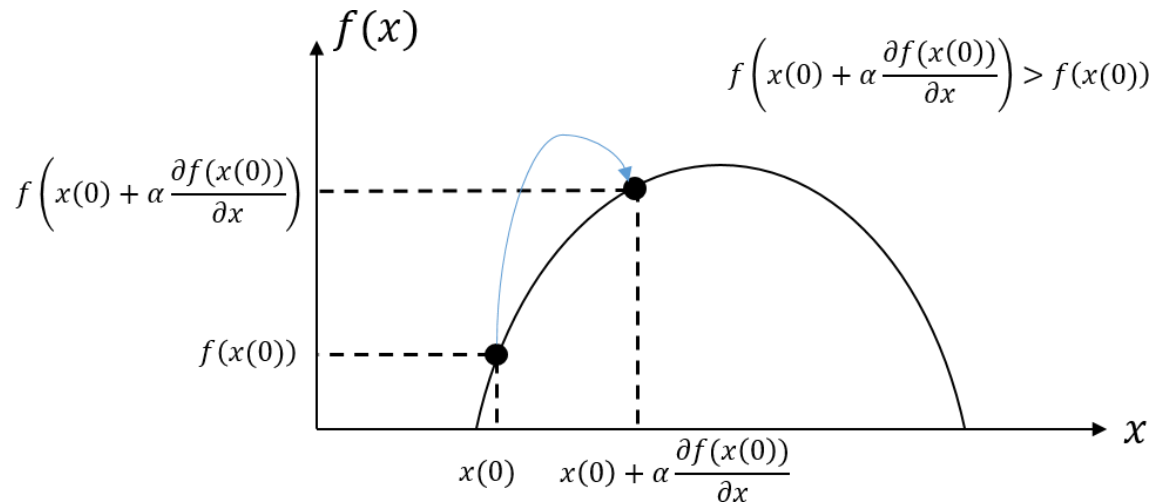


- O vetor gradiente calculado no ponto $x(0)$, $\nabla f(x(0))$, diz **em qual direção** devemos caminhar para **aumentar o valor da função** $f(x)$ mais rapidamente.
- Se **adicionarmos** uma **porcentagem***, α , do gradiente ao valor de $x(0)$, teremos que o **novo ponto**, $x(1)$, terá um valor de $f(x)$ **maior do que o anterior**, ou seja

$$f\left(\underbrace{x(0) + \alpha \frac{\partial f(x(0))}{\partial x}}_{x(1)}\right) > f(x(0)).$$

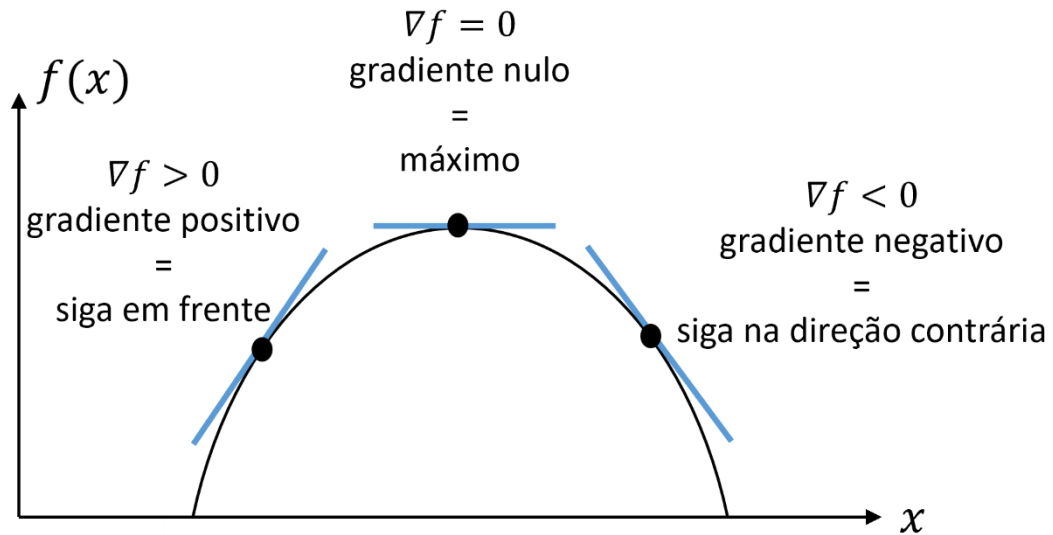
* O vetor gradiente dá a **direção** e não a distância até o ponto de máximo.

O vetor gradiente indica o caminho para o ponto de máximo da função



- Se, a cada **ponto atual**, nós calcularmos o vetor gradiente e adicionarmos uma **porcentagem** dele ao ponto, teremos um **novo ponto** que leva a um **valor da função maior do que o valor anterior**.
- Portanto, podemos criar um **procedimento** que vai **iterativamente caminhando** em **direção ao ponto de máximo da função**.
- Vamos entender como isso pode ser feito.

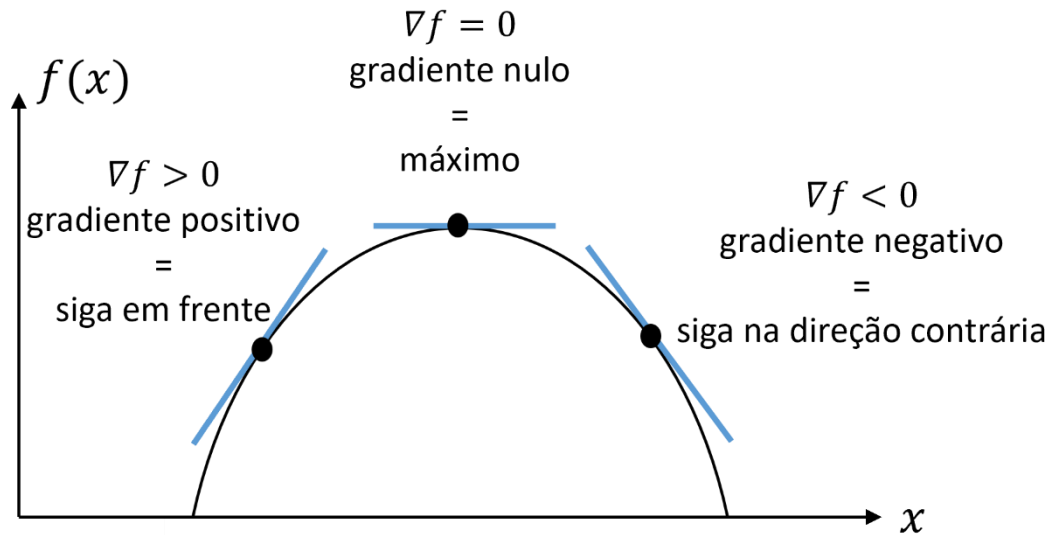
Algoritmo do gradiente ascendente



- Se o vetor gradiente de $f(x)$ em um **ponto** $x(n)$ qualquer dá a **inclinação da reta tangente à função naquele ponto**, então, nesse **ponto**, um valor de gradiente com sinal:

- + (reta com inclinação positiva) indica que o **ponto de máximo está à frente do ponto atual**.
- - (reta com inclinação negativa) indica que o **ponto de máximo está atrás do ponto atual**.
- 0 (reta com inclinação nula) indica que **ponto de máximo foi encontrado**.

Algoritmo do gradiente ascendente



- Portanto, *seguindo na direção* indicada pelo *vetor gradiente*, *chegamos* ao *ponto de máximo da função*.
- Um *algoritmo iterativo de otimização* que *siga a direção* indicada pelo *vetor gradiente* para encontrar o *ponto de máximo* de uma função é conhecido como *gradiente ascendente*.
- Mas como ele funciona?

Algoritmo do gradiente ascendente

- Inicializa-se o argumento $x(0)$ com um valor arbitrário, em geral, um valor **aleatório**.
- A cada **iteração**, i , calcula-se o **vetor gradiente** da função $f(x)$ no ponto atual, $x(i)$, e atualiza-se o valor do argumento usando uma porcentagem do gradiente, ou seja

$$x(i + 1) = x(i) + \alpha \nabla f(x(i)), i \geq 0.$$

- De tal forma, que a cada **iteração** se tenha

$$f(x(i + 1)) > f(x(i)), i \geq 0.$$

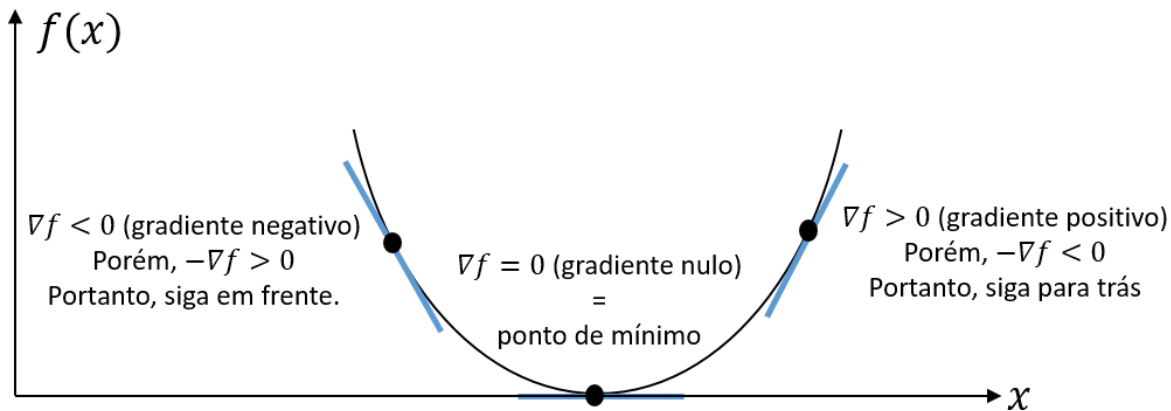
- As iterações se repetem até que o **ponto de máximo** seja atingido, ou seja, $\nabla f(x(i)) = 0$ e, conseqüentemente, o argumento x **não sofre mais atualizações** (i.e., o algoritmo convergiu).

- Chamamos de **convergência** quando a variação da função $f(x)$ entre iterações consecutivas é muito pequena, ou seja, o valor é praticamente constante.

Porém, se vocês se lembram, no problema da regressão linear nós queremos *encontrar o ponto de mínimo da função de erro* ao invés do seu máximo.

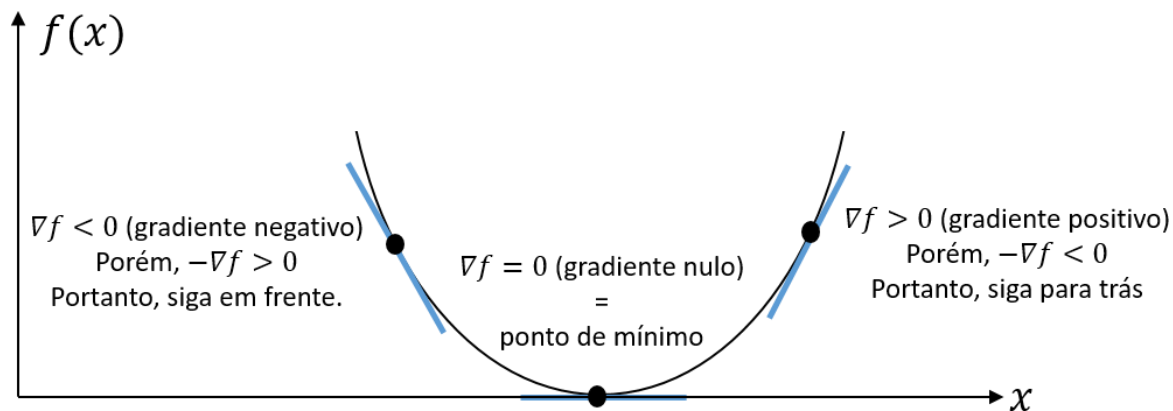
O que fazer?

Algoritmo do gradiente descendente



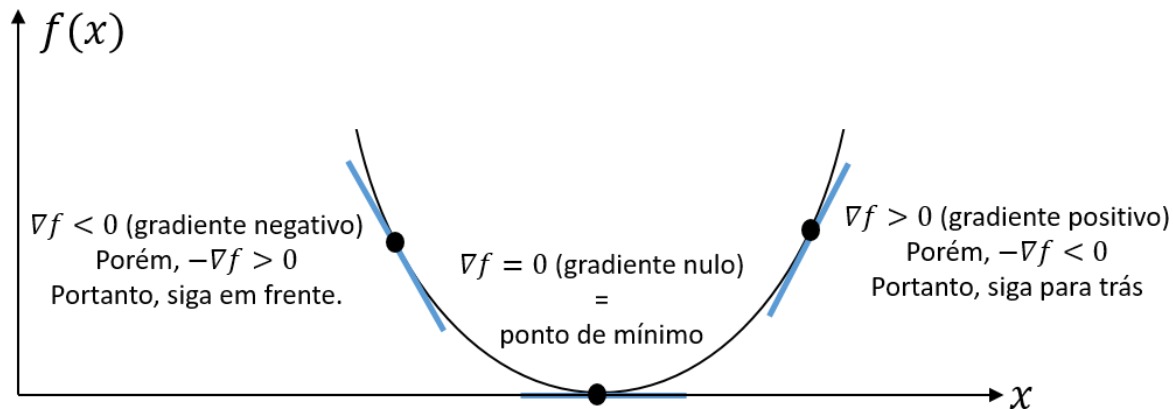
- Se para encontrarmos o **ponto de máximo** de uma função basta seguirmos a direção apontada pelo vetor gradiente.
- Portanto, para encontrarmos o **ponto de mínimo**, basta seguirmos na **direção oposta a apontada pelo vetor gradiente** em um determinado ponto, $x(i)$
 $-\nabla f(x(i))$.

Algoritmo do gradiente descendente



- Quando seguimos na ***direção contrária a da máxima taxa de crescimento***, dada pelo ***vetor gradiente***, estamos indo na ***direção de decrescimento mais rápido da função***.

Algoritmo do gradiente descendente



- Assim, um **algoritmo iterativo de otimização** que siga na **direção contrária** a indicada pelo **vetor gradiente** para encontrar o **ponto de mínimo** de uma função é chamado de **gradiente descendente (GD)**.

Algoritmo do gradiente descendente

- Inicializa-se o argumento $x(0)$ com um valor arbitrário, em geral, um valor **aleatório**.
- A cada **iteração**, i , calcula-se o **vetor gradiente** da função $f(x)$ no ponto atual, $x(i)$, e atualiza-se o valor do argumento usando uma porcentagem do gradiente, ou seja

$$x(i + 1) = x(i) \ominus \alpha \nabla f(x(i)), i \geq 0.$$

Única diferença para o gradiente ascendente.

- De tal forma, que a cada **iteração** se tenha
$$f(x(i + 1)) < f(x(i)), i \geq 0.$$
- As iterações se repetem até que o **ponto de mínimo** seja atingido, ou seja, $\nabla f(x(i)) = 0$ e, conseqüentemente, o argumento x **não sofra mais atualizações** (i.e., o algoritmo convergiu).
 - **Convergência**: valor da função $f(x)$ entre iterações consecutivas é praticamente constante.

Observação

- Os conceitos vistos até agora foram apresentados para uma função com um único argumento, $f(x)$.
- Porém todos eles são válidos para funções com vários argumentos, $f(x_0, x_1, \dots, x_K)$.
- Esse será o caso das funções que iremos utilizar em breve.
- No nosso caso, a função será a do **erro quadrático médio (EQM)**, que terá como argumentos os pesos da função hipótese.

Como neste curso queremos *minimizar a função de erro*, iremos focar no *gradiente descendente*.

Características do gradiente descendente

- Algoritmo **genérico de otimização**: pode ser aplicado não apenas a problemas de aprendizado de máquina, mas a **qualquer problema que envolva encontrar os parâmetros que minimizam uma função**.
- O único requisito é que essa função seja **diferenciável**.
 - No caso da regressão linear, a função de erro deve ser diferenciável.
- Escalona melhor do que o método da **equação normal** para grandes conjuntos de dados.
- É de fácil implementação.
- Não é necessário nos preocupar com matrizes mal condicionadas, ou seja, matrizes com determinante próximo de 0 (i.e., quase **singulares**).
- Pode ser usado com modelos não-lineares.

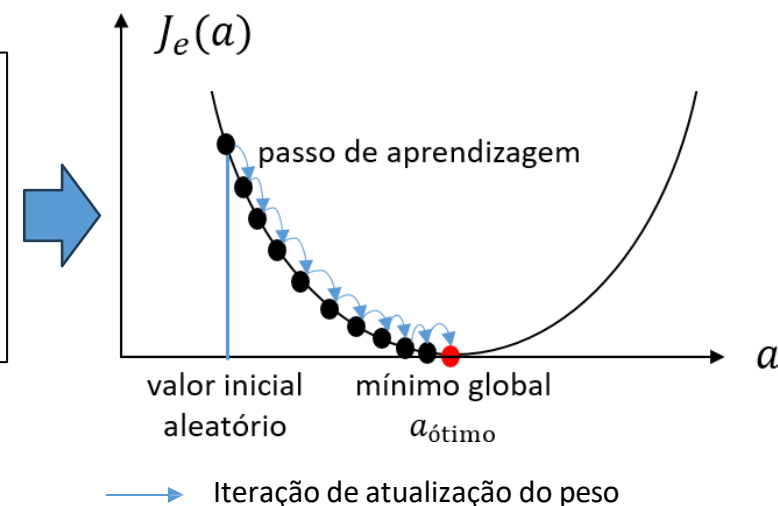
*Como aplicamos o algoritmo do **gradiente descendente** ao problema da **regressão linear**?*

*Ou seja, como encontramos os **pesos ótimos** da **função hipótese** usando o gradiente descendente?*

O algoritmo do gradiente do descendente

$\mathbf{a} \leftarrow$ inicializa o vetor de pesos em um ponto aleatório do espaço de pesos
loop até convergir **ou** atingir o número máximo de iterações **do**

$\nabla J_e(\mathbf{a}) \leftarrow$ cálculo do vetor gradiente, $\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}}$
 $\mathbf{a} \leftarrow \mathbf{a} - \alpha \times \nabla J_e(\mathbf{a})$ (*regra de atualização dos pesos*)



- **OBS.:** O *passo de aprendizagem*, α , dita o tamanho do deslocamento dado na direção oposta a apontada pelo **vetor gradiente**.
 - Ele é sempre um valor maior do que zero.
- Na sequência, encontraremos o **vetor gradiente** da **função de erro**.

Calculando o vetor gradiente

- Para calcularmos o vetor gradiente, vamos considerar o **EQM** como **função de erro** e a **função do hiperplano** como **função hipótese**:

$$\hat{y}(n) = h(n) = a_0 + a_1 x_1(n) + \cdots + a_K x_K(n) = \sum_{i=0}^K a_i x_i(n) = \mathbf{a}^T \mathbf{x}(n)$$

onde n é o **número do exemplo** (ou amostra), K é o **número atributos**, a_i e x_i , $\forall i$ são os **pesos e atributos da função hipótese**, respectivamente, $x_0 = 1$ (i.e., atributo de *bias*) e $\mathbf{a}$ e $\mathbf{x}(n)$ são vetores coluna com todos os pesos e atributos da n -ésima amostra, respectivamente.

- Agora podemos encontrar o vetor gradiente.

Calculando o vetor gradiente

- O **vetor gradiente** da **função de erro em relação aos pesos** é dado por

$$\begin{aligned}\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} &= \frac{\partial}{\partial \mathbf{a}} \left[\frac{1}{N} \sum_{n=0}^{N-1} (y(n) - \hat{y}(n))^2 \right] = \frac{\partial}{\partial \mathbf{a}} \left[\frac{1}{N} \sum_{n=0}^{N-1} (y(n) - \mathbf{a}^T \mathbf{x}(n))^2 \right] \\ &= -\frac{2}{N} \sum_{n=0}^{N-1} [y(n) - \hat{y}(n)] \mathbf{x}(n) = \boxed{-\frac{2}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}})},\end{aligned}$$

Versão matricial do vetor gradiente

onde $\mathbf{X}$ é a matriz de atributos com dimensão $(N \times K + 1)$ e $\mathbf{y}$ e $\hat{\mathbf{y}}$ são vetores coluna $(N \times 1)$ com todos os valores esperados e de saída da função hipótese para os N exemplos coletados, respectivamente.

- **OBS.:** Esse cálculo pode ser diretamente estendido para polinômios.

Atualizando os pesos

- Substituindo o ***vetor gradiente*** na ***equação de atualização dos pesos***, temos

$$\mathbf{a} = \mathbf{a} - \alpha \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \mathbf{a} \boxed{+} \alpha \frac{2}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}),$$

- O sinal positivo (+) é devido ao vetor gradiente encontrado ter sinal negativo.
- Percebam que o vetor gradiente é a ***média da diferença*** entre o rótulo e a saída da função hipótese vezes os atributos ***tomada ao longo de todos os N exemplos do conjunto de treinamento.***
- **OBS.:** Por ser constante, o termo $2/N$ pode ser absorvido por α .

Atualizando os pesos

$\mathbf{a} \leftarrow$ inicializa o vetor de pesos em um ponto aleatório do espaço de pesos
loop até convergir **ou** atingir o número máximo de iterações **do**

$$\mathbf{a} \leftarrow \mathbf{a} + \alpha \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}})$$

- Lembrem-se que *a cada iteração* (i.e., *loop*) do gradiente descendente, precisamos *calcular o vetor gradiente*.
- Isso envolve *calcular as operações matriciais e vetoriais acima*.
- Assim, *se o conjunto de treinamento e o modelo forem muito grandes*, o *treinamento pode ser muito longo e consumir muita CPU e memória*.

Versões do gradiente descendente

- Portanto, para lidarmos com a complexidade computacional, podemos ter **3 versões diferentes, dependendo da quantidade de exemplos** considerados no somatório do vetor gradiente:
 - ***Gradiente descendente em batelada (GDB)***
 - ***Gradiente descendente estocástico (GDE)***
 - ***Gradiente descendente em mini-lotes (GDML)***

Gradiente descendente em batelada

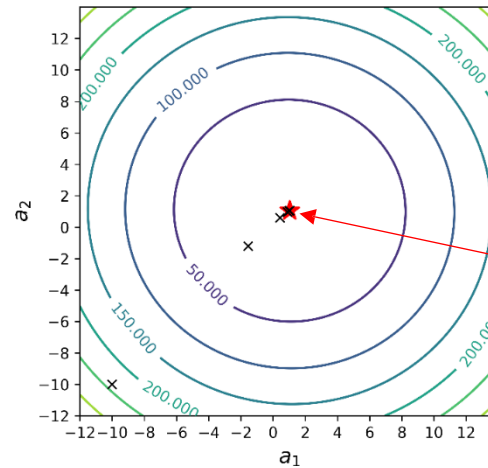
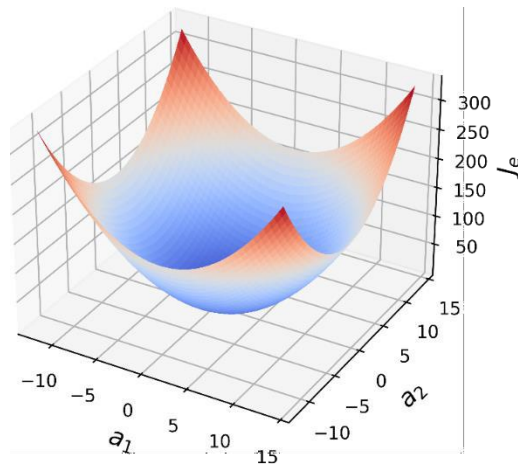
$$\mathbf{a} = \mathbf{a} + \alpha \frac{2}{N} \sum_{n=0}^{N-1} [y(n) - \hat{y}(n)] \mathbf{x}(n) = \mathbf{a} + \alpha \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}})$$

- Considera **todos os exemplos** do conjunto de treinamento para calcular o vetor gradiente.
- Pode ser **computacionalmente custoso** dependendo do tamanho do modelo e do conjunto de dados.
 - Por processar todos os exemplos a cada iteração, pode ser lento e consumir muita CPU e memória com conjuntos muito grandes.
- **Convergência** para o mínimo global é **garantida** quando a função de erro é **convexa** e α não for muito grande.
- É a versão que **obtém os melhores resultados**.

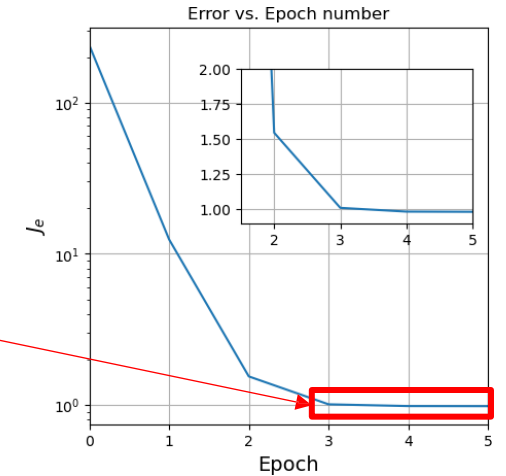
Mais exemplos de implementação do gradiente descendente

- Exemplos de implementação do gradiente descendente
 - https://github.com/luiz10ml/tp555-ml/blob/main/slides/TP555_Exemplos_Gradiente_Descendente.pdf

Características do GD em batelada



convergência



- Por usar todos os exemplos, **segue diretamente**, sem alterar a direção, para o **ponto de mínimo**.
 - Nesse exemplo, segue uma linha reta entre a_1 e a_2 , pois a taxa de decrescimento da superfície de erro é igual para os dois pesos (contornos são circulares).
- **Convergência é garantida** dado que o **passo de aprendizagem não seja muito grande** e se **espere tempo suficiente**.
 - Não fica “oscilando” em torno do ponto de mínimo após alcançá-lo, pois o vetor gradiente neste ponto é praticamente nulo.

Gradiente descendente estocástico

$$\mathbf{a} = \mathbf{a} + \alpha[y(n_{\text{random}}) - \hat{y}(n_{\text{random}})]\mathbf{x}(n_{\text{random}}),$$

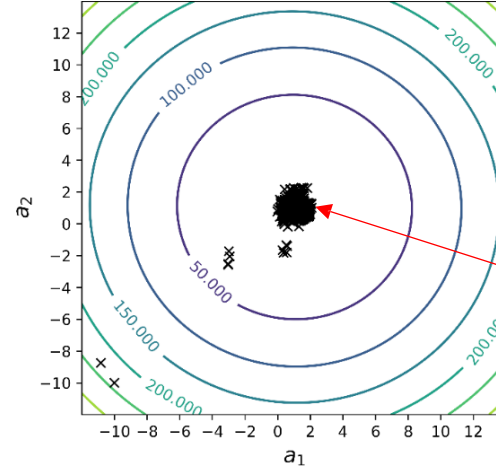
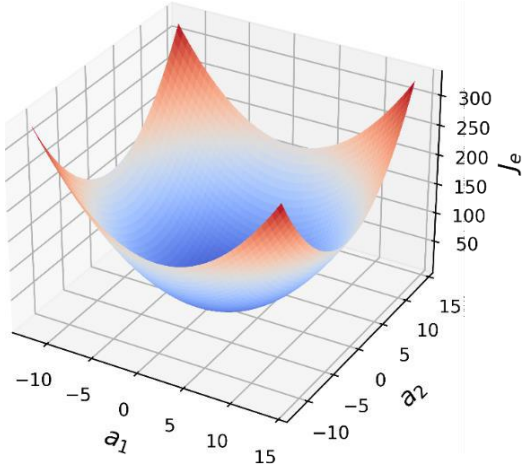
- Utiliza a cada iteração de atualização *apenas um exemplo* do conjunto de treinamento para calcular uma *estimativa estocástica do gradiente*.
 - É estocástica pois a cada iteração toma-se *uma amostra aleatória* do conjunto de treinamento para calcular a estimativa do vetor gradiente.
- Por usar uma estimativa, *não segue diretamente a direção de máxima declividade da função de erro, mudando de direção várias vezes*.

Gradiente descendente estocástico

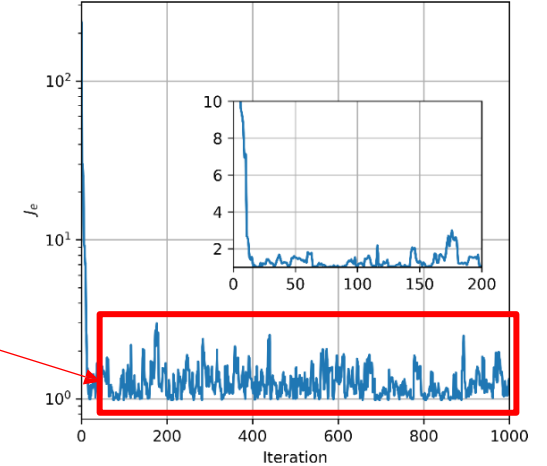
$$\mathbf{a} = \mathbf{a} + \alpha[y(n_{\text{random}}) - \hat{y}(n_{\text{random}})]\mathbf{x}(n_{\text{random}}),$$

- Quando os *dados de treinamento estão contaminados com ruído*, a *estimativa* do gradiente é *ruidosa*, fazendo com que a *convergência não ocorra* ou *não seja garantida*.
 - O algoritmo pode *oscilar* em torno do ponto de mínimo *sem nunca convergir*.
- Entretanto, é mais rápido, usando menos CPU e memória do que o GDB.

Características do GD estocástico



Não
converge



- ***Apresenta um caminho *irregular* para o mínimo, mudando de direção várias vezes.***
- ***Quando as *amostras contém ruído (caso acima), não converge para o ponto de mínimo, “oscilando” em torno dele.****
 - Essa oscilação também pode ser vista na curva de erro.
- Algumas técnicas podem ser usadas para torná-lo mais comportado e talvez convergir: redução do passo, *early-stop*, momentum, etc.

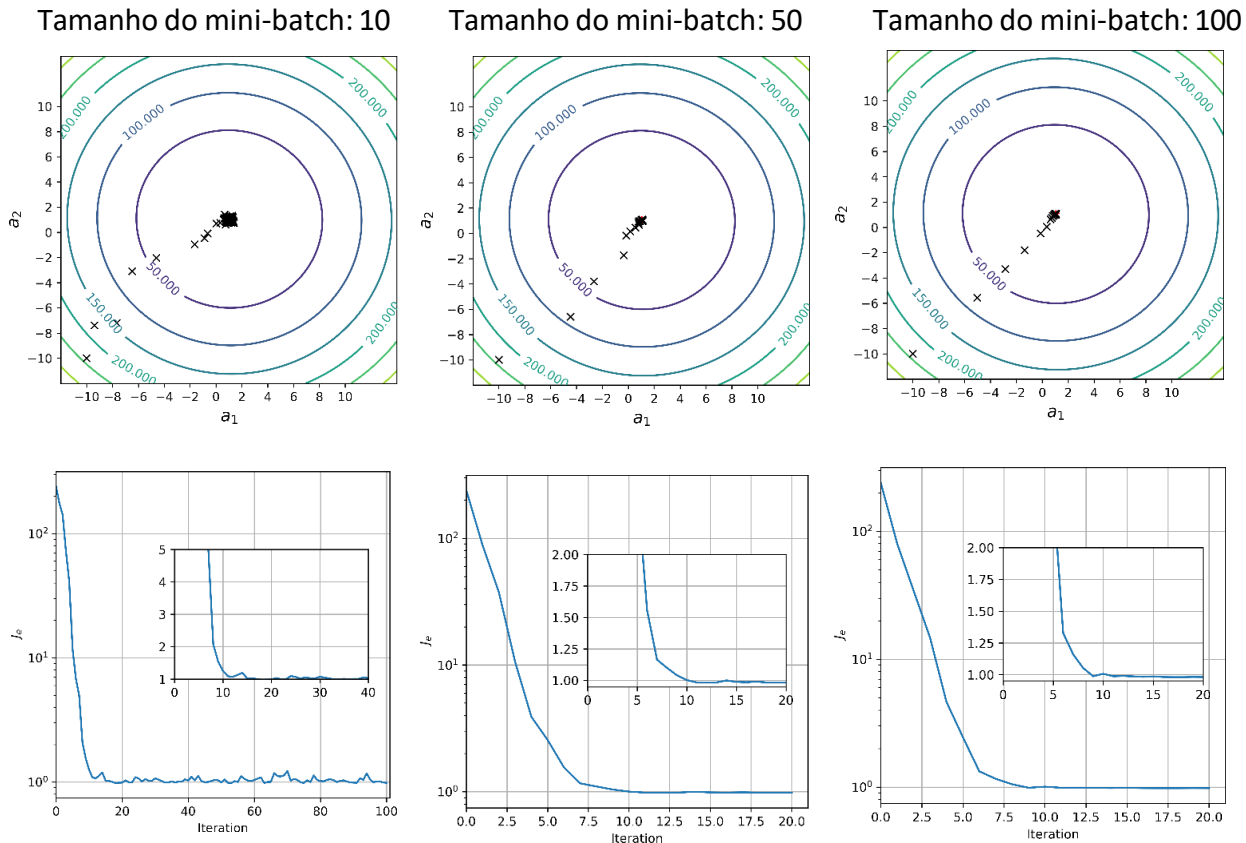
Gradiente descendente em mini-lotes

$$\mathbf{a} = \mathbf{a} + \alpha \frac{2}{MB} \sum_{n=0}^{MB-1} [y(n) - \hat{y}(n)] \mathbf{x}(n) = \mathbf{a} + \alpha \frac{2}{MB} \mathbf{X}_{MB}^T (\mathbf{y}_{MB} - \hat{\mathbf{y}}_{MB}),$$

Contém apenas as amostras do Mini-batch

- Utiliza a cada iteração um **subconjunto aleatório de exemplos**, de tamanho MB , do conjunto de treinamento para o cálculo do gradiente.
- Em geral, $1 < MB < N$, portanto é mais rápido que o GDB e mais preciso e estável do que o GDE.
- Porém, por MB ser variável, essa versão é vista como uma **generalização** das duas versões anteriores, pois MB pode ser feito igual a 1 ou N .
- Em caso de **amostras ruidosas**, a convergência depende do tamanho de MB , **quanto maior, melhor é a estimativa** do vetor gradiente e, consequentemente, **maior a chance de convergência**.

Características do GD em mini-lotes



- Vejam que conforme MB aumenta,
 - o **progresso se torna menos irregular** do que o do GDE,
 - e a **oscilação** (i.e., ruído) ao redor do ponto de mínimo **diminui**.
- Tem comportamento mais próximo do GD em batelada para mini-lotes maiores.
- Para MB pequenos, pode se beneficiar de técnicas para torná-lo mais comportado e talvez convergir.
 - Essas técnicas podem ajudar a **balancear rapidez e convergência**.

Para casa: implementação do GD em mini-batches

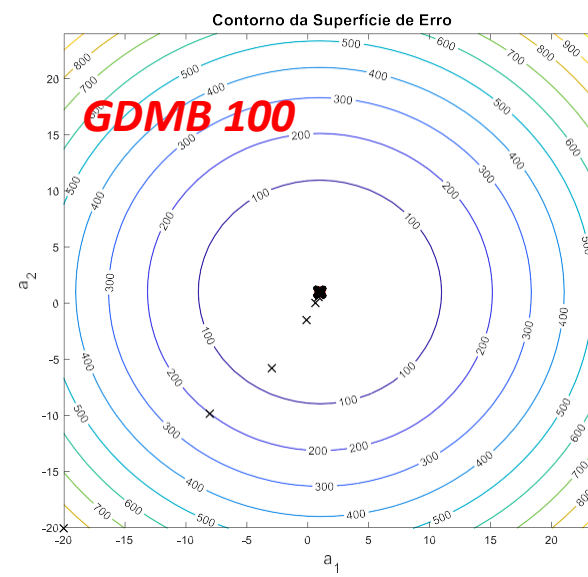
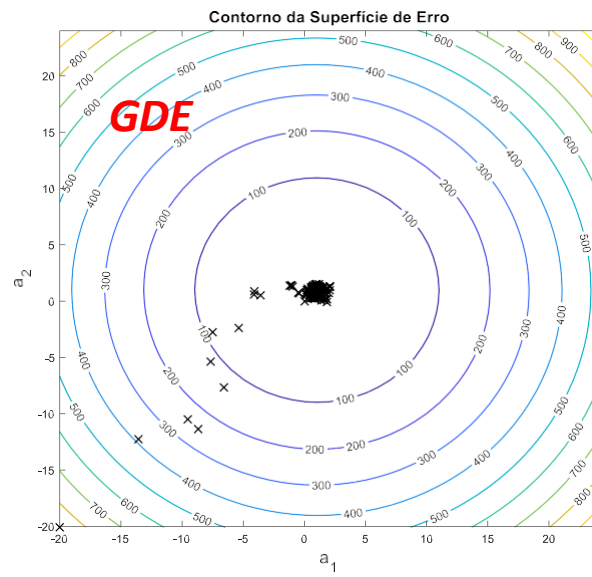
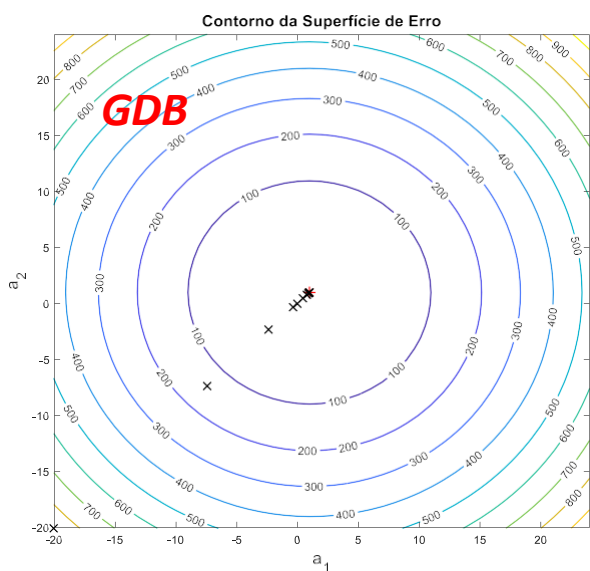
Pseudo-código

1. Inicializa o vetor de pesos $\mathbf{a}$
2. for each época $e \in [1, \dots, E]$:
 1. Embaralhe D
2. for each iteração $i \in [1, \dots, I]$
 1. Retire MB pares $\mathbf{x}$ e $\mathbf{y}$ **sem reposição** de D
 2. Calcule o vetor gradiente $\nabla J_e(\mathbf{a})$ usando MB amostras
 3. Atualize os pesos: $\mathbf{a} \leftarrow \mathbf{a} - \alpha \nabla J_e(\mathbf{a})$
 4. Calcule o erro

Onde E é o **número total de épocas**, D é o **conjunto de treinamento** (i.e., $\mathbf{X}$ e $\mathbf{y}$) com N exemplos, I é o **número de iterações necessárias para se completar uma época** e MB é o **tamanho do mini-batch**.

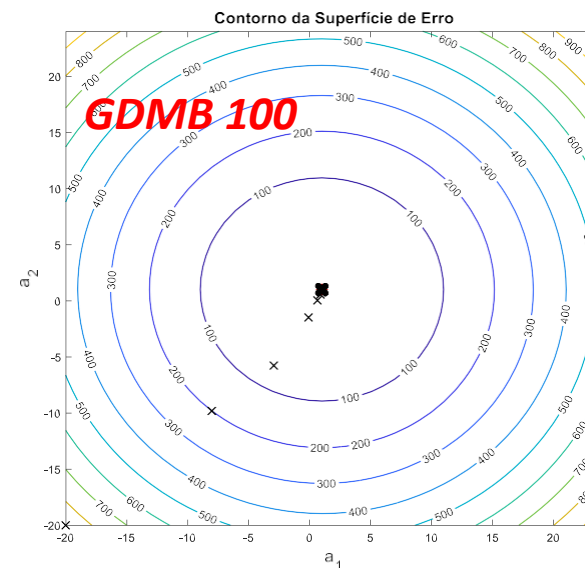
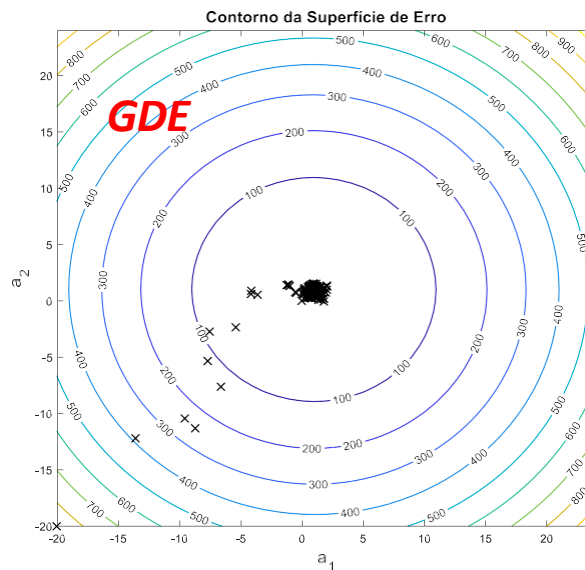
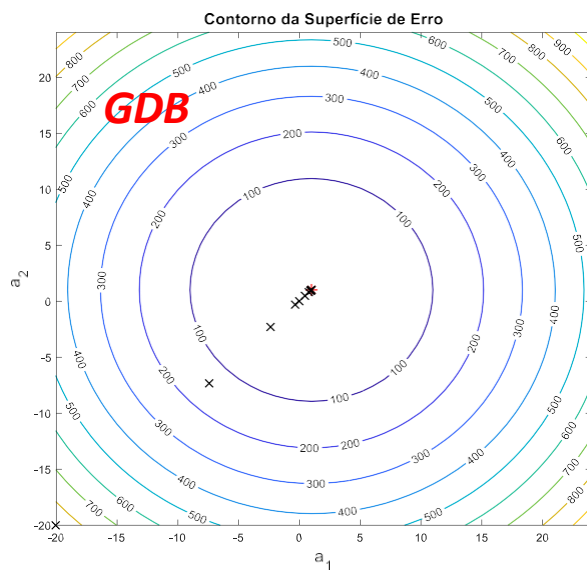
Comparação das versões de GD

- **Todas versões se aproximam do mínimo**, mas o **GDB** caminha diretamente para lá e se estabiliza (i.e., converge).
- Enquanto que o **GDE** e o **mini-batch** (com poucas amostras) oscilam ao redor do mínimo
 - Devido a estimativa ruidosa do gradiente.



Comparação das versões de GD

- O progresso do **MB** é menos irregular do que o do **GDE**, mas depende do tamanho do *mini-batch*.
- Porém, o **GDB** *consome mais memória e tempo* (i.e., CPU) do que o **GDE** e o **MB**, que também *podem* convergir se aplicarmos uma *estratégia adequada de ajuste do passo de aprendizagem*.



Escolha do passo de aprendizagem

$$\mathbf{a} \leftarrow \mathbf{a} - \alpha \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}}$$

- O passo de aprendizagem **controla o quão "grande" ou "pequena" é a atualização aplicada aos pesos** do modelo em cada iteração do processo de treinamento.
- Ou seja, ele determina o **tamanho do passo dado na direção oposta à indicada pelo vetor gradiente**.
- **Porém, qual deve ser o tamanho desse passo?**

Portanto, como veremos, a escolha do passo de aprendizagem é muito importante para o aprendizado de um modelo de ML.

Escolha do passo de aprendizagem

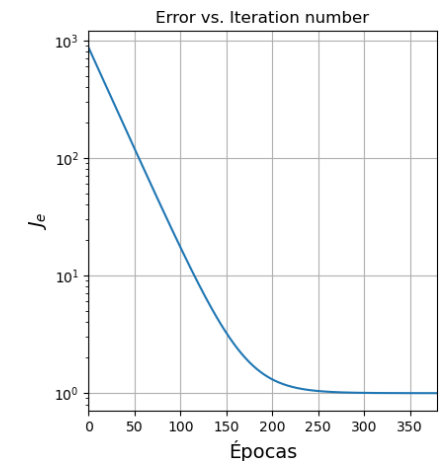
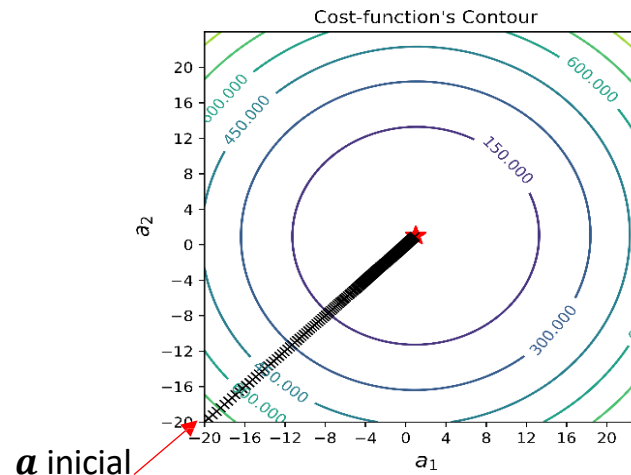
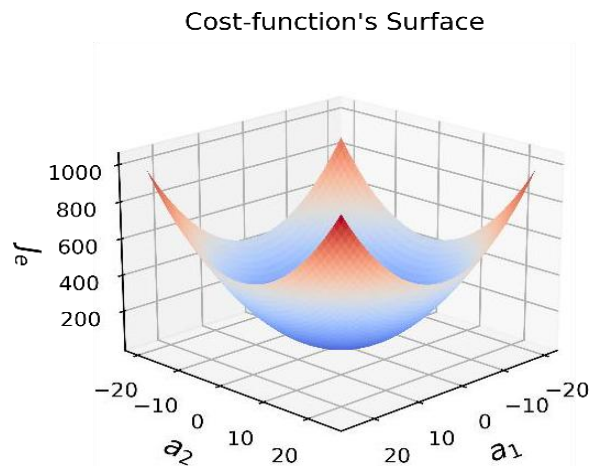
- O passo de aprendizagem é um **hiperparâmetro** que influencia diretamente o **desempenho e a convergência** do algoritmo do gradiente descendente.
 - **Hiperparâmetros**: são **parâmetros que não são aprendidos durante o treinamento** do modelo, mas que influenciam seu aprendizado.
- Valores **muito pequenos** podem resultar em **treinamento lento**, enquanto valores **muito grandes** podem causar **divergência**.
- Em geral, a escolha do passo é feita empiricamente por meio de experimentação.
- Uma regra empírica para **exploração** do passo de aprendizagem é usar a seguinte sequência (**ajuste manual**):

..., 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1.0, ...

$3 \times$ $\approx 3 \times$ $3 \times$ $\approx 3 \times$ $3 \times$ $\approx 3 \times$

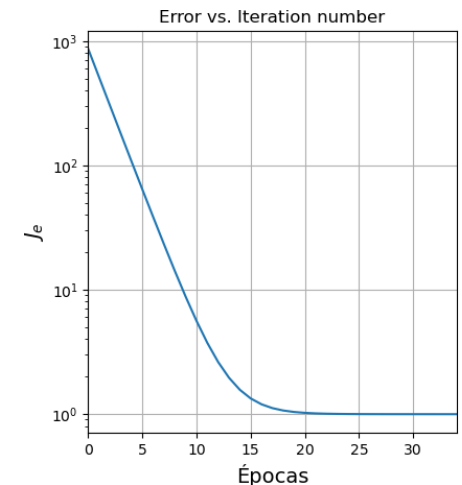
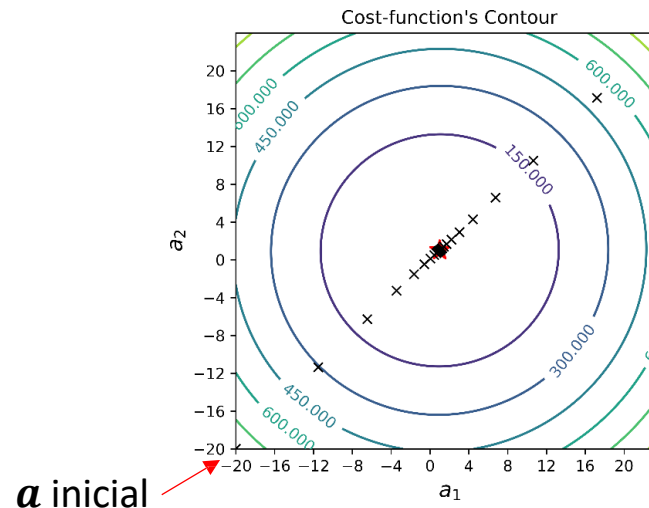
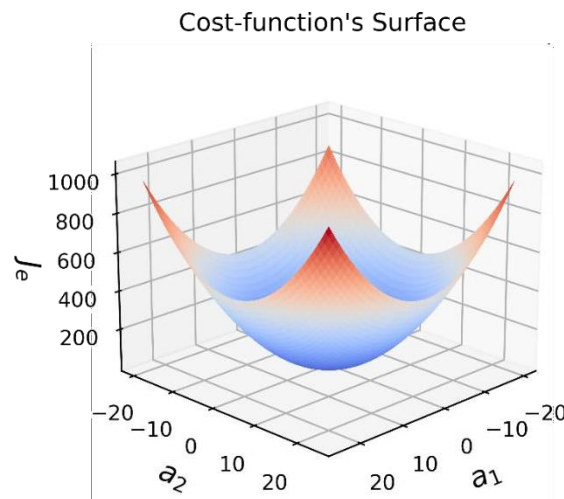
Passo de aprendizado pequeno

- Caso o passo de aprendizagem seja **muito pequeno**, a **convergência do algoritmo será muito lenta**.
- No exemplo abaixo, com $\alpha = 2 \times 10^{-6}$, o algoritmo atinge o ponto de mínimo, i.e., converge, após mais de 250 épocas.
 - **Passos muito curtos**, fazem com que o algoritmo **caminhe vagarosamente em direção ao mínimo global da função de erro**.



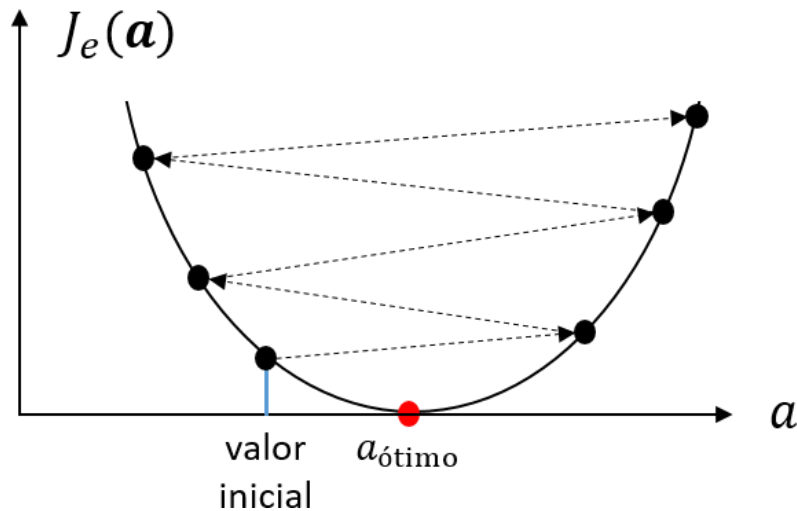
Passo de aprendizado grande

- Caso o passo seja grande, o algoritmo pode nunca convergir.
- Se o passo for grande, *mas não tão grande assim*, o algoritmo pode ficar “*pulando*” ou “*oscilando*” de um lado para o outro da superfície de erro até que, por sorte, ele converge.
 - No exemplo abaixo, com $\alpha = 1.8 \times 10^{-4}$, o algoritmo oscila inicialmente, mas acaba convergindo após 20 épocas.



Passo de aprendizado grande

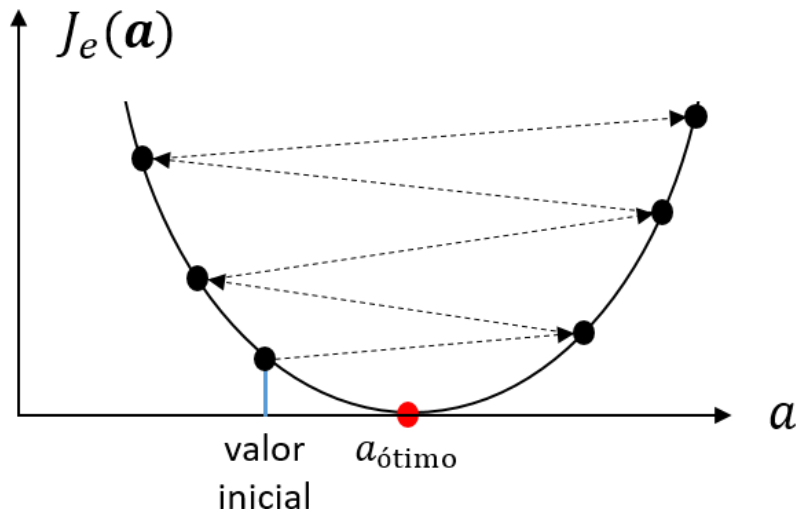
feedback positivo → estouro da precisão numérica



- Em outros casos, quando o passo é ***muito grande*** , a cada época, o algoritmo “pula” para um valor mais alto do que o anterior e, assim, acaba divergindo.
- Ou seja, ao invés de se aproximar do ponto de mínimo a cada época, ele se distancia dele.

Passo de aprendizado grande

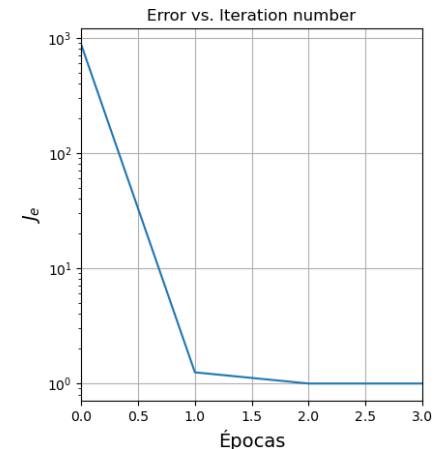
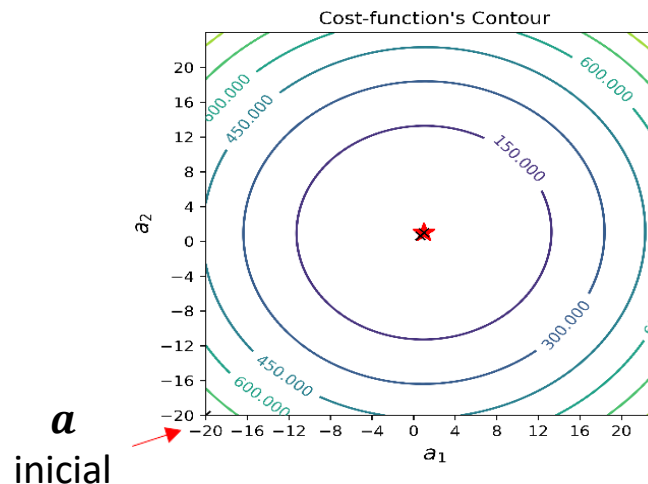
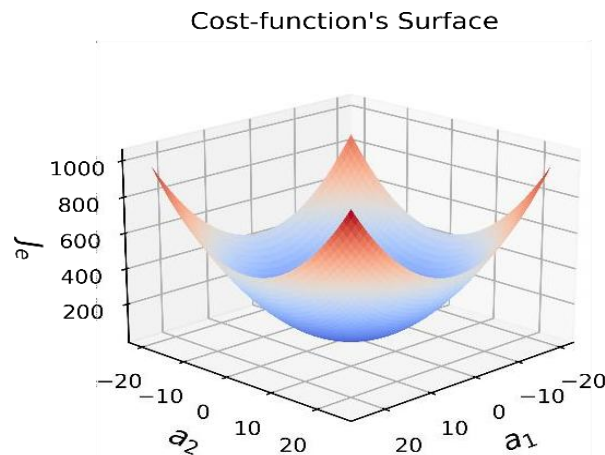
feedback positivo → estouro da precisão numérica



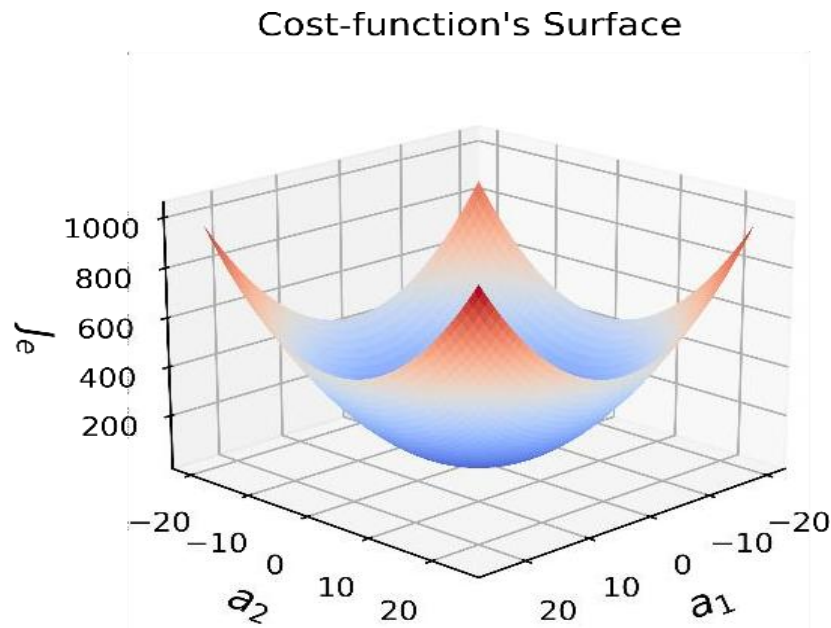
- Nesse caso ocorre um **ciclo de feedback positivo** onde **a cada época** os valores dos **gradientes** e, conseqüentemente, dos **pesos se tornam maiores e maiores** até que ocorra o **estouro da representação numérica**.
 - Problema que ocorre quando uma variável não pode mais representar um valor, pois ele é maior do que o intervalo que ela pode armazenar.

Passo de aprendizado ideal

- Portanto, o valor do passo de aprendizagem deve ser **explorado** para se encontrar um **valor ideal** que **acelere a convergência** de forma **estável**, ou seja, sem oscilações.
- O exemplo abaixo, com $\alpha = 10^{-4}$, o algoritmo converge de forma estável para o **mínimo global** em apenas 3 épocas.

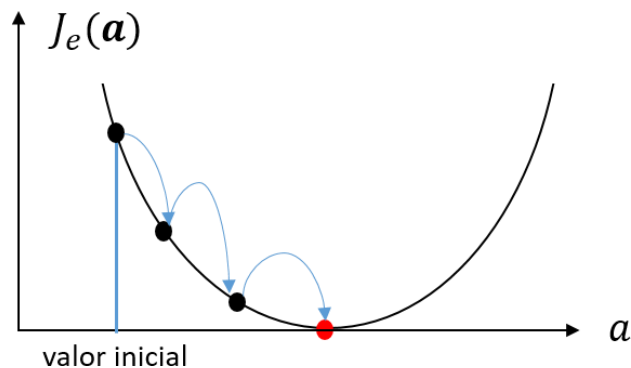
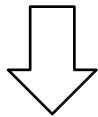
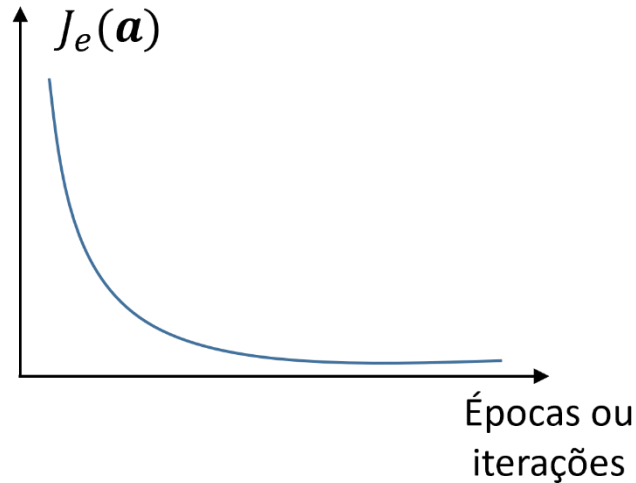


Analizando o treinamento de um modelo



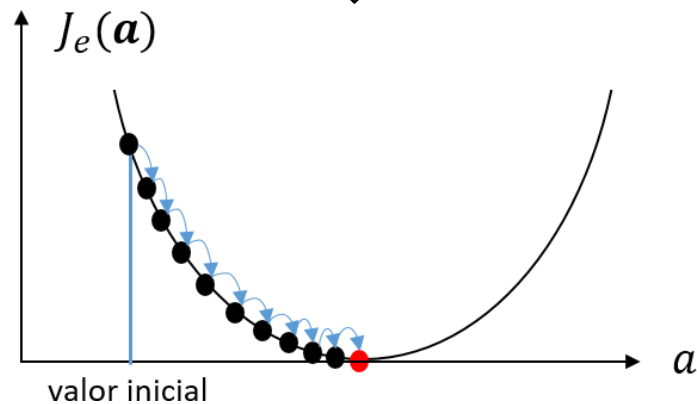
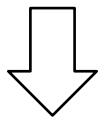
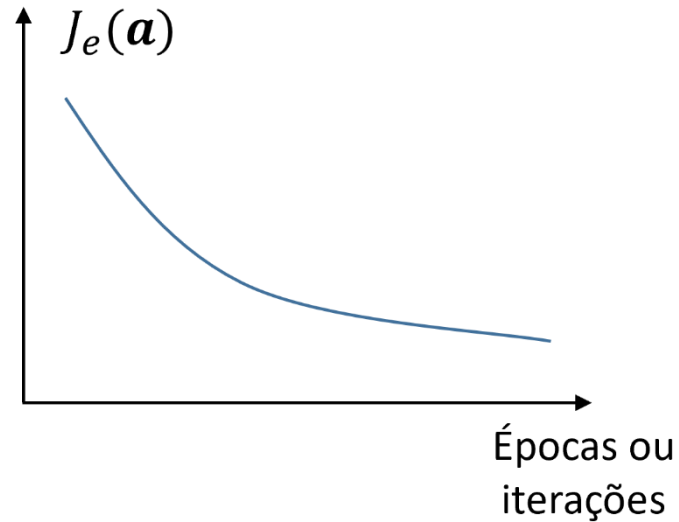
- *Nem sempre iremos conseguir plotar a superfície de erro e de contorno* para analisarmos o treinamento e o desempenho de um modelo.
- Por exemplo, quando tivermos *três atributos*, a *superfície de erro terá quatro dimensões*, tornando sua análise mais difícil.
- Assim, em geral, usamos a *curva do erro (i.e., EQM) em função das iterações* de treinamento para analisar o aprendizado de um modelo.

Analizando o treinamento de um modelo



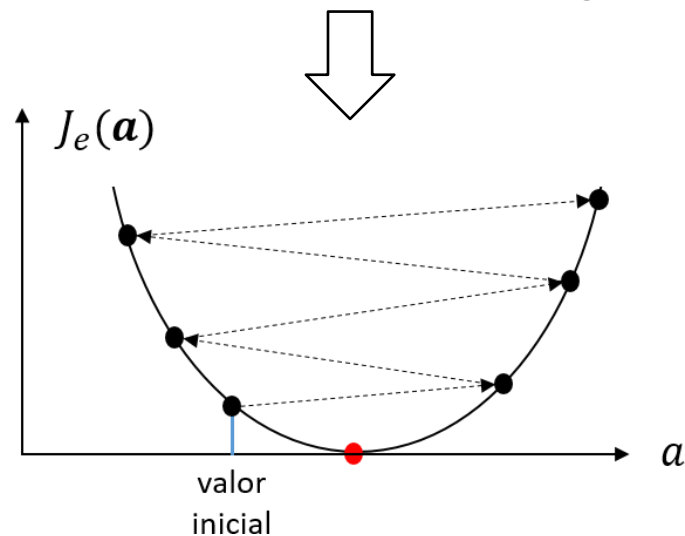
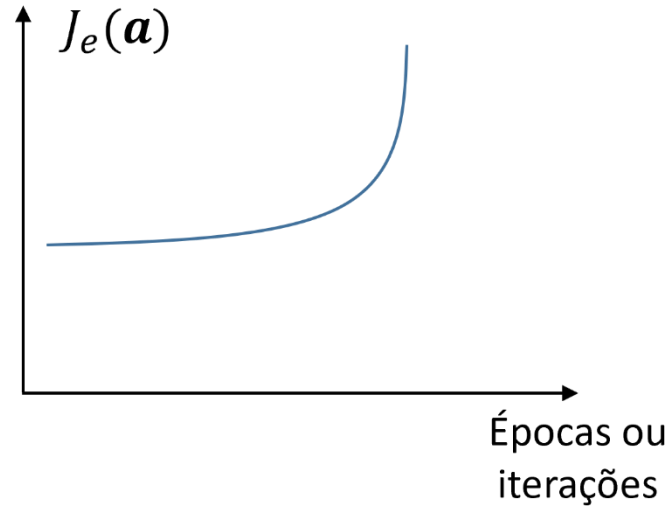
- A figura ao lado mostra o **comportamento esperado** quando o **passo tem o tamanho ideal**.
- A **convergência** nesse caso é **rápida**.
 - O **erro diminui rapidamente nas primeiras iterações**.
 - Conforme o treinamento continua, o **erro se estabiliza e exibe uma redução suave** (i.e., mais lenta).
 - A **convergência é atingida quando o erro se torna praticamente constante** ao longo das iterações, indicando que os **pesos não sofrem grandes atualizações**, pois o mínimo da função foi atingido.
 - Por exemplo, o treinamento pode ser encerrado quando o erro entre duas épocas consecutivas for menor do que um valor pré-definido.

Analizando o treinamento de um modelo



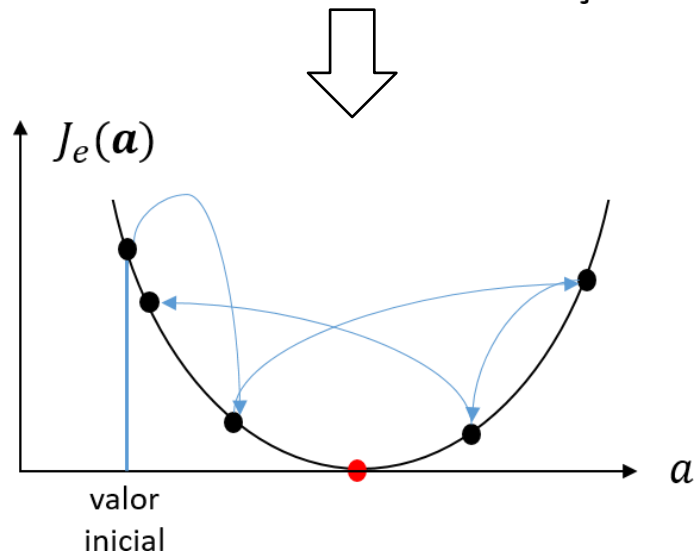
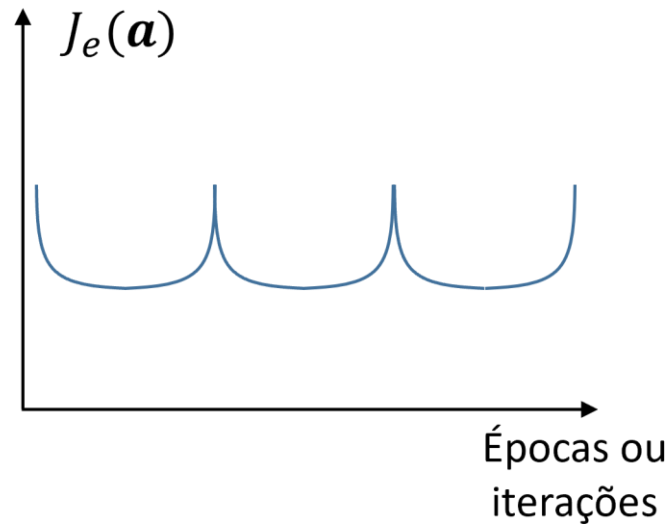
- A figura mostra o caso onde o *passo de aprendizagem é muito pequeno*.
- Nesse caso, a *convergência é muito lenta*.
- *Após várias épocas* de treinamento, o *erro ainda não se estabilizou*.
- Levaria *muito tempo* para que o modelo atingisse o *ponto de mínimo*.

Analizando o treinamento de um modelo



- A figura mostra o caso onde o *passo de aprendizagem é muito grande*.
- Nesse caso, ocorre *divergência*.
- Ou seja, o *erro aumenta mais e mais ao longo do treinamento*, indicando que o algoritmo está se *distanciando do ponto de mínimo*.
- Se o treinamento continuar, os gradientes e pesos podem se tornar tão grandes que ocorre o *estouro da representação numérica*.

Analizando o treinamento de um modelo



- A figura mostra o caso onde o *passo de aprendizagem é grande, mas não tão grande assim*.
- Nesse caso, o *erro oscila* entre valores grandes e pequenos.
- Por ventura, a *convergência pode ocorrer após algumas épocas*.

Como configurar o passo de aprendizagem?

As abordagens mais usadas para se configurar o passo de aprendizagem são:

- **Ajuste manual:** envolve a escolha de valores através de *tentativa e erro*.
- **Redução programada:** *diminuição gradual do passo de aprendizagem* ao longo das iterações de treinamento.

Existem também outras abordagens para configuração do passo de aprendizagem que **não serão tratadas na nossa disciplina**:

- **Variação adaptativa:** adapta *os passos* de aprendizagem *de acordo com as estatísticas do vetor gradiente*.
- **Termo momentum:** usa a *média do histórico de gradientes* para atualizar os pesos.

Ajuste manual do passo de aprendizagem

- Como o **valor ideal do passo de aprendizagem** não é conhecido à priori, podemos encontrá-lo através do **ajuste manual**.
- O ajuste é feito através de **tentativa e erro** até que um valor bom o suficiente seja encontrado.
- Uma forma programática de encontrar o passo consiste em realizar uma **busca em grade (grid search)**.
 - **Grid search** é um método que faz a **busca exaustiva** pelo valor ótimo de um ou mais **hiperparâmetros** de um algoritmo de aprendizagem.

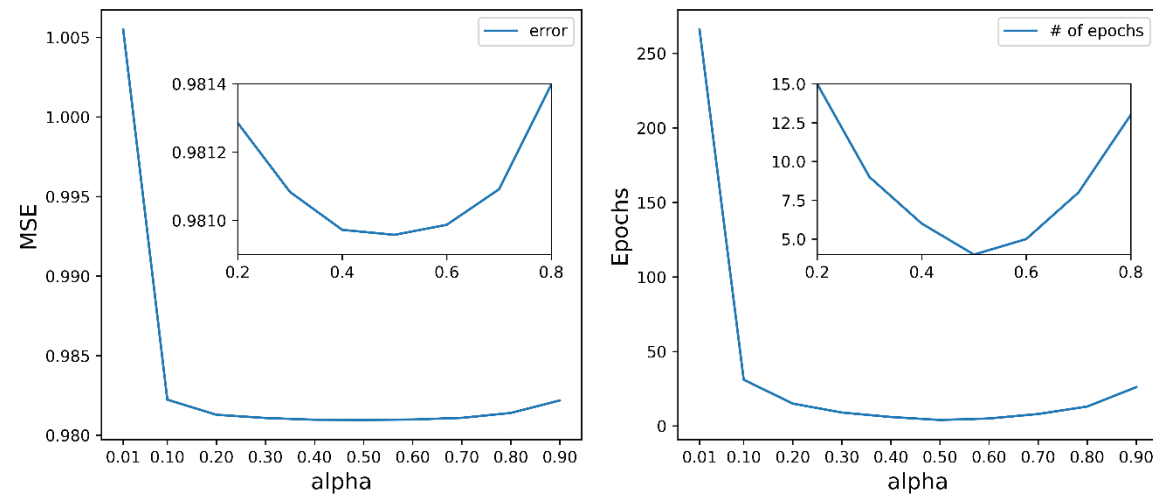
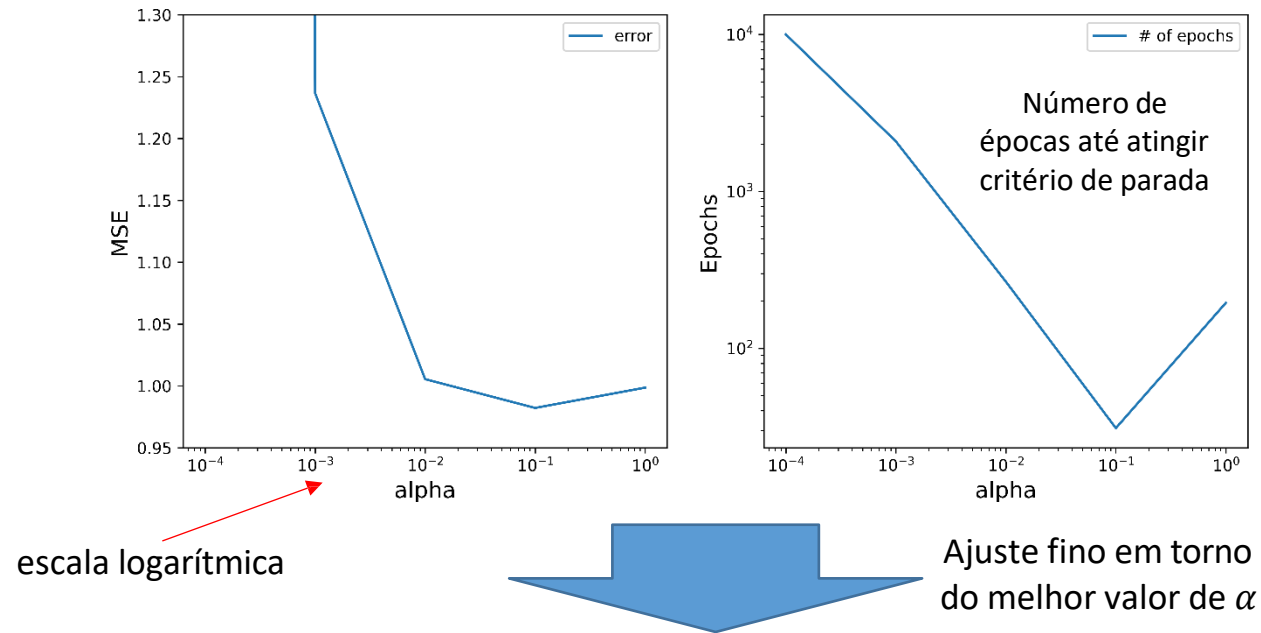
Ajuste manual do passo de aprendizagem

- A ***busca em grade*** nos ajuda a encontrar uma ordem de magnitude onde podem residir bons passos de aprendizagem.
- Típicamente, a ***busca em grade*** é iniciada com valores em uma escala ***logarítmica*** (***ajuste grosso***), por exemplo

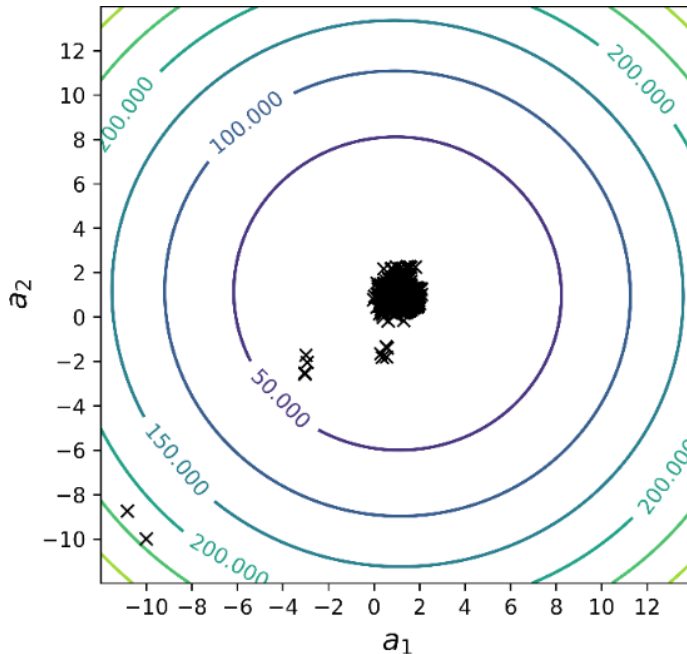
$$\{1, 10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\} \quad .$$

- Em seguida, faz se um ***ajuste fino*** em torno do melhor valor encontrado durante a busca logarítmica.
- Depois de escolhido, o valor do passo é sempre constante.

Exemplo: ajuste manual de α com *grid search*



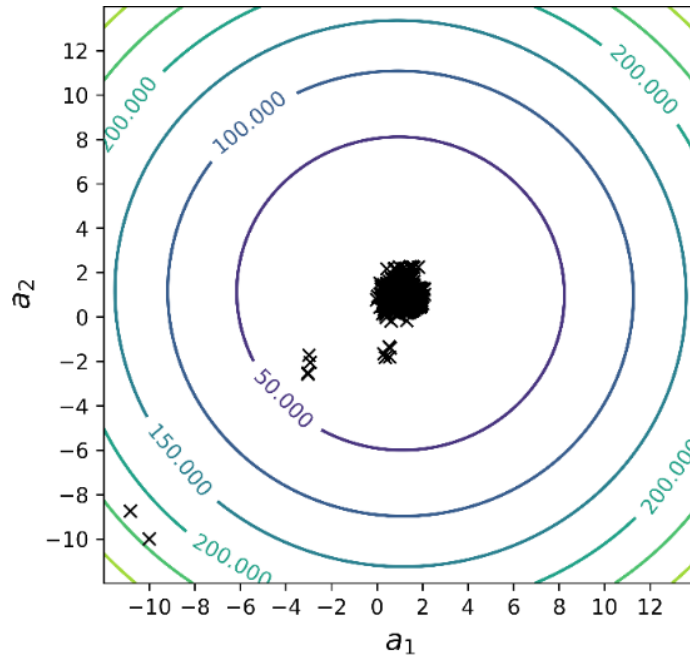
Melhorando a convergência das versões estocásticas



Gradiente descendente estocástico (SGD)

- As **versões estocásticas** do gradiente descendente, i.e., **GDE** e **mini-batch** (principalmente quando MB é pequeno), têm um **caminho irregular até o ponto de mínimo**.
- Além disso, quando as **amostras** do conjunto de treinamento estão **contaminadas com ruído**, eles **podem não convergir** para o mínimo (i.e., oscilam ao redor dele).

Melhorando a convergência das versões estocásticas



Gradiente descendente estocástico (SGD)

- Esses problemas *impactam* o *desempenho do modelo* e deixam o *treinamento lento* e, possivelmente, *instável*.
- Entretanto, existem *técnicas para minimizar* esses problemas, deixando essas versões do GD *mais comportadas*.
- As mais conhecidas envolvem o *ajuste do passo de aprendizagem* e/ou do *termo de atualização dos pesos*.

Ajuste do passo de aprendizagem

- *Redução gradual* (ou decaimento) do passo de aprendizagem *diminui gradualmente o passo de aprendizagem* ao longo do treinamento.
- A redução da taxa de aprendizagem faz com que as *atualizações dos pesos se tornem cada vez menores* à medida que o treinamento progride, o que pode *melhorar (ou forçar) a convergência*.

$$\mathbf{a}(i + 1) = \mathbf{a}(i) - \boxed{\alpha(i)} \nabla \hat{J}_e(\mathbf{a}(i))$$

onde i é número da iteração de atualização atual e $\nabla \hat{J}_e(\mathbf{a}(i))$ é a *estimativa estocástica do vetor gradiente*.

- Essa é uma *técnica muito simples, mas precisamos encontrar os hiperparâmetros* que dão a *taxa ideal de redução* do passo de aprendizagem.
- Veremos a seguir um exemplo de como ela funciona.

Técnicas mais comuns para a redução gradual

- As três técnicas mais comuns para a **redução gradual** do passo de aprendizagem são:
 - **Dcaimento por etapas ou degraus:** é dado pela equação $\alpha(i) = \alpha_0 \times \tau^{\lfloor \frac{i+1}{\beta} \rfloor}$. Reduz o passo de aprendizagem inicial, α_0 , de um fator, τ , a cada número pré-definido de iterações, β . Um valor típico para reduzir a taxa de aprendizado é de $\tau = 0.5$ a cada β de iterações.
 - **Dcaimento exponencial:** é dado pela equação $\alpha(i) = \alpha_0 e^{-ki}$, onde α_0 , k e i são passo de aprendizagem inicial, a taxa de decrescimento e o número da iteração de atualização atual, respectivamente.
 - **Dcaimento temporal:** é dado pela equação $\alpha(i) = \alpha_0 / (1 + ki)$ onde α_0 , k e i têm o mesmo significado que no decaimento exponencial.
- Entretanto, percebam que ***ainda temos que encontrar os valores ideais para os hiperparâmetros α_0 , τ , β e k .***

Ajuste do passo por redução programada

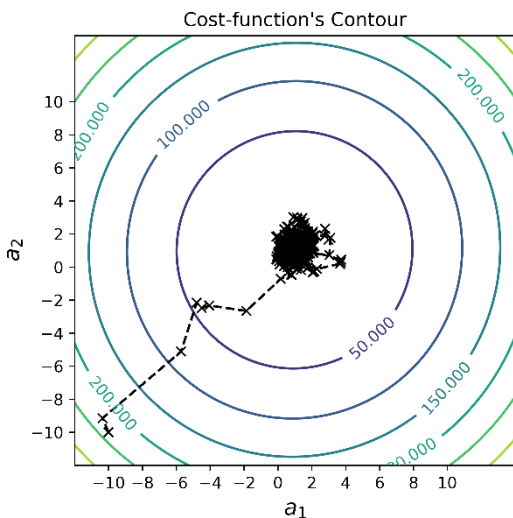
- Os valores dos ***hiperparâmetros*** precisam ser encontrados para evitar a ***diminuição***:
 - ***lenta de um passo grande***: o algoritmo ***perde tempo saltando caoticamente*** de um lado para o outro da superfície, com pouca melhoria no aprendizado por um longo tempo.
 - ***agressiva do passo***: o ***modelo para de aprender rapidamente***, pois o passo fará com que a atualização diminua rapidamente, incapacitando o algoritmo de alcançar o mínimo.
- Portanto, cai-se novamente no ***problema da tentativa e erro***, o qual é muitas vezes custoso.

Ajuste do passo por redução programada

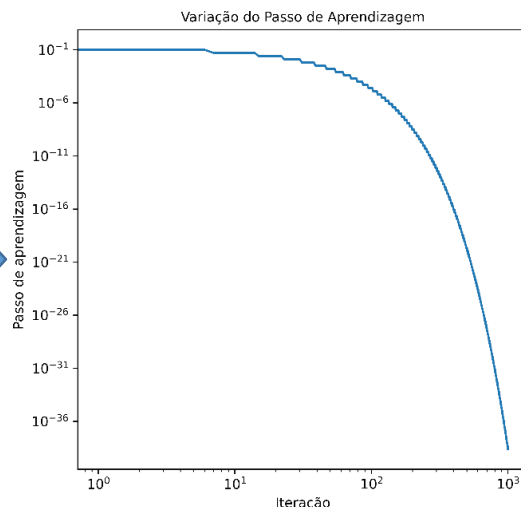
- Outra ***desvantagem*** é que ***o mesmo passo de aprendizagem é aplicado ao ajuste de todos os pesos*** (assim como no ajuste manual).
 - Se a ***inclinação da superfície de erro é muito grande*** em uma direção, o elemento correspondente do vetor gradiente terá um valor muito grande, o que pode levar o algoritmo a dar passos muito grandes e divergir (ou oscilar).
 - Por outro lado, se a ***inclinação for muito pequena***, o elemento do gradiente terá um valor muito pequeno, o que pode levar o algoritmo a dar passos muito pequenos e demorar para convergir.

Exemplo de redução programada com GDE

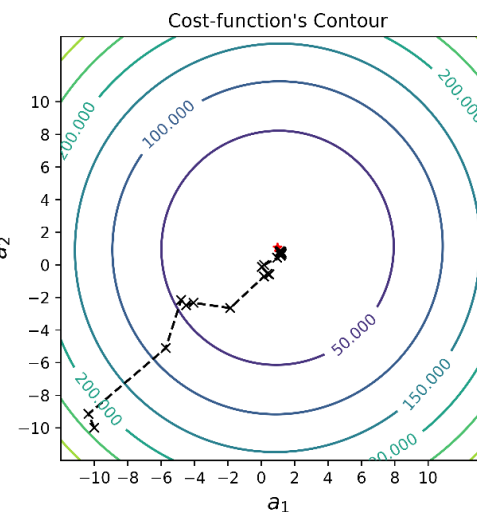
Sem redução gradual



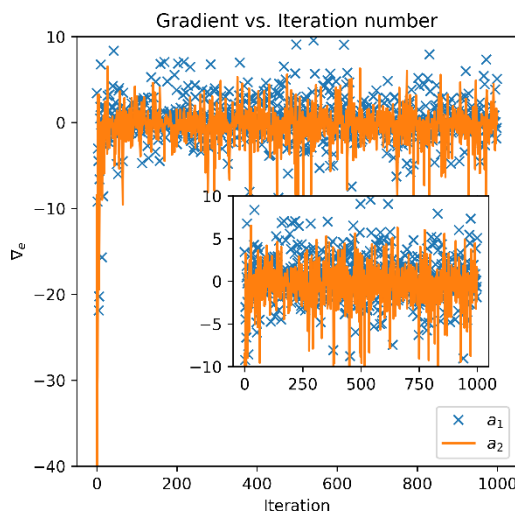
Redução gradual por degraus



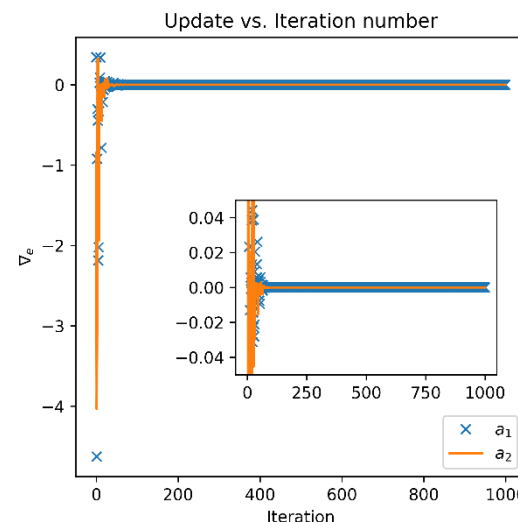
Com redução gradual



- O caminho com ***decaimento gradual*** também não é regular para o ponto de mínimo.
- Ele apresenta ***algumas mudanças de direção*** ao longo do caminho.
- O ***passo não influencia na direção***, apenas no tamanho do deslocamento.

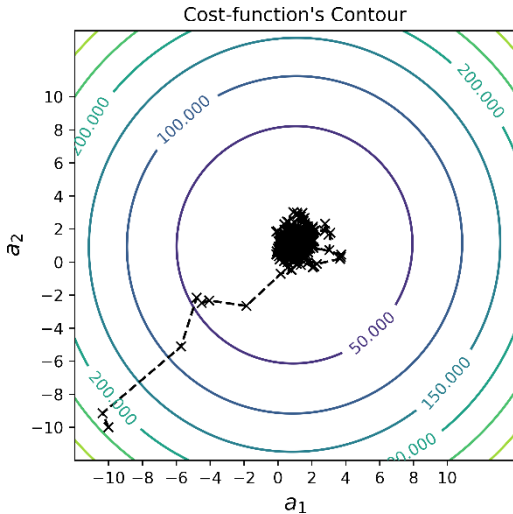


```
# learning schedule: Gradual decay.  
def stepDecay(alpha_init, t, beta=6.0, drop=0.5):  
    """Gradual decay."""  
    alpha = alpha_init * math.pow(drop, math.floor((1+t)/beta))  
    return alpha
```

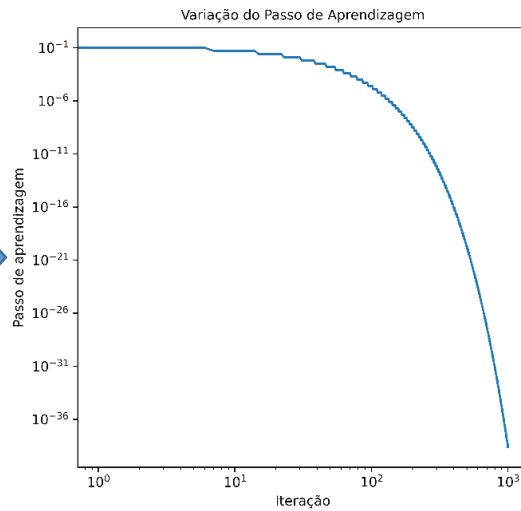


Exemplo de redução programada com GDE

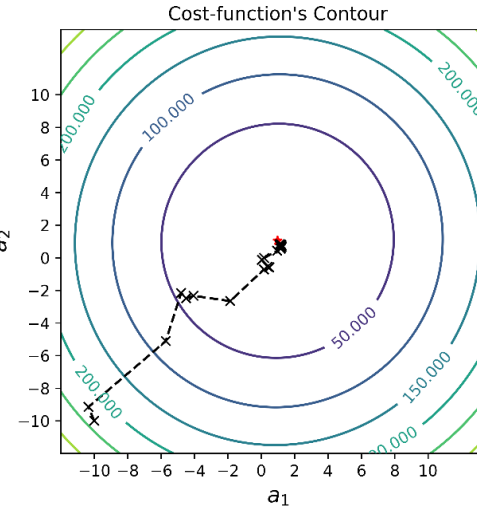
Sem redução gradual



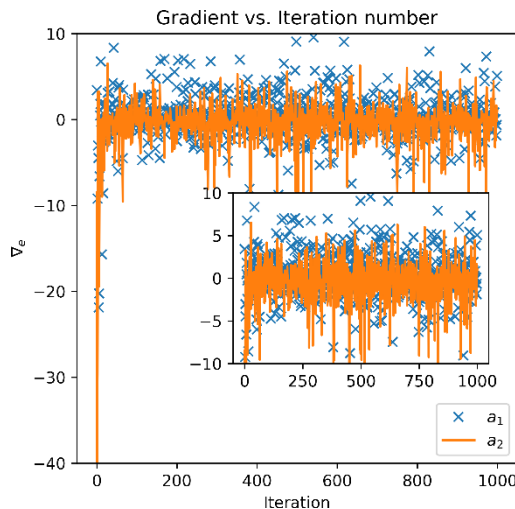
Redução gradual por degraus



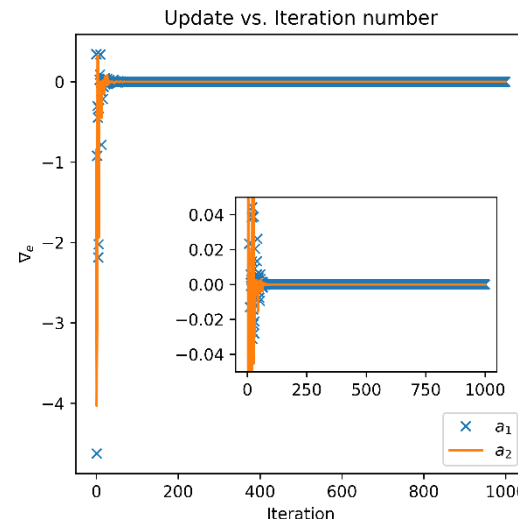
Com redução gradual



- Porém, a **oscilação em torno do mínimo é bastante reduzida** devido à **diminuição gradual** do passo de aprendizagem, α .
- Conseguimos visualizar melhor o efeito da redução de α nas figuras que mostram os elementos do vetor gradiente.

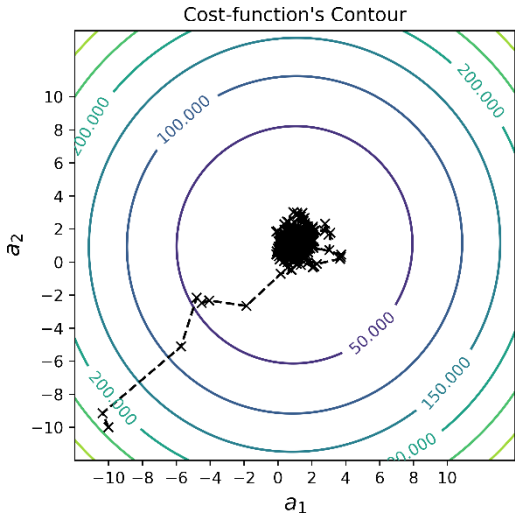


```
# learning schedule: Gradual decay.  
def stepDecay(alpha_init, t, beta=6.0, drop=0.5):  
    """Gradual decay."""  
    alpha = alpha_init * math.pow(drop, math.floor((1+t)/beta))  
    return alpha
```

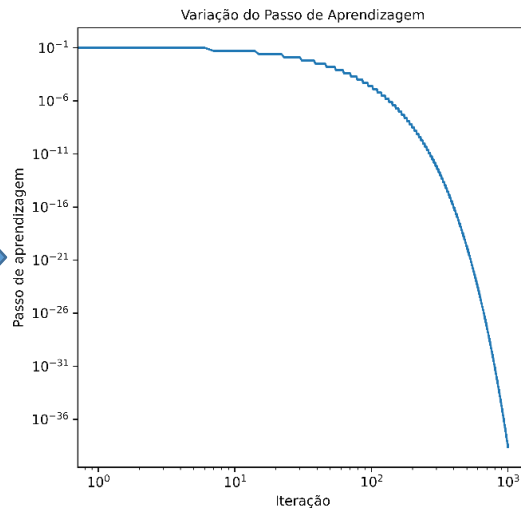


Exemplo de redução programada com GDE

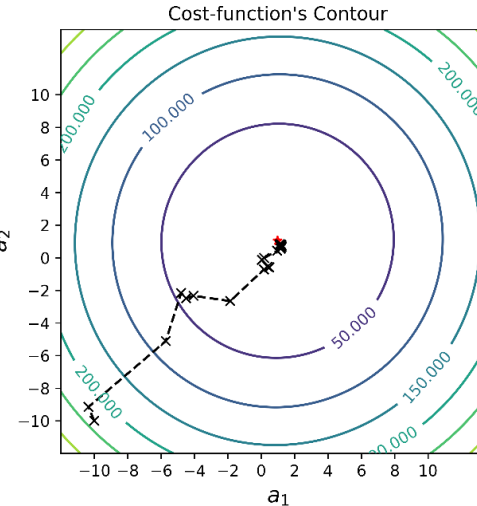
Sem redução gradual



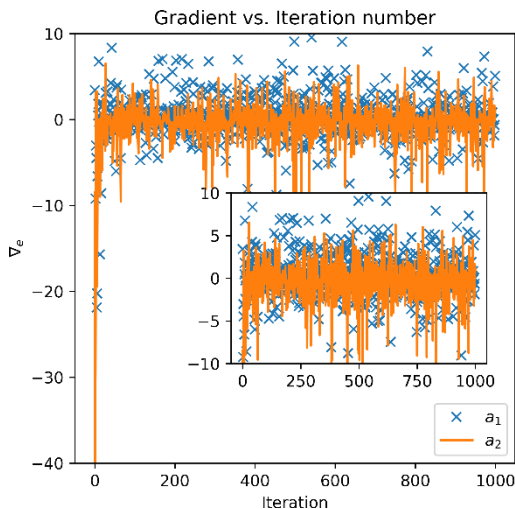
Redução gradual por degraus



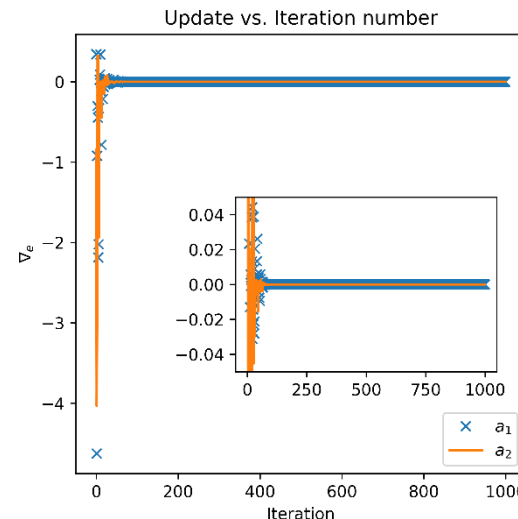
Com redução gradual



- **Conclusão:** um passo de aprendizagem que tem seu valor reduzido ao longo das iterações de treinamento permite que as versões estocásticas do gradiente descendente se estabilizem próximo ao ponto de mínimo global.



```
# learning schedule: Gradual decay.  
def stepDecay(alpha_init, t, beta=6.0, drop=0.5):  
    """Gradual decay."""  
    alpha = alpha_init * math.pow(drop, math.floor((1+t)/beta))  
    return alpha
```



Para casa

- Exercício sobre gradiente descendente estocástico com decaimento exponencial.
 - Exercício #8 da lista #2.

Variações do formato da superfície de erro

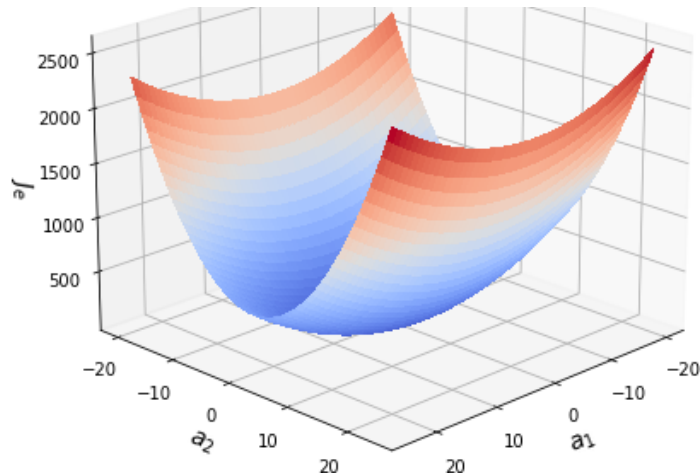
- Como vimos, nem toda superfície de erro criada a partir da função do EQM tem formato de tigela (i.e., com curvas de erro circulares).
- Dependendo dos *intervalos de variação dos atributos*, podemos ter *superfícies com formato de vale*.
- Por exemplo, se o intervalo de valores de $x_1 \gg x_2$, ele *dominará o erro* e fará com que a superfície de erro tenha *formato de vale*.
- Se $x_2(n)$ for importante para o modelo, sua *contribuição será possivelmente ignorada*, levando a resultados tendenciosos e imprecisos.

$$J_e(\mathbf{a}) = \frac{1}{N} \sum_{n=0}^{N-1} \left[y_{\text{noisy}}(n) - \underbrace{(\hat{a}_1 x_1(n) + \hat{a}_2 x_2(n))}_{\text{Função Hipótese}} \right]^2 \quad \approx \frac{1}{N} \sum_{n=0}^{N-1} \left[y_{\text{noisy}}(n) - \hat{a}_1 x_1(n) \right]^2$$

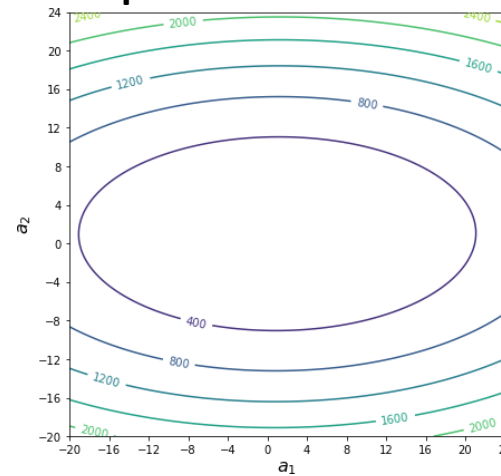
$x_1 \gg x_2$

Superfícies com formato de vale

Superfície de erro



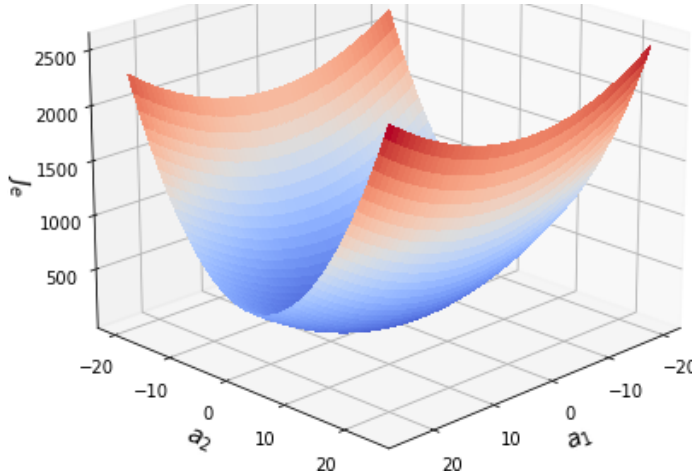
Superfície de contorno



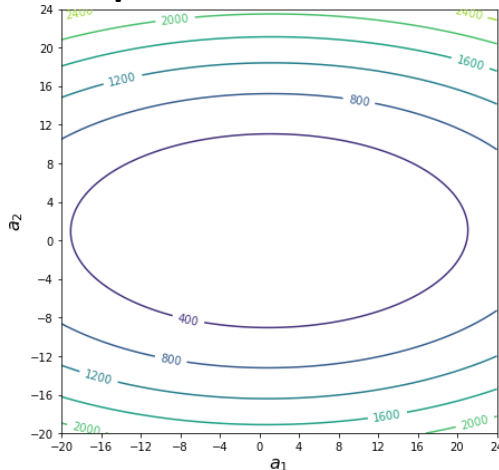
- Superfícies com *formato de vale* fazem com que a *convergência do GD se torne muito lenta*.
- A *convergência* se torna *lenta devido à superfície ser plana ou quase plana* em algumas direções (i.e., inclinação ≈ 0).

Superfícies com formato de vale

Superfície de erro



Superfície de contorno



- Nessas direções, o **gradiente da função de erro é muito pequeno**, tornando as **atualizações dos pesos**, consequentemente, **muito pequenas**.

$$\mathbf{a} \leftarrow \mathbf{a} - \alpha \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}}$$

- Na figura ao lado, as derivadas parciais do EQM em relação ao peso a_1 serão muito pequenas devido à **pequena inclinação da superfície nessa direção**.

O que pode ser feito?

Escalonamento de atributos

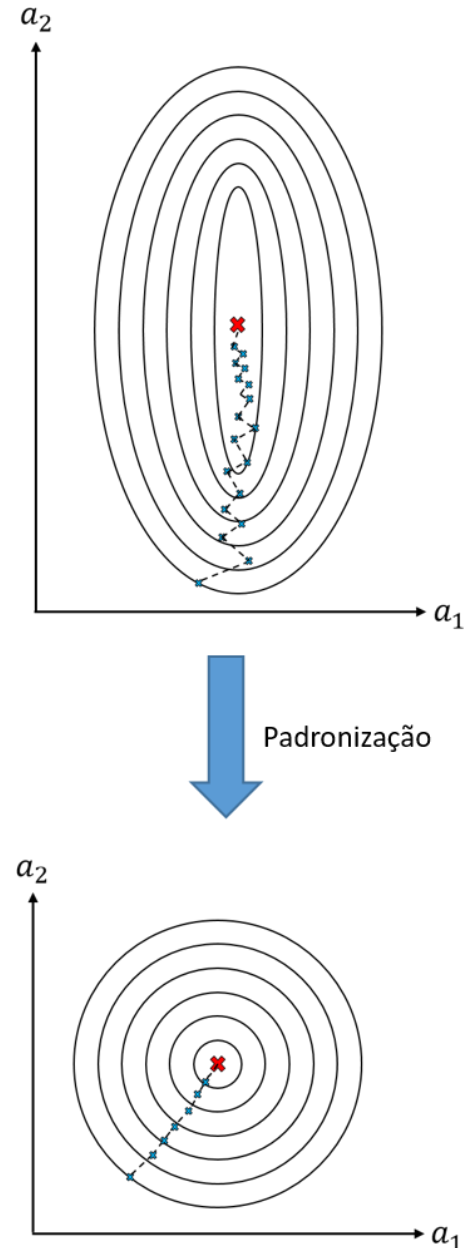
- Para evitar esse problema, o *intervalo de variação de todos os atributos* pode ser *escalonado*, trazendo-os para uma escala similar.
- Assim, *cada atributo influenciará da mesma forma o cálculo do erro*.

$$J_e(\mathbf{a}) = \frac{1}{N} \sum_{n=0}^{N-1} \left[y_{\text{noisy}}(n) - \underbrace{(\hat{a}_1 x_1(n) + \hat{a}_2 x_2(n))}_{\text{Função Hipótese}} \right]^2$$

- Consequentemente, a *superfície se tornará mais circular*, facilitando o aprendizado.
- As duas formas mais comuns de escalonamento são:
 - Normalização Mín-Max
 - Padronização

Escalonamento de atributos

- Na figura ao lado, após o escalonamento, o erro varia similarmente em todas as direções, fazendo com que os elementos do vetor gradiente tenham magnitudes similares.
 - O escalonamento deixa as atualização dos pesos mais uniformes, o que acelera a convergência do GD.
- Assim, o escalonamento ***pode acelerar a convergência*** do GD, pois evita que ele
 - se mova muito lentamente em regiões planas (fundo do vale).
 - ou oscile caoticamente de um lado para o outro do vale, apontando mais na direção do ponto de mínimo do que da máxima declividade.



Escalonamento de atributos

- Em geral, a **normalização mín-max** faz com que os **atributos variem entre 0 e 1**, mas pode-se definir outros intervalos.
- A equação usada para normalizar os atributos é apresentada abaixo

$$x'_k(n) = \frac{x_k(n) - \min(\mathbf{x}_k)}{\max(\mathbf{x}_k) - \min(\mathbf{x}_k)}, \quad 0 \leq x'_k(n) \leq 1,$$

onde x_k representa o k -ésimo atributo, n é o número da amostra, $\min(\mathbf{x}_k)$ e $\max(\mathbf{x}_k)$ são os valores mínimo e máximo, respectivamente, **calculados ao longo de todas as amostras do k -ésimo vetor de atributo, $\mathbf{x}_k$.**

Escalonamento de atributos

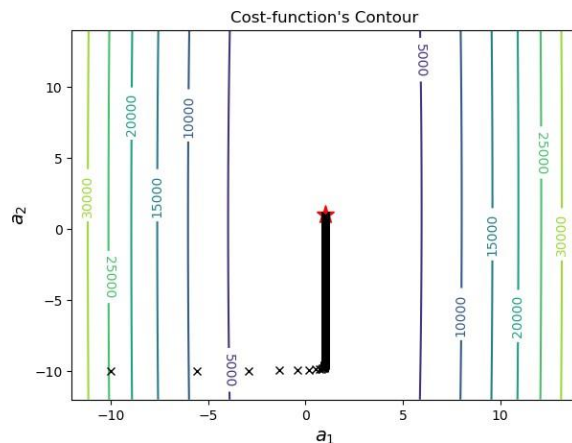
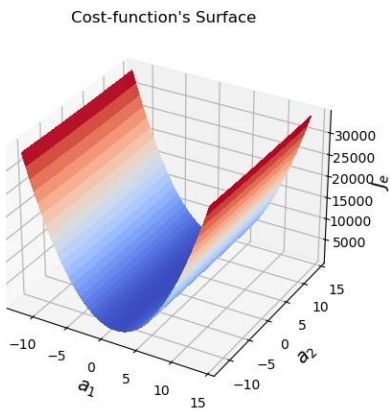
- **Padronização** faz com que os **atributos passem a ter média zero e desvio padrão unitário**.
- Notem que, neste caso, os **valores não ficam restritos a um intervalo específico**.
- A equação usada para padronizar os atributos é apresentada abaixo.

$$x'_k(n) = \frac{x_k(n) - \mu_{x_k}}{\sigma_{x_k}},$$

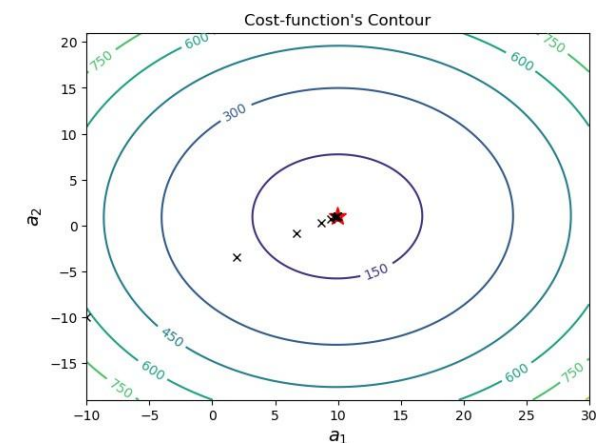
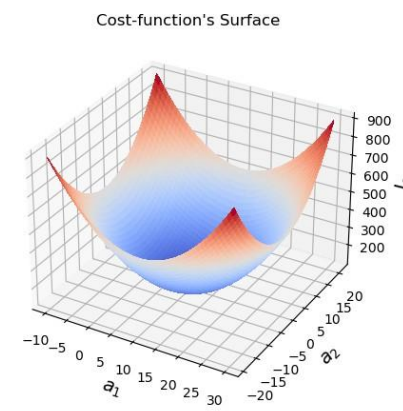
onde x_k representa o k -ésimo atributo, n é o número da amostra, μ_{x_k} e σ_{x_k} são as estimativas da média e do desvio padrão, respectivamente, **calculados ao longo de todas as amostras do k -ésimo vetor de atributo, $\mathbf{x}_k$** .

Vantagens do escalonamento de atributos

- Ajuda a **acelerar a convergência** do GD, pois deixa a **superfície de erro mais circular**, fazendo com que os pesos sejam atualizados de maneira mais uniforme.
 - Pois a inclinação da superfície se torna similar em todas as direções.

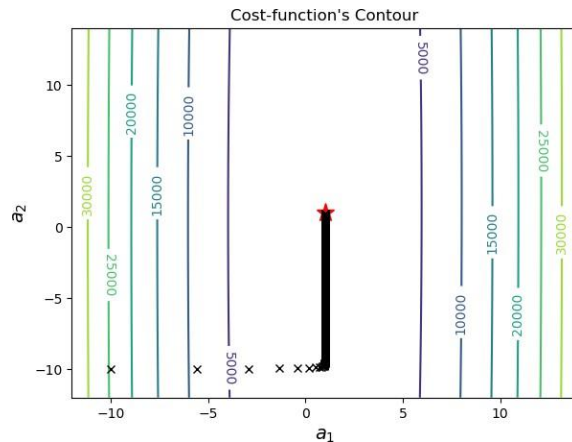
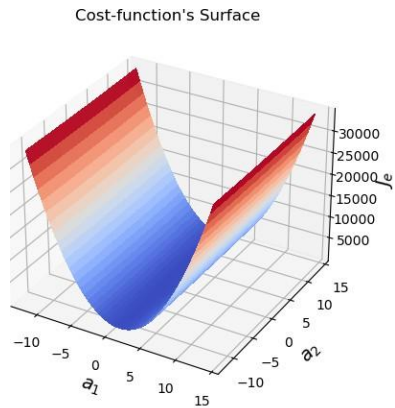


Escalonamento

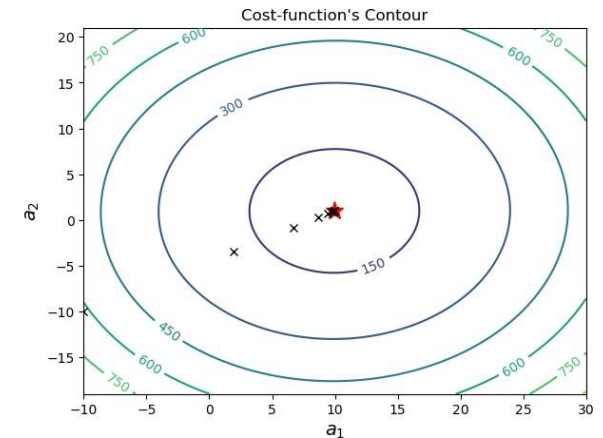
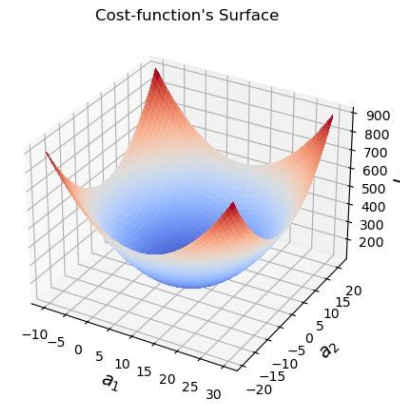
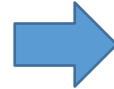


Vantagens do escalonamento de atributos

- Reduz a probabilidade de *problemas de precisão numérica*, mantendo a estabilidade do algoritmo durante o treinamento.
 - Por exemplo, *atributos com intervalos muito grandes podem gerar erros/gradientes extremamente grandes* que podem não ser representados pelas variáveis.



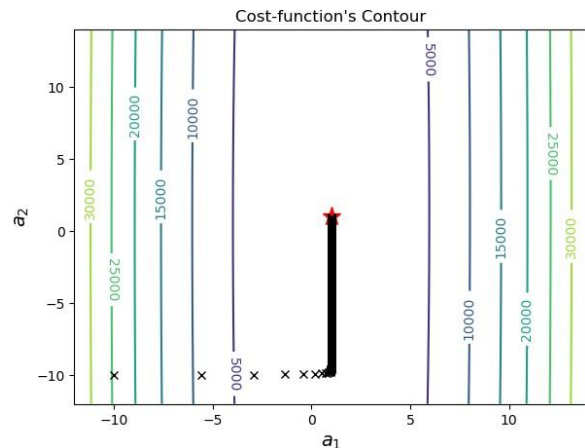
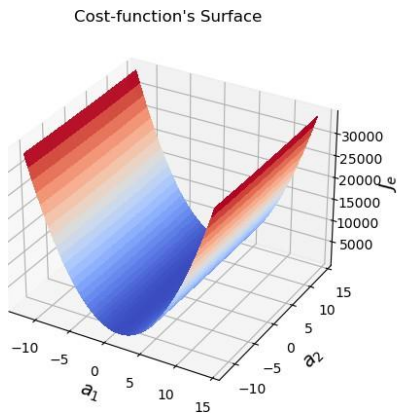
Escalonamento



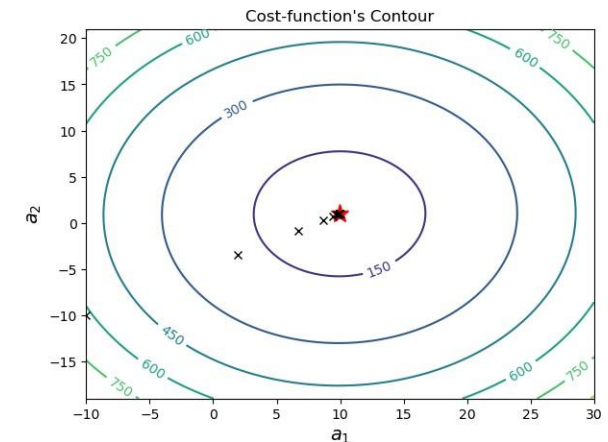
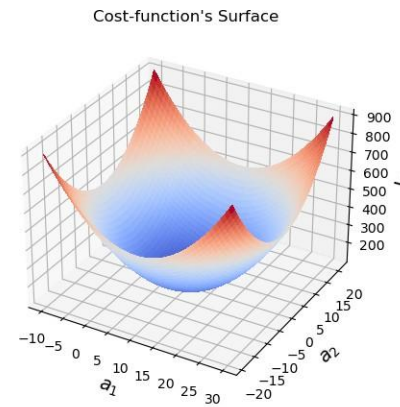
Vantagens do escalonamento de atributos

- Possibilita a **comparação justa do peso/influência** de cada **atributo** no modelo e até remover aqueles **sem muita importância**.
 - Pois os pesos representam o impacto relativo de cada atributo nas predições.

$$\hat{y}(n) = \hat{a}_0 + \hat{a}_1 x_1(n) + \hat{a}_2 x_2(n) + \dots + \hat{a}_K x_K(n)$$



Escalonamento

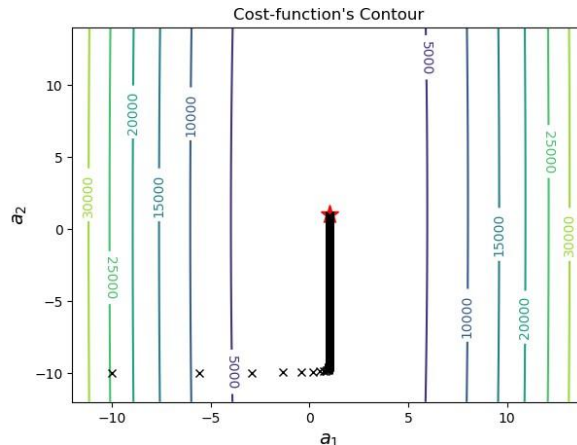
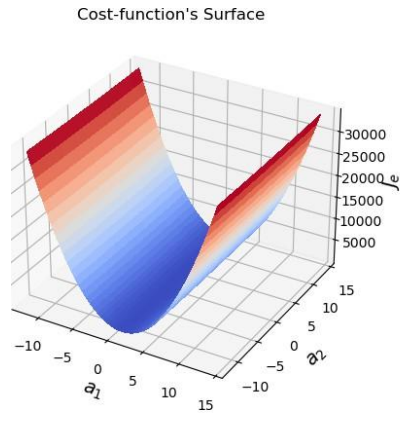


Vantagens do escalonamento de atributos

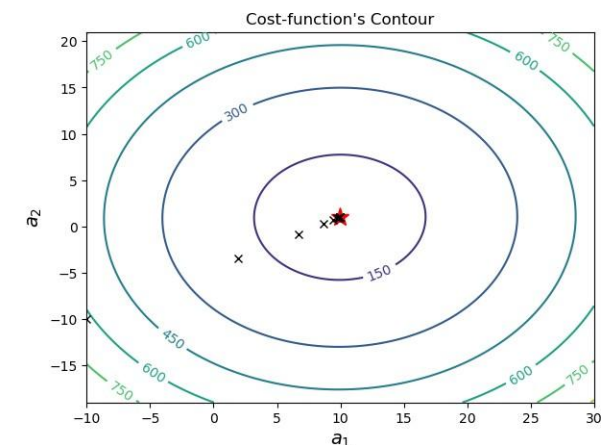
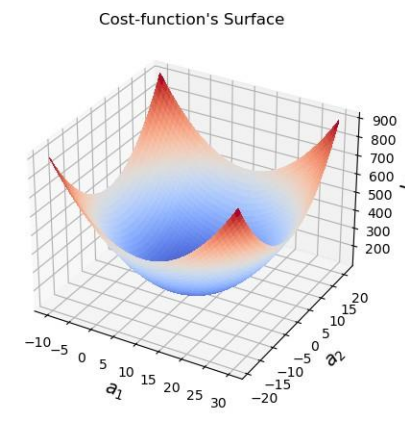
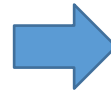
- Evita que **atributos com escalas muito diferentes dominem o modelo**.
 - Sem escalonamento, o modelo pode dar mais importância a atributos com intervalos maiores e menos importância aos atributos com intervalos menores.

$$J_e(\mathbf{a}) = \frac{1}{N} \sum_{n=0}^{N-1} \underbrace{\left[y_{\text{noisy}}(n) - (\hat{a}_1 x_1(n) + \hat{a}_2 x_2(n)) \right]^2}_{\text{Função Hipótese}} \approx \frac{1}{N} \sum_{n=0}^{N-1} \left[y_{\text{noisy}}(n) - \hat{a}_1 x_1(n) \right]^2$$

$x_1 \gg x_2$



Escalonamento



Escalonamento de atributos

Observações:

- O escalonamento é **aplicado antes do treinamento** do modelo, durante a **fase de pré-processamento**.
- Em geral, aplicamos o escalonamento **apenas aos atributos** e não aos rótulos, pois são os **atributos que influenciam o formato da superfície de erro**.
- Além disso, ao escalonar os rótulos, perde-se seu significado.
 - Por exemplo, a predição do preço de casas deixa de ser em reais.
- O **escalonamento (qualquer tipo) altera os valores dos pesos originais**.
 - Se a escala dos atributos é alterada, para que o modelo ainda prediga os mesmos valores de saída (i.e., rótulos), os pesos precisam ter seus valores alterados.

Escalonamento de atributos

Observações:

- O escalonamento pode ser usado com qualquer versão do GD.
- Quando temos *conjuntos de validação e teste* do modelo (conjuntos não utilizados para treinar o modelo, servindo para medir sua capacidade de generalização), a regra é aplicar os parâmetros de escalonamento (i.e., média, variância, valores mín. e máx.) obtidos durante o escalonamento do conjunto de treinamento a eles.
- Outras formas de pré-processamento podem ser encontradas em <https://scikit-learn.org/stable/modules/preprocessing.html>

Escalonamento de atributos

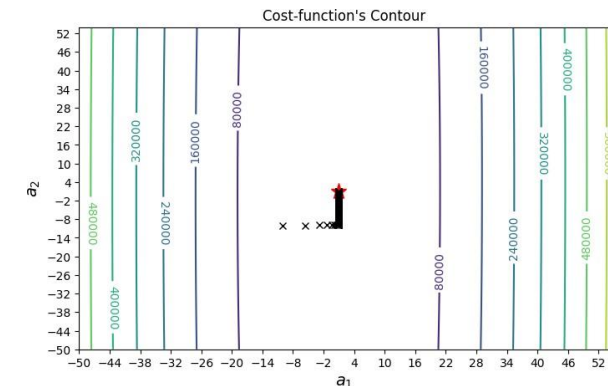
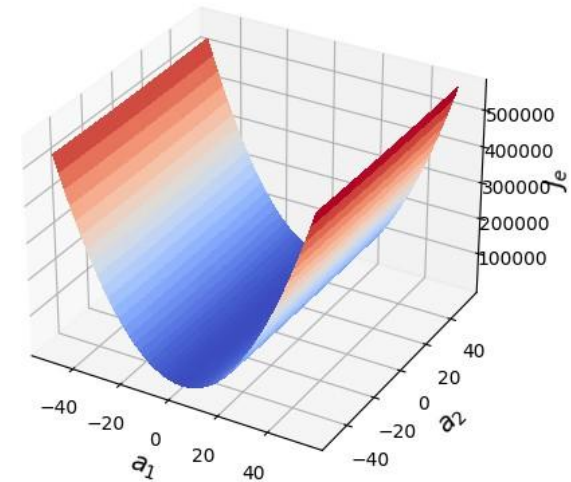
Observações:

- A **padronização não restringe o intervalo dos atributos**, assim, mesmo que eles tenham valores **discrepantes**, eles não serão afetados.
 - Ou seja, a **padronização** não mascara **outliers**.
- Já a **normalização os mascara**, pois força que eles fiquem dentro de um intervalo específico.
 - Isso torna sua identificação e remoção difíceis.
- Não existe uma regra rígida e direta para dizer quando **normalizar** ou **padronizar** os atributos.
- Em geral, é comum treinar o modelo com os dados brutos, normalizados e padronizados e, em seguida, comparar o desempenho para cada um dos casos.

Escalonamento de atributos: Exemplo

- Superfície de erro em forma de vale, com maior taxa de variação do erro na direção de a_1 .
- Taxa de variação do erro é praticamente constante na direção de a_2 (reta com inclinação de $\approx 0^\circ$).
- Pesos de atributos com variação muito grande são atualizados mais rapidamente do que pesos de atributos com variação pequena.
 - x_1 contribui muito mais no valor final do erro, fazendo com que a_1 seja rapidamente atualizado.
- Como o gradiente na direção de a_2 é muito pequeno, o treinamento fica lento.

Cost-function's Surface - without scaling

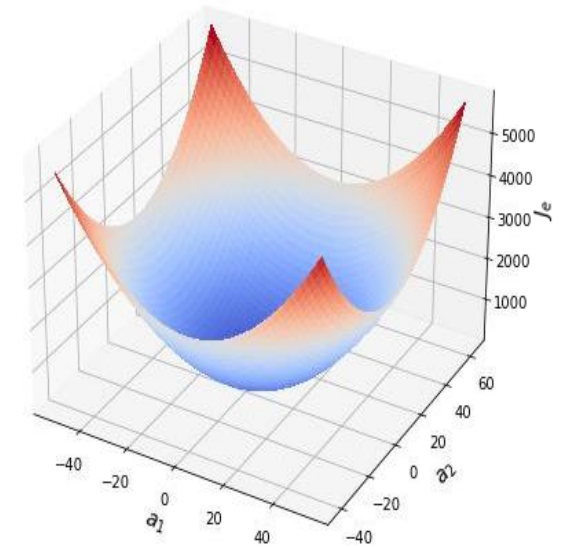


Escalonamento de atributos: Exemplo

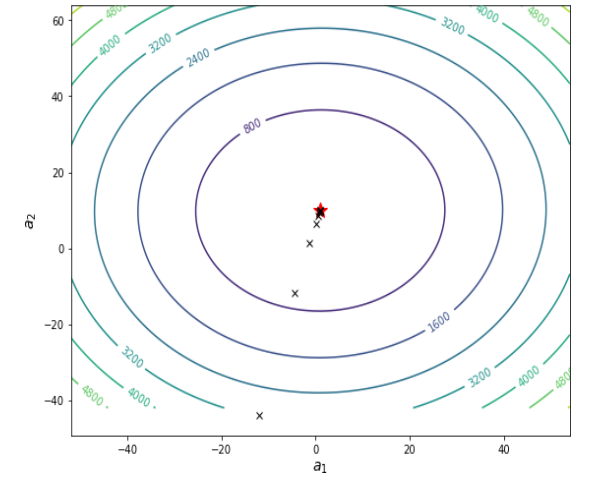
- Agora aplicamos **padronização** aos atributos.
- Após a padronização, a superfície se aproxima mais do formato de uma “*tigela*”.
- As linhas de contorno se tornam mais “circulares”, indicando que a superfície tem **inclinação similar em todas as direções** dado que os atributos agora têm intervalos similares.
- O treinamento se torna mais rápido, pois a inclinação da superfície se torna mais íngreme em todas as direções.

Padronização

Cost-function's Surface - with standardization



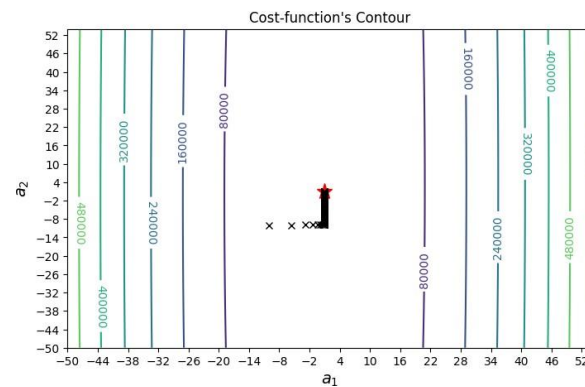
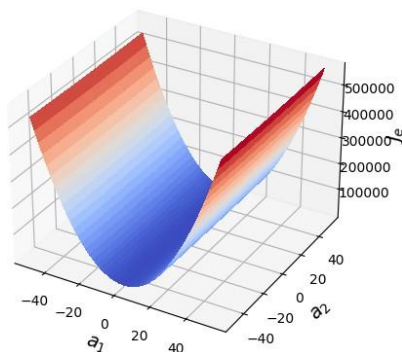
Cost-function's Contour



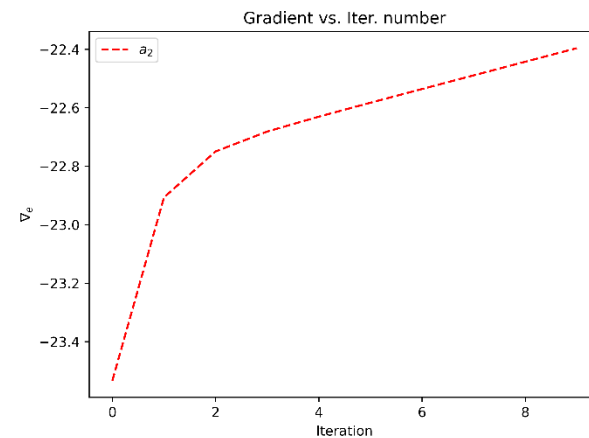
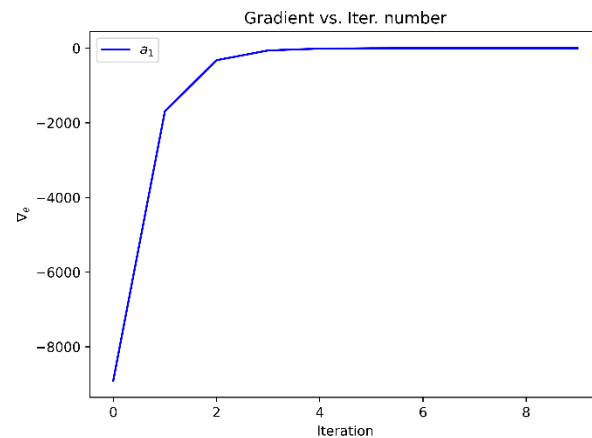
Escalonamento de atributos: Exemplo

Sem escalonamento

Cost-function's Surface - without scaling

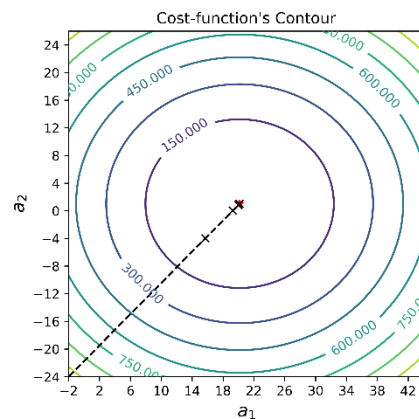
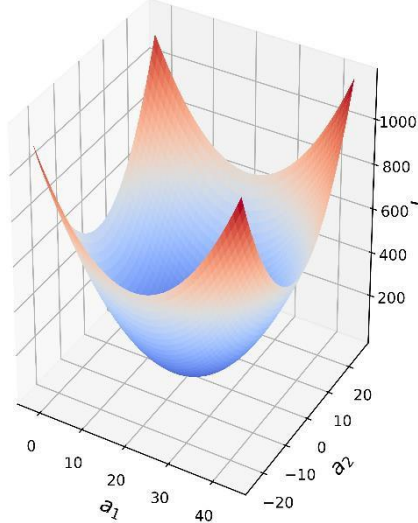


Derivadas parciais não variam de forma similar, pois a superfície não tem a mesma inclinação em ambas as direções.

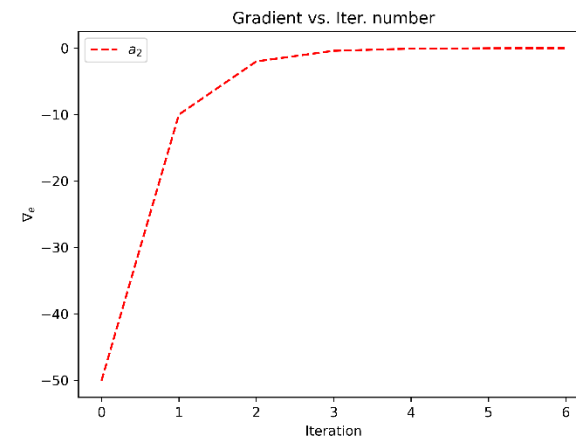
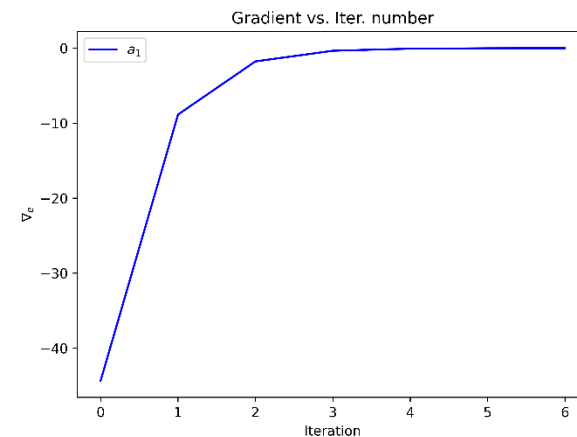


Padronização

Cost-function's Surface



Derivadas parciais variam de forma similar, pois a superfície tem a mesma inclinação em ambas as direções.



Pré-processamento dos dados

- O pré-processamento de dados é uma etapa importante em qualquer tarefa de aprendizado de máquina.
- Ele pode incluir várias técnicas de transformação dos dados para melhorar a qualidade e a eficácia dos modelos e de seu treinamento.
- Além do escalonamento, temos:
 - Detecção e remoção de ***outliers***.
 - Identificação de valores faltantes.
- Outras técnicas incluem:

Pré-processamento dos dados

- **Redução da dimensionalidade:** visa *diminuir o número de atributos* (i.e., as dimensões) em um conjunto de dados, *mantendo ao máximo a informação relevante*.
- **Transforma** os dados originais para um *novo espaço com menos dimensões*.
- Essa técnica ajuda a:
 - *Reduzir a complexidade do modelo*, tornando-os mais rápidos de treinar e mais fáceis de interpretar.
 - *Reduzir o sobreajuste*, pois com menos atributos, o modelo tem menos probabilidade de memorizar o conjunto de dados de treinamento, aumentando sua capacidade de generalização.
 - *Visualizar e interpretar os dados*, especialmente quando a redução é para duas ou três dimensões.

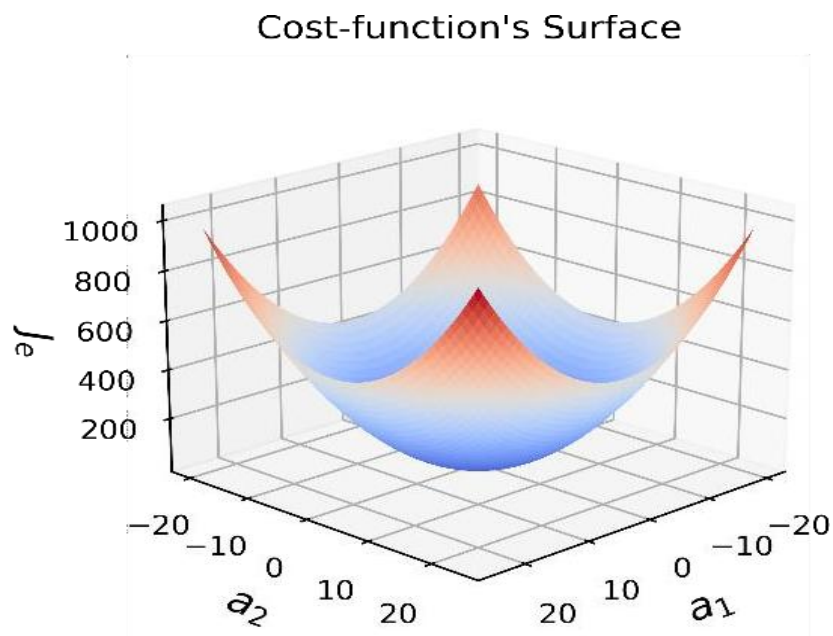
Pré-processamento dos dados

- **Aumento de dados:** visa a *geração de novos dados* a partir dos dados existentes.
- Aplica técnicas de transformação, como rotação, espelhamento, translação, *zoom*, corte, adição de ruído entre outras aos dados existentes.
- Pode-se usar *modelos generativos* como GANs, modelos de difusão, etc. para gerar dados sintéticos.
- Essa técnica é especialmente *útil* em situações onde o *conjunto de dados é limitado*, o que pode levar a problemas como sobreajuste e, consequente, baixa capacidade de generalização dos modelos.
 - Portanto, tem como vantagem a redução do sobreajuste.
- Técnica é muito utilizada em problemas de visão computacional.

Pré-processamento dos dados

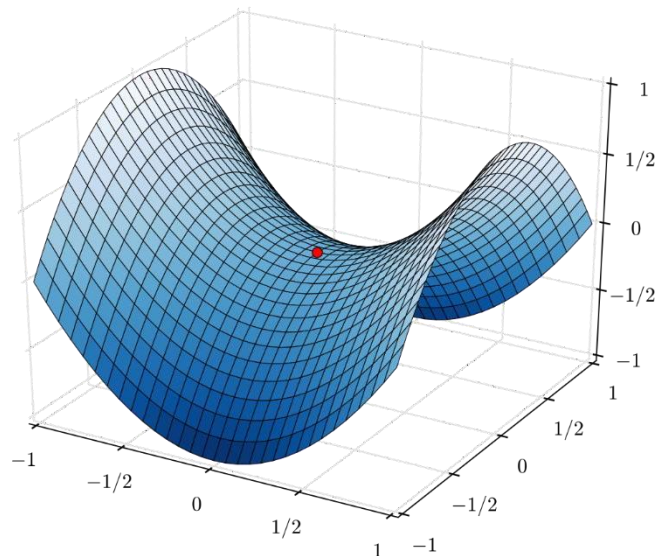
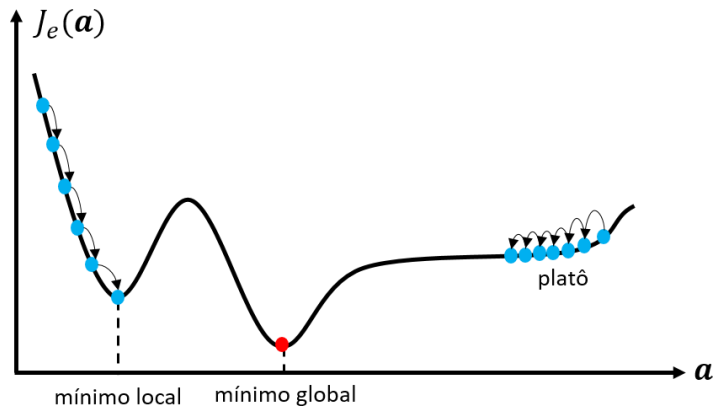
- **Seleção de atributos:** visa identificar e selecionar um *subconjunto dos atributos mais relevantes* de um conjunto de dados, com o objetivo de *reduzir a complexidade do modelo e melhorar seu desempenho*.
- *Elimina características redundantes ou irrelevantes*, o que, além das vantagens acima, *aumenta a interpretabilidade dos modelos e reduz o risco de sobreajuste*.
- Isso se deve ao fato de que nem todos os atributos em um conjunto de dados são igualmente importantes para a tarefa de aprendizado de máquina.

Desafios encontrados pelo GD



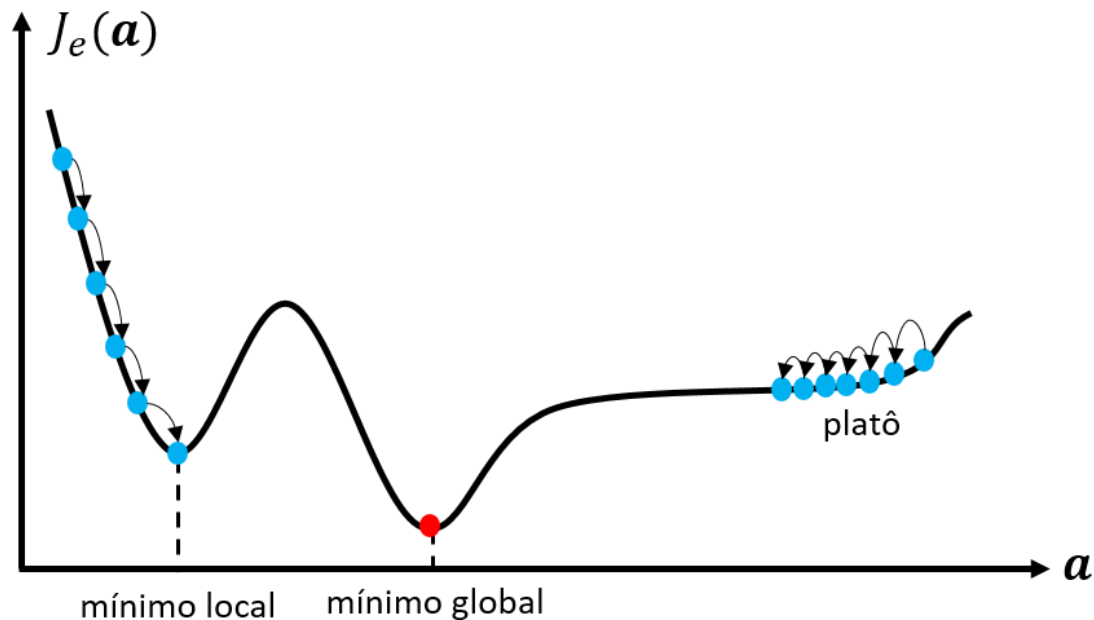
- Como vimos, as superfícies de ***funções de erro*** que utilizam o ***erro quadrático médio*** para treinamento de modelos de regressão linear são sempre ***convexas***, se assemelhando a um vale ou a uma tigela.
 - **Implicações:** ***não existem mínimos locais***, apenas um mínimo global. São ***superfícies contínuas*** com inclinação que nunca muda abruptamente.
 - **Consequência:** o ***gradiente descendente*** ***garantidamente converge*** para o mínimo global (dado que se espere tempo suficiente e que o ***passo de aprendizagem*** não seja grande demais).
- Porém, nem todas as ***superfícies de erro*** são ***convexas***.

Desafios encontrados pelo GD



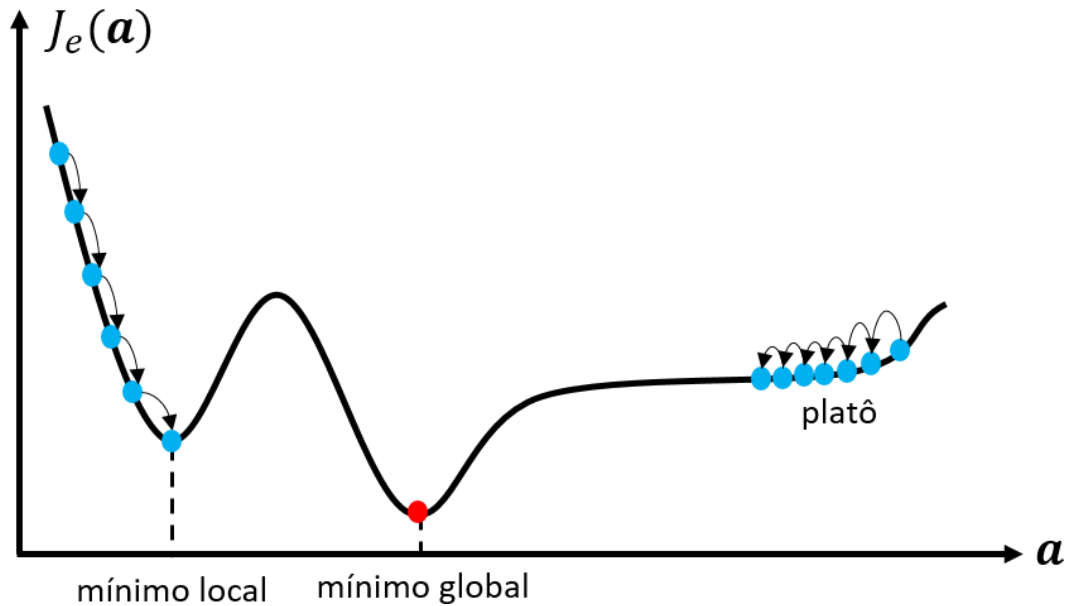
- Algumas podem ter vários mínimos locais, platôs, pontos de sela, e todo tipo de “**irregularidade**”, dificultando a convergência para o mínimo global.
 - **Platô**: região elevada e plana, i.e., erro alto e pequena inclinação.
 - **Ponto de sela**: ponto de uma função onde a inclinação muda de direção de forma que o ponto é um mínimo em uma direção e um máximo em outra.
- Redes neurais são exemplos de modelos com **funções de erro não-convexas** e, conseqüentemente, bastante irregulares.

Desafios encontrados pelo GD



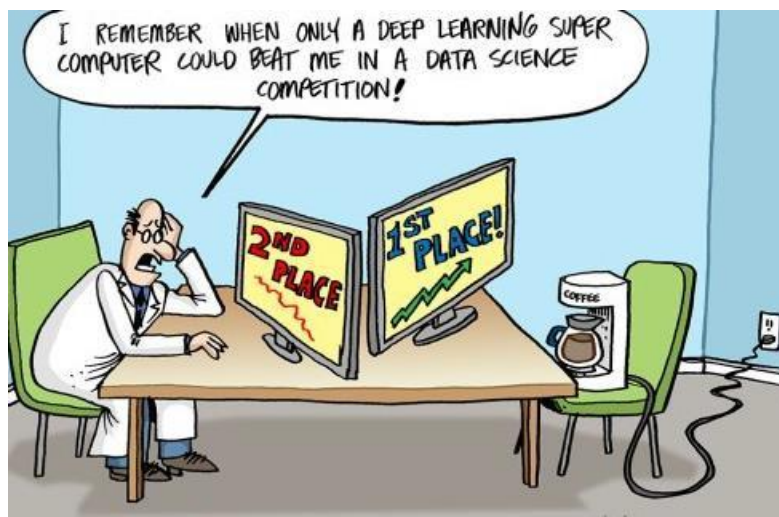
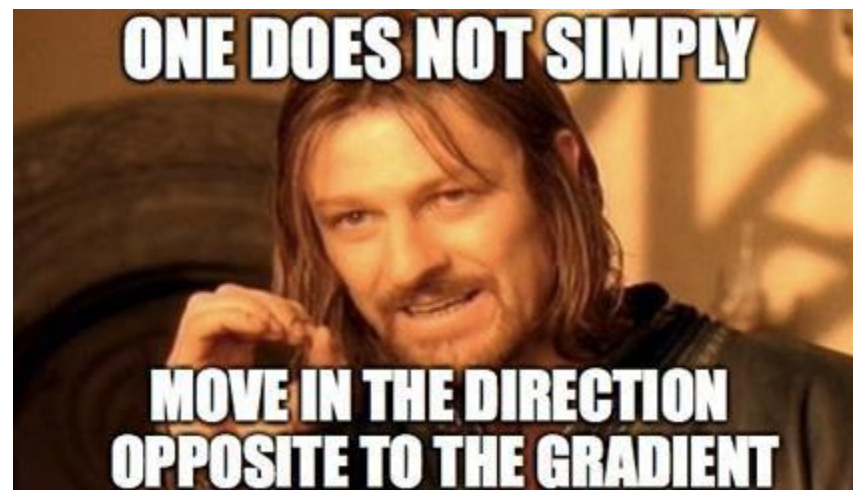
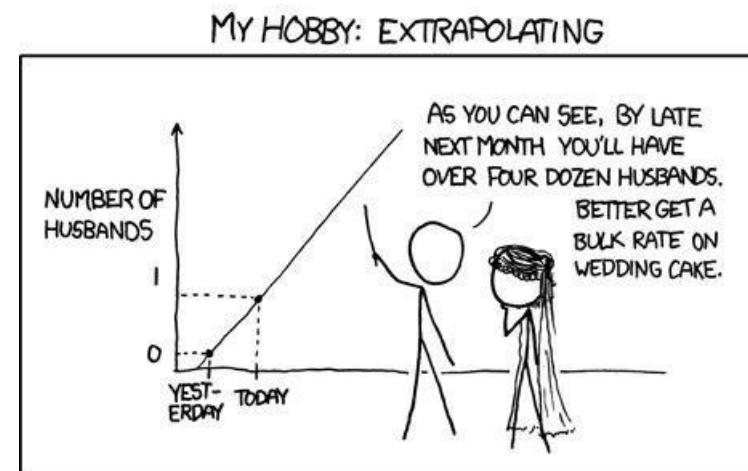
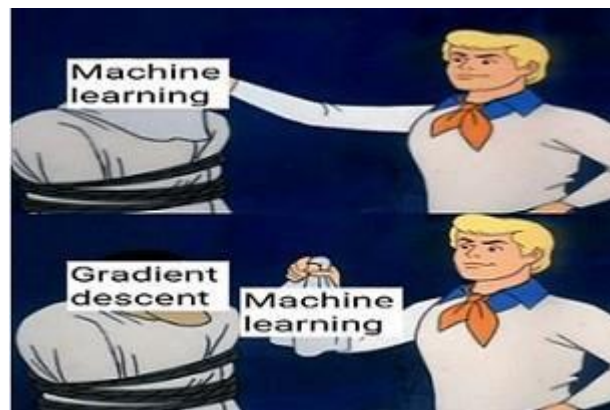
- A figura mostra dois grandes desafios encontrados pelo **algoritmo do GD em superfícies irregulares**.
- Se os pesos forem inicializados no ponto à
 - **esquerda**, o algoritmo **convergirá para um mínimo local**, que não é tão bom quanto o mínimo global.
 - **direita**, levará muito tempo para atravessar o platô, pois o gradiente é próximo de zero, devido à pequena inclinação.

Desafios encontrados pelo GD



- Como garantir que o mínimo encontrado é o global e não um mínimo local?
 - Treinar o modelo várias vezes, sempre *inicializando os pesos aleatoriamente*, com a esperança de que em alguma dessas vezes ele inicie mais próximo do mínimo global.
 - Uma boa *inicialização dos pesos* pode ajudar a evitar mínimos locais e pontos de sela.
 - *Otimizadores como Adam, RMSprop e SGD com momentum* podem navegar melhor por superfícies irregulares.

Obrigado!



Anexo I:

Cálculo do vetor gradiente

Cálculo do vetor gradiente

Vamos considerar o hiperplano como a função hipótese

$$\hat{y}(n) = \mathbf{a}^T \mathbf{x}(n).$$

O vetor gradiente é calculado como

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \left[\frac{\partial J_e(\mathbf{a})}{\partial a_0} \quad \cdots \quad \frac{\partial J_e(\mathbf{a})}{\partial a_K} \right]^T$$

Assim, o vetor gradiente da função de erro em relação aos pesos é dado por

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \frac{\partial}{\partial \mathbf{a}} \left[\frac{1}{N} \sum_{n=0}^{N-1} (y(n) - \hat{y}(n))^2 \right].$$

Cálculo do vetor gradiente

Como a operação de derivada é distributiva, podemos reescrever a equação acima como

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \frac{1}{N} \sum_{n=0}^{N-1} \frac{\partial (y(n) - \hat{y}(n))^2}{\partial \mathbf{a}}.$$

Substituindo a função hipótese na equação acima, temos

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \frac{1}{N} \sum_{n=0}^{N-1} \frac{\partial (y(n) - \mathbf{a}^T \mathbf{x}(n))^2}{\partial \mathbf{a}}.$$

Cálculo do vetor gradiente

Aplicando a regra da cadeia, reescrevemos a equação anterior como

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{2}{N} \sum_{n=0}^{N-1} (y(n) - \mathbf{a}^T \mathbf{x}(n)) \frac{\partial \mathbf{a}^T \mathbf{x}(n)}{\partial \mathbf{a}}.$$

Sabendo que a derivada de $\frac{\partial \mathbf{a}^T \mathbf{x}(n)}{\partial \mathbf{a}}$ é igual a $\mathbf{x}(n)$, reescrevemos a equação anterior como

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{2}{N} \sum_{n=0}^{N-1} [y(n) - \hat{y}(n)] \mathbf{x}(n).$$

Cálculo do vetor gradiente

Fazendo $y(n) - \hat{y}(n) = d(n)$

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{2}{N} \sum_{n=0}^{N-1} [y(n) - \hat{y}(n)] \mathbf{x}(n).$$

$$= -\frac{2}{N} \left\{ d(0) \begin{bmatrix} x_0(0) \\ \vdots \\ x_K(0) \end{bmatrix} + d(1) \begin{bmatrix} x_0(1) \\ \vdots \\ x_K(1) \end{bmatrix} + \cdots + d(N-1) \begin{bmatrix} x_0(N-1) \\ \vdots \\ x_K(N-1) \end{bmatrix} \right\}$$

$$= -\frac{2}{N} \left\{ \begin{bmatrix} d(0)x_0(0) + d(1)x_0(1) + \cdots + d(N-1)x_0(N-1) \\ \vdots \\ d(0)x_K(0) + d(1)x_K(1) + \cdots + d(N-1)x_K(N-1) \end{bmatrix} \right\}$$

Notem que a equação acima é um ***vetor coluna*** com dimensão $K + 1 \times 1$.

Cálculo do vetor gradiente

Podemos reescrever a equação (i.e., vetor) anterior como uma multiplicação matricial

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{2}{N} \begin{bmatrix} x_0(0) & x_0(1) & \dots & x_0(N-1) \\ \vdots & \vdots & \ddots & \vdots \\ x_K(0) & x_K(1) & \dots & x_K(N-1) \end{bmatrix} \begin{bmatrix} d(0) \\ d(1) \\ \vdots \\ d(N-1) \end{bmatrix}$$

Percebam que temos a multiplicação de uma matriz com dimensão $K + 1 \times N$ por um vetor coluna de dimensão $N \times 1$.

A matriz contém em cada linha todos os valores de $n = 0$ a $n = N - 1$ de um **único** atributo.

O vetor contém em cada linha a diferença $d(n) = y(n) - \hat{y}(n)$ para $n = 0$ até $n = N - 1$

Cálculo do vetor gradiente

Se definirmos uma matriz que contém todos os N exemplos de todos os $K + 1$ atributos e que tem dimensão $N \times K + 1$

$$\mathbf{X} = \begin{bmatrix} x_0(0) & \dots & x_K(0) \\ x_0(1) & \dots & x_K(1) \\ \vdots & \ddots & \vdots \\ x_0(N-1) & \dots & x_K(N-1) \end{bmatrix}$$

e dois vetores coluna com dimensões $N \times 1$ contendo todos os valores esperados (i.e., rótulos) e todos os valores preditos

$$\mathbf{y} = \begin{bmatrix} y(0) \\ \vdots \\ y(N-1) \end{bmatrix} \quad \text{e} \quad \hat{\mathbf{y}} = \begin{bmatrix} \hat{y}(0) \\ \vdots \\ \hat{y}(N-1) \end{bmatrix}$$

Cálculo do vetor gradiente

Usando a matriz e os vetores definidos no slide anterior, podemos reescrever o vetor gradiente como

$$\begin{aligned} & \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} \\ &= -\frac{2}{N} \begin{bmatrix} x_0(0) & x_0(1) & \dots & x_0(N-1) \\ \vdots & \vdots & \dots & \vdots \\ x_K(0) & x_K(1) & \dots & x_K(N-1) \end{bmatrix} \left(\begin{bmatrix} y(0) \\ y(1) \\ \vdots \\ y(N-1) \end{bmatrix} - \begin{bmatrix} \hat{y}(0) \\ \hat{y}(1) \\ \vdots \\ \hat{y}(N-1) \end{bmatrix} \right) \\ &= -\frac{2}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}) \end{aligned}$$

O resultado da multiplicação matricial acima continuando resultando em um **vetor coluna** com dimensão $K + 1 \times 1$, ou seja, $(K + 1 \times N) \times (N \times 1) = K + 1 \times 1$.

Equação de atualização dos pesos

Utilizando o resultado anterior, podemos reescrever a equação de atualização dos pesos como

$$\begin{aligned}\mathbf{a} &= \mathbf{a} - \alpha \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} \\ &= \mathbf{a} + \alpha \frac{2}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}).\end{aligned}$$

A soma acima deve resultar em um vetor coluna com dimensão $K + 1 \times 1$, pois esta é a dimensão dos dois vetores sendo somados.

Lembrem-se que $K + 1 \times 1$ é a dimensão do vetor $\mathbf{a}$, o qual contém todos os pesos do modelo e que $\frac{2}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}})$ tem dimensão $K + 1 \times 1$ também

Equação de atualização dos pesos

Podemos reescrever a equação de atualização dos pesos como

$$\begin{aligned}\mathbf{a} &= \mathbf{a} - \alpha \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \begin{bmatrix} a_0 \\ \vdots \\ a_K \end{bmatrix} - \alpha \begin{bmatrix} \frac{\partial J_e(\mathbf{a})}{\partial a_0} \\ \vdots \\ \frac{\partial J_e(\mathbf{a})}{\partial a_K} \end{bmatrix} \\ &= \begin{bmatrix} a_0 \\ \vdots \\ a_K \end{bmatrix} + \alpha \frac{2}{N} \begin{bmatrix} x_0(0) & x_0(1) & \dots & x_0(N-1) \\ \vdots & \vdots & \vdots & \vdots \\ x_K(0) & x_K(1) & \dots & x_K(N-1) \end{bmatrix} \left(\begin{bmatrix} y(0) \\ y(1) \\ \vdots \\ y(N-1) \end{bmatrix} - \begin{bmatrix} \hat{y}(0) \\ \hat{y}(1) \\ \vdots \\ \hat{y}(N-1) \end{bmatrix} \right) \\ &= \begin{bmatrix} a_0 \\ \vdots \\ a_K \end{bmatrix} + \alpha \frac{2}{N} \left\{ \begin{bmatrix} d(0)x_0(0) + d(1)x_0(1) + \dots + d(N-1)x_0(N-1) \\ \vdots \\ d(0)x_K(0) + d(1)x_K(1) + \dots + d(N-1)x_K(N-1) \end{bmatrix} \right\}\end{aligned}$$

Anexo II:

Cálculo do vetor gradiente de
uma função hipótese com 2
pesos

Cálculo do vetor gradiente

Função hipótese com 2 pesos, a_1 e a_2

$$\hat{y}(n) = h(\mathbf{x}(n)) = a_1 x_1(n) + a_2 x_2(n)$$

A função de erro é dada por

$$J_e(\mathbf{a}) = \frac{1}{N} \sum_{n=0}^{N-1} [y(n) - (a_1 x_1(n) + a_2 x_2(n))]^2$$

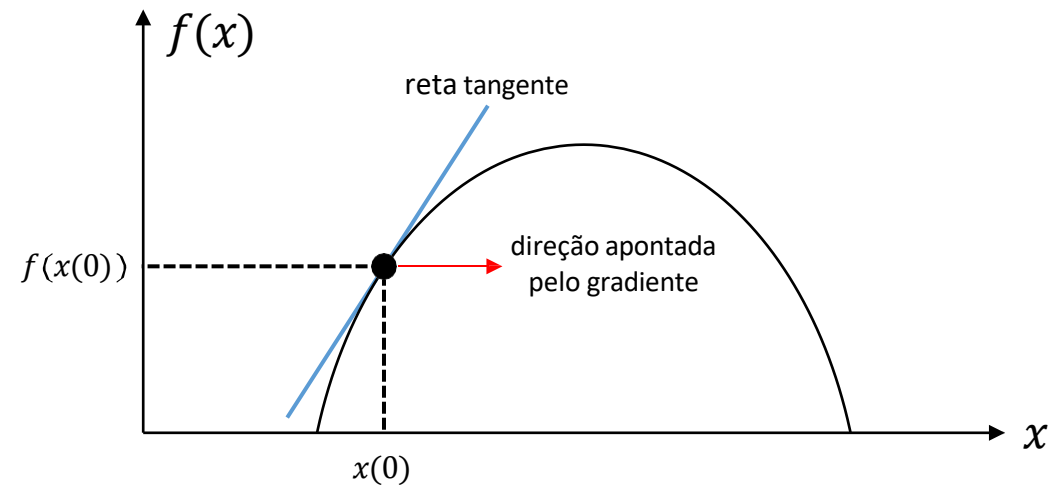
Cálculo do vetor gradiente

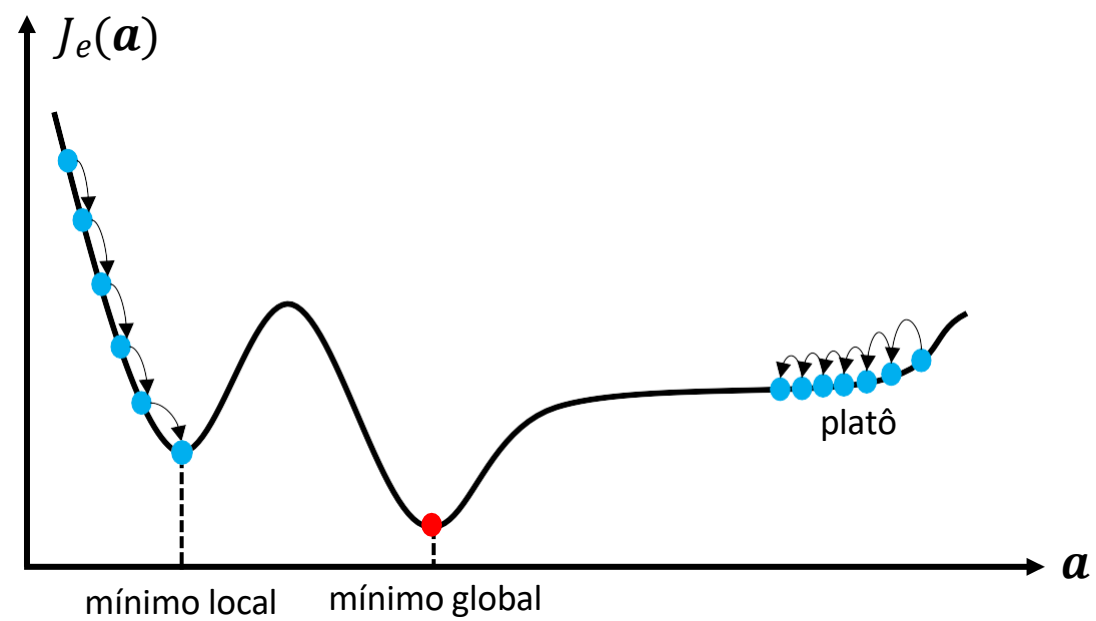
Cada elemento do vetor gradiente é dado por

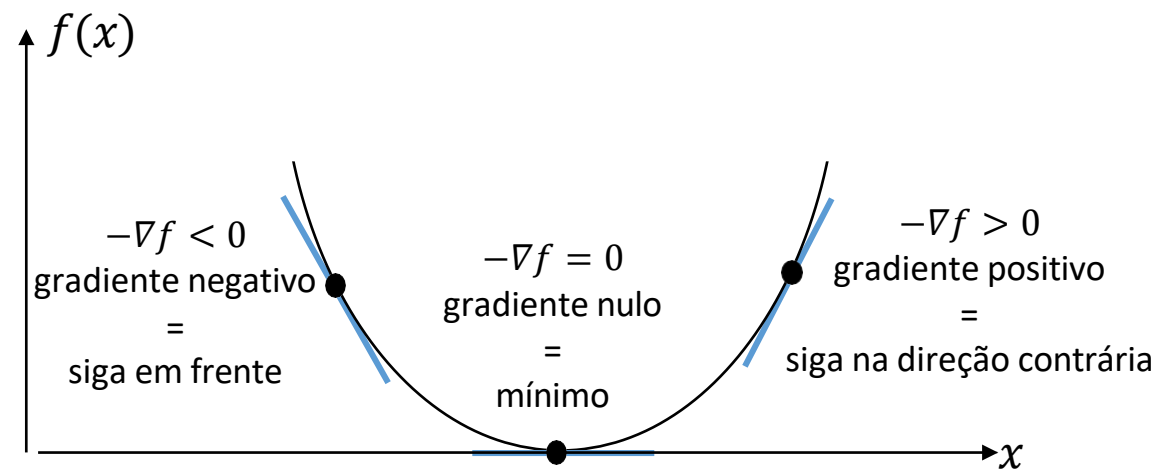
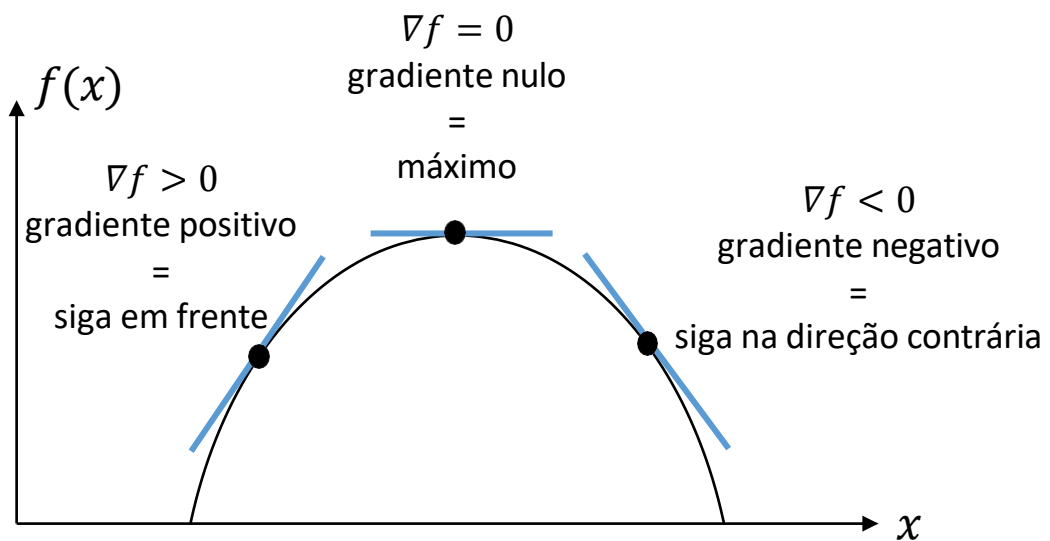
$$\begin{aligned}\frac{\partial J_e(\mathbf{a})}{\partial a_K} &= \frac{\partial \frac{1}{N} \sum_{n=0}^{N-1} [y(n) - (a_1 x_1(n) + a_2 x_2(n))]^2}{\partial a_K} \\ &= \frac{1}{N} \sum_{n=0}^{N-1} \frac{\partial [y(n) - (a_1 x_1(n) + a_2 x_2(n))]^2}{\partial a_K} \\ &= -\frac{2}{N} \sum_{n=0}^{N-1} [y(n) - (a_1 x_1(n) + a_2 x_2(n))] x_K(n), \quad k = 1, 2\end{aligned}$$

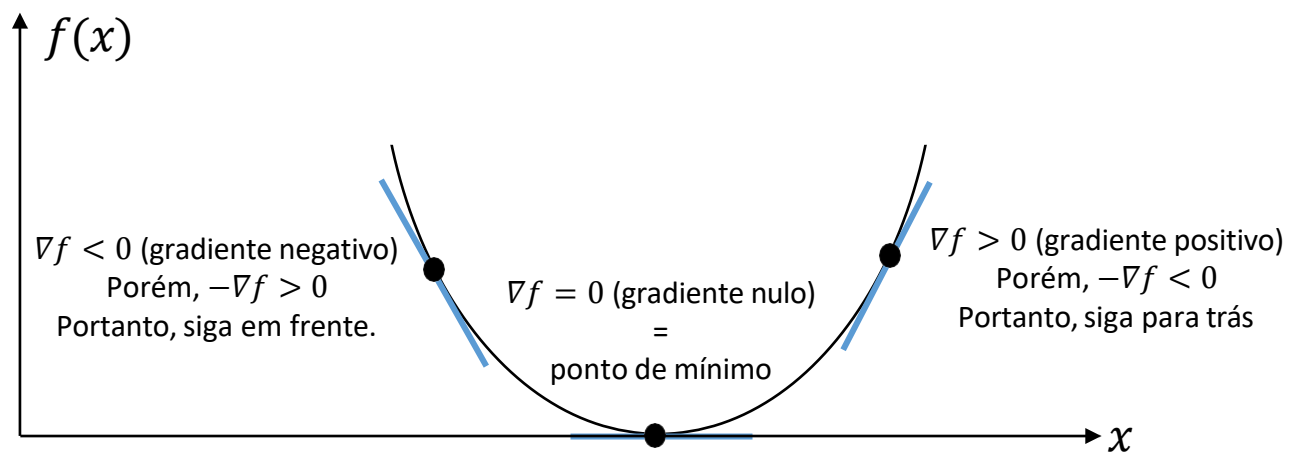
Operação da derivada parcial é distributiva.

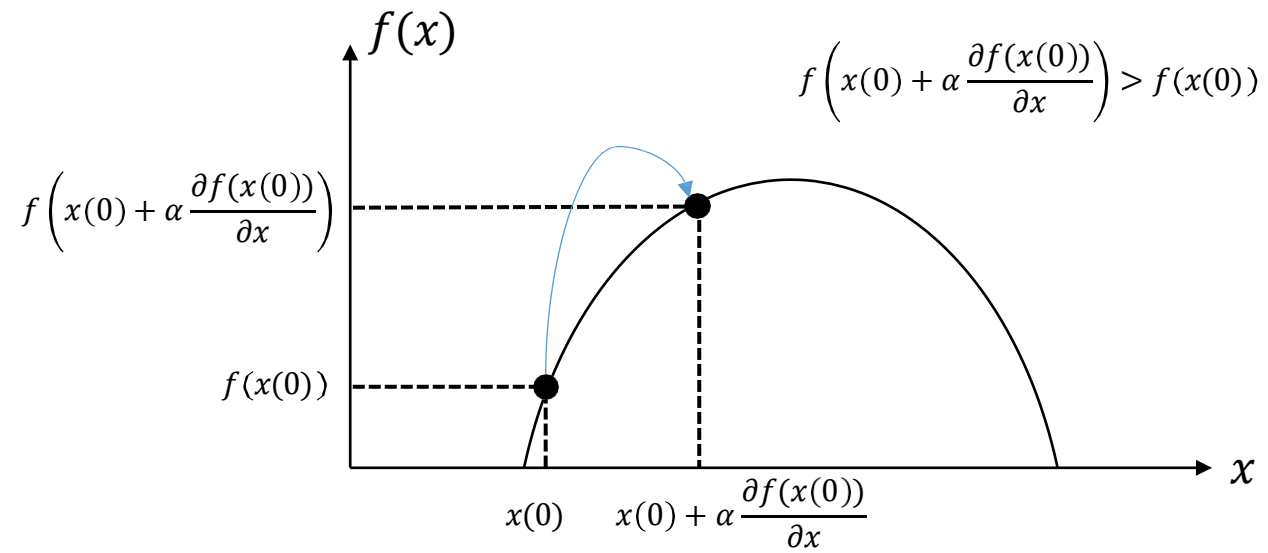
FIGURAS

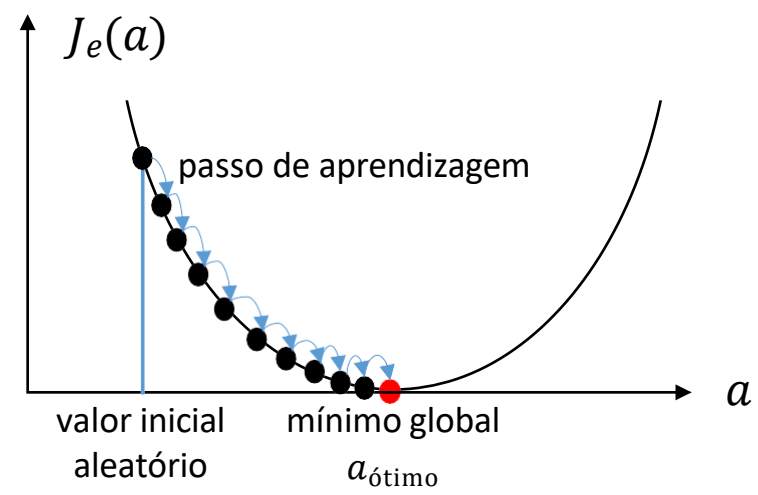


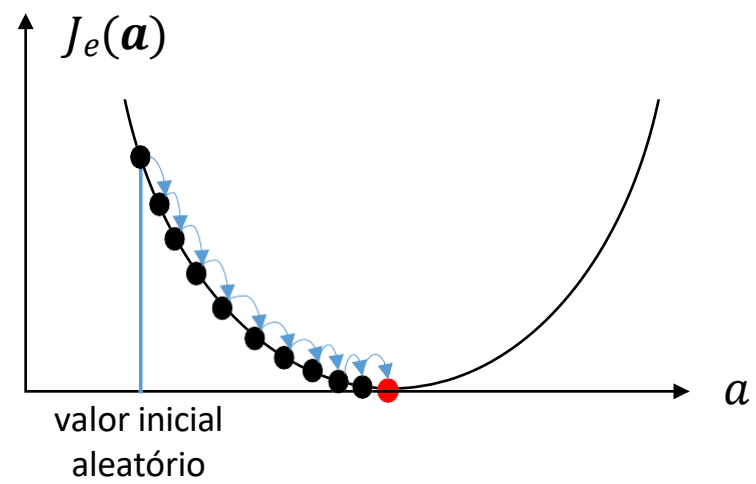
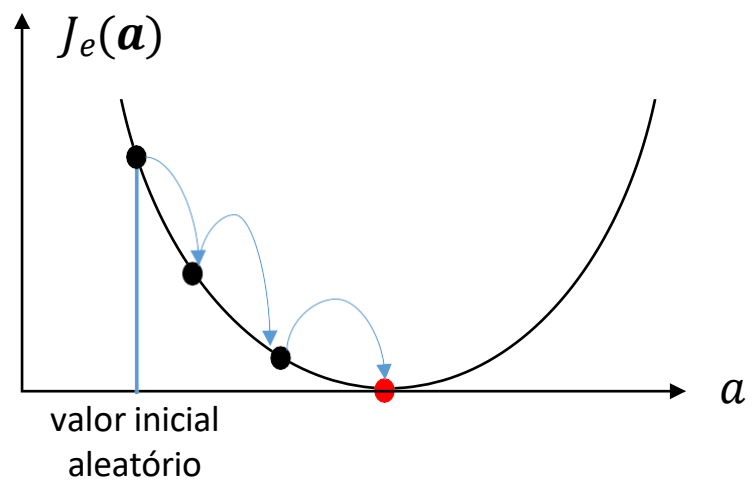


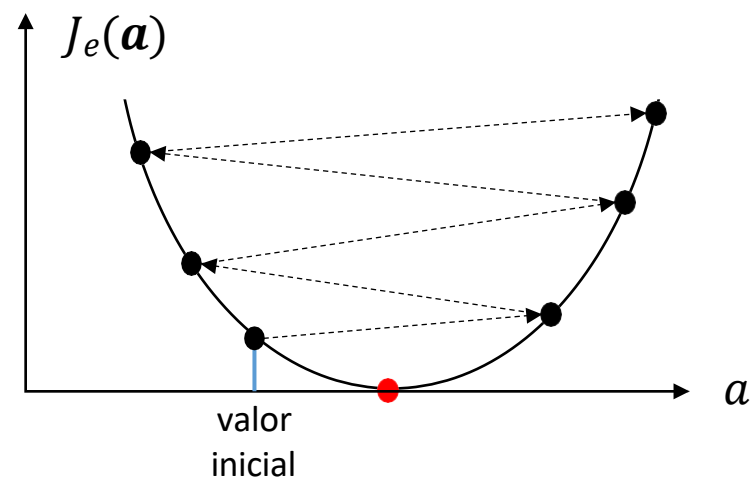
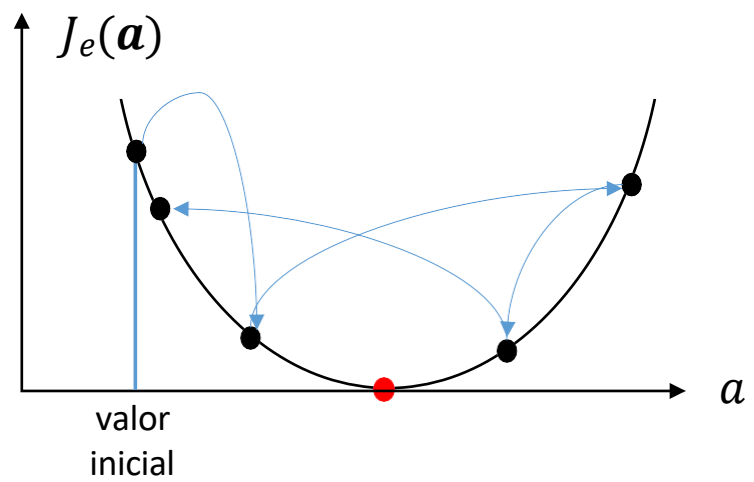


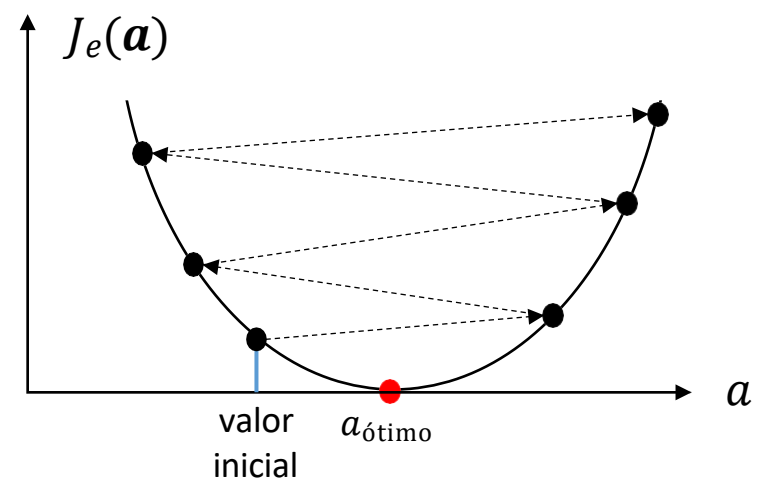


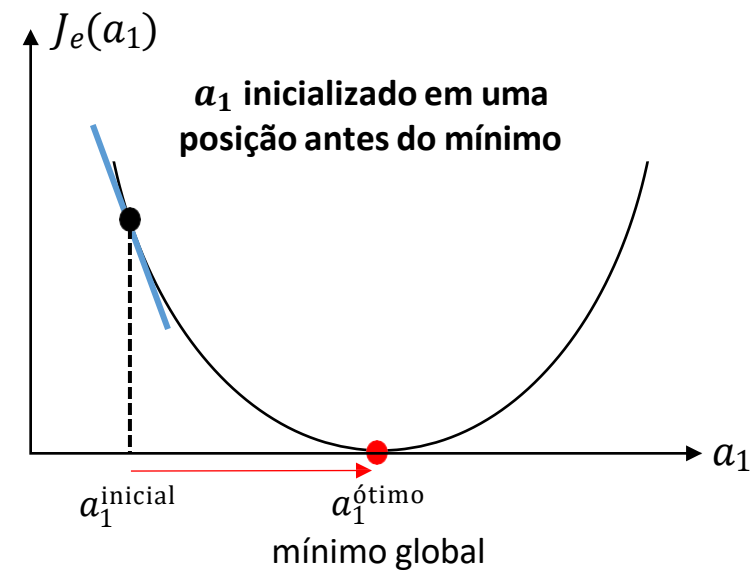




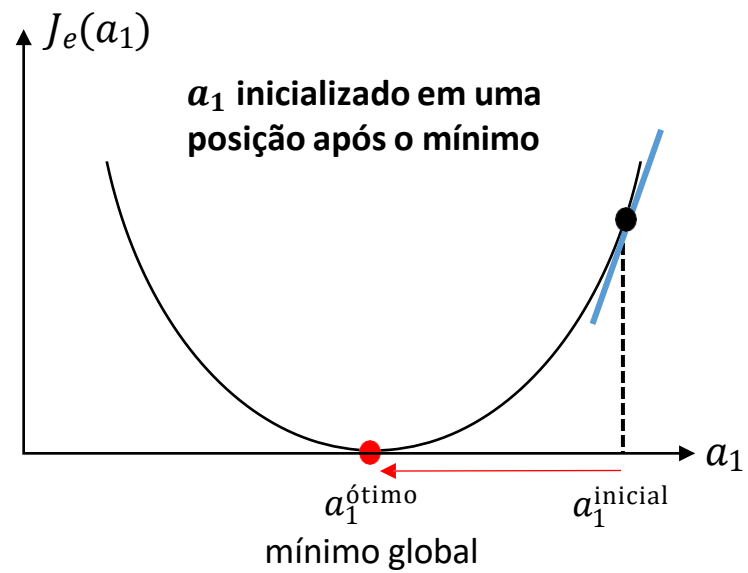
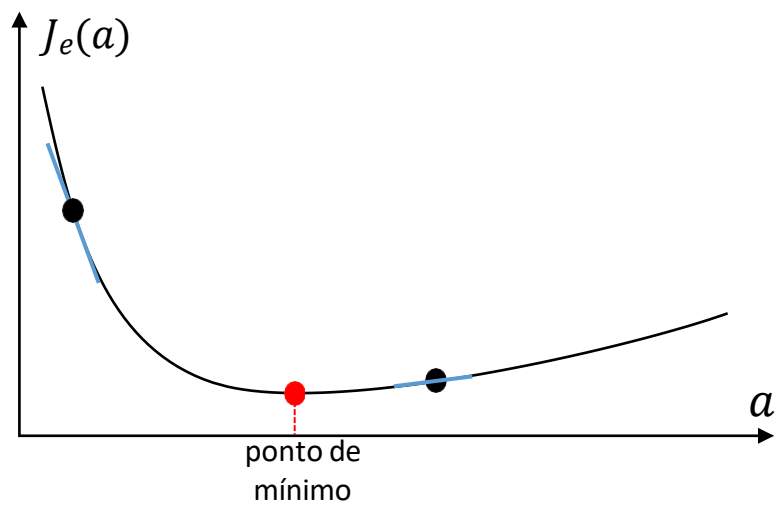








gradiente negativo: $a_1 = a_1^{\text{inicial}} + \alpha \nabla J_e(a_1)$
 a_1 aumenta e se aproxima do mínimo



gradiente positivo: $a_1 = a_1^{\text{inicial}} - \alpha \nabla J_e(a_1)$
 a_1 diminuiu e se aproxima do mínimo

